



Vivekanand Education Society's

Institute of Technology

An Autonomous Institute Affiliated to University of Mumbai

Hashu Advani Memorial Complex, Collector Colony, Chembur East, Mumbai - 400074.

Department of Information Technology

CERTIFICATE

This is to certify that **Rujuta Vinit Medhi** of **D20A** semester **VIII**, have successfully completed necessary experiments in the **ITC801: Blockchain & DLT Lab** under my supervision in **VES Institute of Technology** during the academic year **2025-2026**.

Subject Teacher

Name: Mrs. Kajal Joseph

Signature:

Head of Department

Name: Dr. Mrs. Shalu Chopra

Signature:

Lab Teacher

Name: Kajal Joseph

Signature:



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26

Lab Code	Lab Name	Credit
ITC801	Blockchain and DLT Lab	1

Programme Outcomes: The graduate will be able to:

PO1	Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.
PO2	Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)
PO3	Design/Development of Solutions: Design creative solutions for complex engineering problems and design / develop systems /components / processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)
PO4	PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).
PO5	Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)
PO6	The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).
PO7	Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)
PO8	Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
PO9	Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26

PO10	Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.
PO11	Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.
PSO 1	Computing & Emerging Technology: Apply core computing and modern technologies, such as AI, Cloud Computing, IoT, cybersecurity, and data-driven tools, to build efficient and innovative solutions.
PSO 2	Professionalism & Innovation : Demonstrate ethics, teamwork, and an entrepreneurial mindset to deliver sustainable solutions for society.

Course Outcomes: Student will be able	
ITC801.1	Describe the basic concept of Blockchain and Distributed Ledger Technology.
ITC801.2	Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions
ITC801.3	Implement smart contracts in Ethereum using different development frameworks.
ITC801.4	Develop applications in permissioned Hyperledger Fabric network.
ITC801.5	Interpret different Crypto assets and Crypto currencies
ITC801.6	The use of Blockchain with AI, IoT and Cyber Security using case studies.

ITC801 Blockchain and DLT Lab													
CO	PO											PSO	
	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2
ITC801.1	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.2	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.3	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.4	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.5	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.6	2	3	3	3	3	3	3	3	3	2	2	3	3
Avg PO AL	2	3	3	3	3	3	3	3	3	2	2	3	3



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26

ITC801 Blockchain and DLT Lab												
CO	Experiment											
	Exp 1	Exp 2	Exp 3	Exp 4	Exp 5	Exp 6	Exp 7	Exp 8	Exp 9	Exp 10	Exp 11	Exp 12
ITC801.1	3	3	2	-	2	2	2	-	-	3	-	3
ITC801.2	2	3	3	-	2	2	3	2	-	3	-	3
ITC801.3	-	-	2	3	3	3	3	3	2	-	-	3
ITC801.4	-	-	-	-	-	-	-	-	-	-	3	3
ITC801.5	2	3	3	-	-	2	2	2	-	3	-	3
ITC801.6	-	-	-	-	-	-	-	-	-	2	-	3

INDEX

Sr. No.	List of Experiments
1	Write a Python program to understand SHA and Cryptography in Blockchain, Merkle root tree hash
2	Create a Blockchain using Python
3	Create a Cryptocurrency using Python and perform mining in the Blockchain created.
4	Hands-on Solidity Programming Assignments for creating Smart Contracts
5	Deploying a Voting/Ballot Smart Contract
6	Creating Smart Contracts and performing transactions using Solidity and Remix IDE
7	Implement the embedding wallet (Metamask) and transaction using Solidity
8	To implement the Blockchain platform Ganache and deploy a simple Solidity smart contract using Metamask and Remix IDE.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26

9	To implement a Private Ethereum Blockchain using Geth.
10	Explore the Cryptocurrency Landscape
11	Implementation of Hyperledger Fabric Network and Asset Management using Chaincode
12	Mini Project
12	Assignment-1
13	Assignment-2
14	Assignment-3
15	Mock Viva

Subject teacher



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment No. 01

Aim: Write a Python program to understand SHA and Cryptography in Blockchain, Merkle root tree hash

Roll No.	33
Name	Rujuta Vinit Medhi
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO5: Interpret different Crypto assets and Crypto currencies

Aim: Cryptographic hash functions in blockchain, merkle tree

Theory:

Q1. Cryptographic Hash Functions in Blockchain

A cryptographic hash function is a mathematical algorithm that takes an input of any size and produces a fixed-length output, called a hash value or digest. In blockchain technology, cryptographic hash functions are fundamental for ensuring data integrity, security, immutability, and trust in a decentralized environment.

Common hash functions used in blockchains include SHA-256 (Bitcoin), Keccak-256 / SHA-3 (Ethereum), and RIPEMD-160.

Q2. Characteristics of Cryptographic Hash Functions

A secure cryptographic hash function must satisfy the following properties:

1. Deterministic

- The same input always produces the same hash.
- Ensures consistency across all nodes in the blockchain.

2. Fixed Output Length

- Regardless of input size, the output hash has a fixed length.
- Example: SHA-256 always produces a 256-bit hash.

3. Pre-image Resistance

- Given a hash, it is computationally infeasible to find the original input.
- Protects transaction data from reverse engineering.

4. Second Pre-image Resistance

- Given an input and its hash, it is infeasible to find another input with the same hash.
- Prevents data substitution attacks.

5. Collision Resistance

- It is extremely difficult to find two different inputs that produce the same hash.
- Ensures uniqueness of transaction identifiers.

6. Avalanche Effect

- A small change in input results in a completely different hash.
Helps detect even minor data tampering.

Q3. Role of Cryptographic Hash Functions in Blockchain

3.1 Transaction Hashing

- Each transaction is hashed to create a transaction ID (TXID).
- Any modification to a transaction changes its hash.

3.2 Block Hashing

- Each block contains:
 - Transaction data
 - Previous block hash
 - Timestamp
 - Nonce
- All these fields are hashed to generate a block hash, linking blocks together.
This creates a chain of blocks, ensuring immutability.

Q4. Hash Functions in Proof of Work (PoW)

In Proof of Work:

Miners repeatedly hash block data with different nonce values.

The goal is to find a hash that satisfies a difficulty target (e.g., leading zeros).

Example:

$\text{Hash}(\text{Block Data} + \text{Nonce}) < \text{Target}$

This process:

- Requires computational effort
Prevents spam and malicious attacks
- Secures the blockchain against tampering

Q5. Hash Functions in Merkle Trees

- Transactions are hashed and organized into a Merkle Tree.
- The final hash (Merkle Root) summarizes all transactions.
Only the Merkle Root is stored in the block header.

Benefits:

- Efficient transaction verification
- Reduced storage
 - Enables **Simplified Payment Verification (SPV)**

Q6. What is a Merkle Tree?

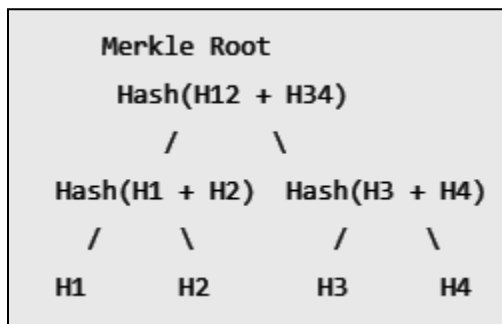
A Merkle Tree (also called a Hash Tree) is a binary tree data structure in which:

- Each leaf node contains the hash of a data block (e.g., transaction).
- Each non-leaf node contains the hash of its two child nodes.
- The top hash is called the Merkle Root.

It is used to efficiently verify data integrity in distributed systems like blockchains

Structure of a Merkle Tree

- **Leaf Nodes:** Hashes of individual transactions/data blocks.
- **Intermediate Nodes:** Hashes of concatenated child hashes.
- **Root Node (Merkle Root):** Final hash representing all data



4. Merkle Tree Rules

1. Every transaction is hashed first.
2. Hashes are combined in pairs.
3. If the number of hashes is odd, the last hash is duplicated.
4. Parent node hash = Hash(left child + right child)
5. Process continues until only one hash remains (Merkle Root).

Q7. Working of Merkle Tree (Step-by-Step)

1. Start with a list of transactions.
2. Generate hash for each transaction.
3. Pair adjacent hashes and hash their concatenation.

4. Repeat the pairing and hashing at each level.
5. Continue until a single hash (Merkle Root) is produced.

Benefits of Merkle Tree

- **Data Integrity** – Detects tampering instantly.
- **Efficient Verification** – Verifies large data with small hash proofs.
- **Reduced Storage** – Only hashes are stored, not full data.
- **Scalability** – Works efficiently with millions of transactions.
- **Security** – Cryptographic hashes prevent forgery.

Uses of Merkle Tree

- Blockchain transaction verification
- Distributed systems
- File integrity checking
- Version control systems
- Peer-to-peer (P2P) networks

Q8. Use Cases of Merkle Tree

1. Blockchain (Bitcoin, Ethereum)

- Stores transaction summaries in each block.
- Enables light clients (SPV) to verify transactions without full blockchain.

2. Cryptocurrency Mining

- Merkle Root is included in the block header.
- Changing a transaction changes the block hash.

3. Distributed Databases

- Efficient data synchronization between servers.

4. Git Version Control

- Tracks file changes and ensures data integrity.

5. Cloud Storage Systems

- Verifies file integrity without downloading entire files.

9. Why Merkle Trees Are Important

Merkle Trees allow secure, fast, and efficient verification of large datasets, making them essential for blockchain technology and distributed computing.

Implementation:

1. Write a Python program to understand SHA and Cryptography in Blockchain, Merkle root tree hash

```
import hashlib

def sha256_hash(data):
    """
    Generates SHA-256 hash for the given input string.
    """
    sha = hashlib.sha256()
    sha.update(data.encode('utf-8'))
    return sha.hexdigest()

# User input
message = input("Enter a string to hash using SHA-256: ")

# Generate hash
hash_value = sha256_hash(message)

print("\nSHA-256 Hash:")
print(hash_value)
```

```
Enter a string to hash using SHA-256: rujuta

SHA-256 Hash:
fb95267b17e4b69adfed9e84b2075e8a7eec20cb67645021fdb8d3c23e8ef556
```

2. Target Hash Generation with Nonce: Create a program to generate a hash code by concatenating a user input string and a nonce value to simulate the mining process.

```
import hashlib

def sha256_hash(data):
    """
    Returns SHA-256 hash of the given data
    """
```

```

    return hashlib.sha256(data.encode('utf-8')).hexdigest()

# User input
message = input("Enter the data to be mined: ")

# Difficulty level (number of leading zeros)
difficulty = int(input("Enter difficulty level : "))

target_prefix = "0" * difficulty
nonce = 0

while True:
    combined_data = message + str(nonce)
    hash_result = sha256_hash(combined_data)

    if hash_result.startswith(target_prefix):

        print("Data      :", message)
        print("Nonce      :", nonce)
        print("Hash Result :", hash_result)
        break

    nonce += 1

```

```

Enter the data to be mined: rujuta
Enter difficulty level : 2
2
Data      : rujuta
Nonce      : 766
Hash Result : 00bd9c61ebfdb50a96eac836ebc6b421310660a6b88d81fa2dd30420380ae
a3

```

3. Proof-of-Work Puzzle Solving: Implemented a program to find the nonce that, when combined with a given input string, produces a hash starting with a specified number of leading zeros.

```

import hashlib

def proof_of_work(data, difficulty):
    nonce = 0
    target = "0" * difficulty

    while True:
        combined = data + str(nonce)

```



```

hash_result = hashlib.sha256(combined.encode()).hexdigest()

if hash_result.startswith(target):
    return nonce, hash_result
nonce += 1

# Example usage
data = input("Enter data: ")
difficulty = int(input("Enter difficulty (number of leading zeros): "))

nonce, hash_result = proof_of_work(data, difficulty)
print("Nonce found:", nonce)
print("Valid Hash:", hash_result)

```

```

Enter data: h
Enter difficulty (number of leading zeros): 1
Nonce found: 26
Valid Hash: 0d368145227c320365c84b2120a560309a5fab745b05270a632d8128fb1ddf7b

```

```

Enter data: h
Enter difficulty (number of leading zeros): 2
Nonce found: 513
Valid Hash: 0032c8b348d54d996bf5926715b00732e8a0afbbee49c7180bdf9d42a50f9995

```

```

Enter data: h
Enter difficulty (number of leading zeros): 3
Nonce found: 1103
Valid Hash: 000b469ca1628133ccaf405ae947e57b5191484d16da7d34996513473c82aa42

=== Code Execution Successful ===

```

4. Merkle Tree Construction: Built a Merkle Tree from a list of transactions by recursively hashing pairs of transaction hashes, doubling up last nodes if needed, and generated the Merkle Root hash for blockchain transaction integrity.

```

import hashlib

def sha256(data):
    return hashlib.sha256(data.encode()).hexdigest()

def merkle_root(transactions):
    # Hash all transactions

```

```

hashes = [sha256(tx) for tx in transactions]

# Build the Merkle Tree
while len(hashes) > 1:
    if len(hashes) % 2 != 0:
        hashes.append(hashes[-1]) # Duplicate last hash if odd

    new_level = []
    for i in range(0, len(hashes), 2):
        combined_hash = sha256(hashes[i] + hashes[i + 1])
        new_level.append(combined_hash)

    hashes = new_level

return hashes[0]

# Example usage
transactions = [
    "Tx1: Alice pays Bob",
    "Tx2: Bob pays Charlie",
    "Tx3: Charlie pays Dave",
    "Tx4: Dave pays Eve"
]

print("Merkle Root:", merkle_root(transactions))

```

```

Merkle Root: 8fc5b2e4190bad429227fcd3d63a35aebbbbd72ab6ee0fb6425c05b7e9d0316
9

```

Conclusion: In this experiment, we implemented SHA-256 hashing, nonce-based mining, Proof-of-Work, and Merkle Tree construction to understand the core cryptographic mechanisms used in blockchain. The programs demonstrated how hash functions ensure data integrity, how nonce and difficulty simulate the mining process, how Proof-of-Work secures blocks through computational effort, and how Merkle Trees efficiently verify transaction integrity using a single Merkle Root hash.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801: Blockchain & DLT Lab
Experiment 02

Aim: Create a Blockchain using Python

Roll No.	33
Name	Rujuta Vinit Medhi
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO5: Interpret different Crypto assets and Crypto currencies

Experiment No: 02

Aim: Create a Blockchain using Python

Theory:

1. What is Blockchain?

Blockchain is decentralized and distributed, meaning no single authority controls it. Instead, multiple computers (nodes) on a network each have a copy of the blockchain, keeping the ledger synchronized. This setup ensures that once data, like a transaction, is recorded and confirmed, it becomes immutable almost impossible to alter or delete. The name "blockchain" comes from its structure data is organized in blocks, with each new block linked to the one before it, forming a continuous chain.

Each block contains crucial data, such as a list of transactions, a timestamp, and a unique identifier called a cryptographic hash. This hash is generated from the block's contents and the hash of the previous block, ensuring that each block is tightly connected to the one before it.

- Blockchain's linked structure makes data tampering detectable by altering hashes and breaking the chain.
- It acts as a distributed database, storing transactions across the network.
- Each transaction is verified by the majority, ensuring legitimacy.
- This decentralization prevents any single party from manipulating the data.

2. What is Block?

A block in a blockchain network is similar to a link in a chain. In the field of cryptocurrency, blocks are like records that store transactions like a record book, and those are encrypted into a hash tree.

There are a huge number of transactions occurring every day in the world. The users need to keep track of those transactions, and they do it with the help of a block structure.

3. Components of block

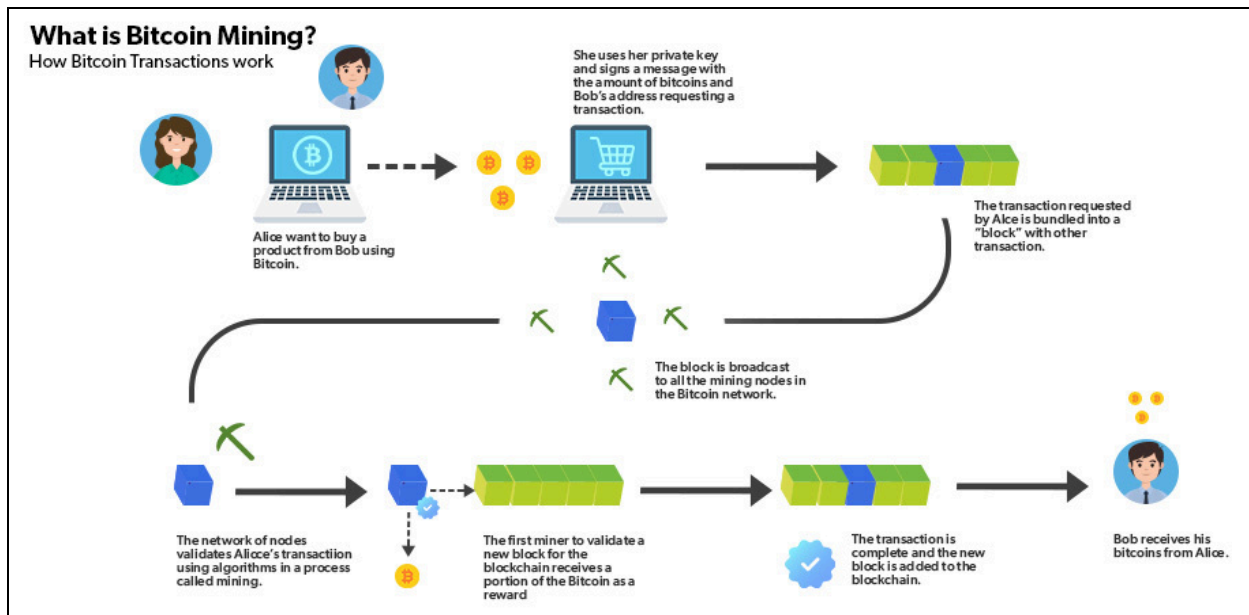
1. Header: It is used to identify the particular block in the entire blockchain. It handles all blocks in the blockchain. A block header is hashed periodically by miners by changing the nonce value as part of normal mining activity, also Three sets of block metadata are contained in the block header.

2. Previous Block Address/ Hash: It is used to connect the $i+1$ th block to the i th block using the hash. In short, it is a reference to the hash of the previous (parent) block in the chain.
3. Timestamp: It is a system that verifies the data into the block and assigns a time or date of creation for digital documents. The timestamp is a string of characters that uniquely identifies the document or event and indicates when it was created.
4. Nonce: A nonce number which used only once. It is a central part of the proof of work in the block. It is compared to the live target if it is smaller or equal to the current target. People who mine, test, and eliminate many Nonce per second until they find that Valuable Nonce is valid.
5. Merkle Root: It is a type of data structure frame of different blocks of data. A Merkle Tree stores all the transactions in a block by producing a digital fingerprint of the entire transaction. It allows the users to verify whether a transaction can be included in a block or not.

Structure of block

```
Block {  
  Index: 2  
  Timestamp: 2026-01-22 10:05:00  
  Data: {"Sender": "Alice", "Receiver": "Bob", "Amount": 50}  
  Nonce: 102345  
  Previous Hash: "0000a1b2c3d4e5..."  
  Hash: "0000f3b7d6c1e8..."  
}
```

3. Process of mining



Mining is the process of adding a new block to the blockchain.

It ensures security and integrity using Proof-of-Work (PoW).

Step 1: Collect Transactions

- All pending transactions are collected into a new block.
- Example: Alice sends 50 coins to Bob.

Step 2: Create Block Header

Block header includes:

- Previous hash
- Transactions
- Timestamp
- Nonce (initially 0)

Step 3: Proof of Work (PoW)

- The miner repeatedly changes the nonce to find a hash that satisfies the difficulty requirement.
- Difficulty = number of leading zeroes required in hash (e.g., 4 leading zeroes).

$\text{SHA256}(\text{block_data} + \text{nonce}) \rightarrow \text{hash starts with "0000"}$

- This requires computational work, hence the name Proof-of-Work.

Step 4: Validate and Broadcast

- Once a valid hash is found, the block is broadcast to all nodes in the network.

- Other nodes verify:

1. Hash correctness
2. Nonce validity
3. Previous hash link

Step 5: Add to Blockchain

- If all nodes validate, the block is added to the chain.
- The miner receives a reward (e.g., cryptocurrency).

4. How to check validity of blocks in blockchain

Step 1: Validate Previous Hash

- Each block's previous_hash must match the hash of the previous block.

if Block[n].previous_hash != Hash(Block[n-1]):

Invalid

Step 2: Recalculate Current Hash

- Compute hash of current block's data and nonce.
- It must match the stored hash in the block.

if SHA256(Block[n].data + Block[n].nonce) != Block[n].hash:

Invalid

Step 3: Check Proof-of-Work

- Verify that the block hash satisfies the difficulty (e.g., 4 leading zeroes).

if not Block[n].hash.startswith("0000"):

Invalid

Step 4: Validate Entire Chain

- Repeat steps 1–3 for all blocks from Genesis to the latest block.
- If any block fails → blockchain is invalid.

Step 5: Validate Genesis Block

- The first block (Genesis block) is hardcoded.
- Previous hash = "0"

- Must not be altered.

Implementation:

Installing Flask

```
PS C:\NRK> python -m pip install flask
Collecting flask
  Downloading flask-3.1.2-py3-none-any.whl.metadata (3.2 kB)
Collecting blinker>=1.9.0 (from flask)
  Downloading blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from flask)
  Downloading click-8.3.1-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2.0 (from flask)
  Downloading itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2>=3.1.2 (from flask)
  Downloading jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting markupsafe>=2.1.1 (from flask)
  Downloading markupsafe-3.0.3-cp314-cp314-win_amd64.whl.metadata (2.8 kB)
Collecting werkzeug>=3.1.0 (from flask)
  Downloading werkzeug-3.1.5-py3-none-any.whl.metadata (4.0 kB)
Collecting colorama (from click>=8.1.3->flask)
  Downloading colorama-0.4.6-py2.py3-none-any.whl.metadata (17 kB)
Downloading flask-3.1.2-py3-none-any.whl (103 kB)
Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
Downloading click-8.3.1-py3-none-any.whl (108 kB)
Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Downloading jinja2-3.1.6-py3-none-any.whl (134 kB)
Downloading markupsafe-3.0.3-cp314-cp314-win_amd64.whl (15 kB)
Downloading werkzeug-3.1.5-py3-none-any.whl (225 kB)
```

```
Installing collected packages: markupsafe, itsdangerous, colorama, blinker, werkzeug, jinja2, click, flask
7/8 [flask] WARNING: The script flask.exe is installed in 'C:\Users\admin\AppData\Local\Python\pythoncore-3.14-64\Scripts' which is not on PATH.
Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
Successfully installed blinker-1.9.0 click-8.3.1 colorama-0.4.6 flask-3.1.2 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.3 werkzeug-3.1.5
```

Code:

blockchain.py

```
import datetime
import hashlib
import json
class Blockchain:
    def __init__(self):
        self.chain = []
        self.difficulty = 4 # 4 leading zeroes
        self.create_genesis_block()
    def create_genesis_block(self):
        genesis_block = {
            'index': 1,
            'timestamp': str(datetime.datetime.now()),
            'data': 'Genesis Block',
```



```

        'nonce': 0,
        'previous_hash': '0'
    }
    genesis_block['hash'] = self.calculate_hash(genesis_block)
    self.chain.append(genesis_block)

def calculate_hash(self, block):
    block_copy = block.copy()
    block_copy.pop('hash', None)
    encoded = json.dumps(block_copy, sort_keys=True).encode()
    return hashlib.sha256(encoded).hexdigest()

def mine_block(self, data):
    previous_block = self.chain[-1]
    nonce = 0
    while True:
        block = {
            'index': len(self.chain) + 1,
            'timestamp': str(datetime.datetime.now()),
            'data': data,
            'nonce': nonce,
            'previous_hash': previous_block['hash']
        }
        block_hash = self.calculate_hash(block)

        if block_hash.startswith('0' * self.difficulty):
            block['hash'] = block_hash
            self.chain.append(block)
            return block
        nonce += 1

def is_chain_valid(self):
    for i in range(1, len(self.chain)):
        current = self.chain[i]
        previous = self.chain[i - 1]
        if current['previous_hash'] != previous['hash']:
            return False
        recalculated_hash = self.calculate_hash(current)
        if recalculated_hash != current['hash']:
            return False

```

```
        if not current['hash'].startswith('0' * self.difficulty):
            return False
    return True
```

app.py

```
from flask import Flask, jsonify
from blockchain import Blockchain

app = Flask(__name__)
blockchain = Blockchain()

@app.route('/mine_block', methods=['GET'])
def mine_block():
    block = blockchain.mine_block("Some transaction data")
    return jsonify({
        'message': '🔨 Block mined successfully!',
        'block': block
    }), 200

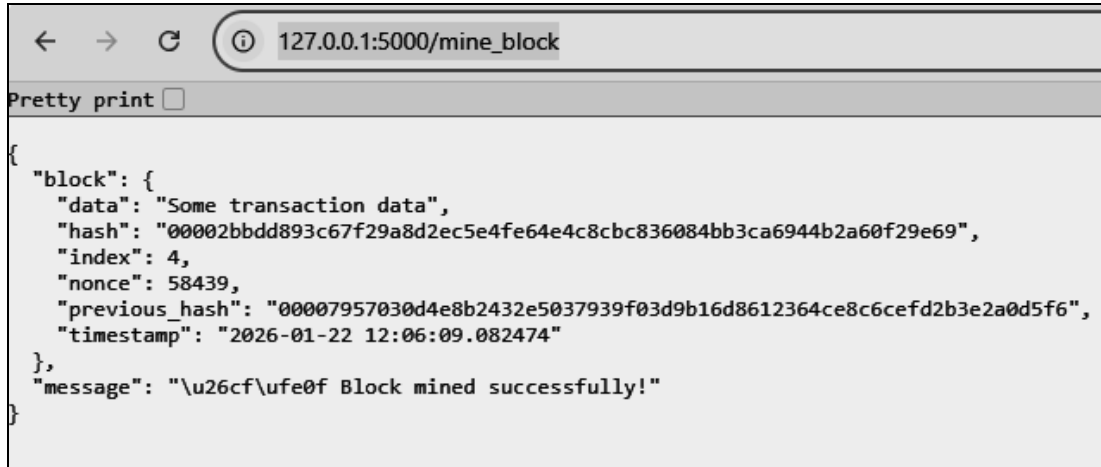
@app.route('/get_chain', methods=['GET'])
def get_chain():
    return jsonify({
        'chain': blockchain.chain,
        'length': len(blockchain.chain)
    }), 200

@app.route('/is_valid', methods=['GET'])
def is_valid():
    return jsonify({
        'is_valid': blockchain.is_chain_valid()
    }), 200

if __name__ == '__main__':
    print(" Mining blockchain with 4 leading zeroes...")
    app.run(debug=True, use_reloader=False)
```

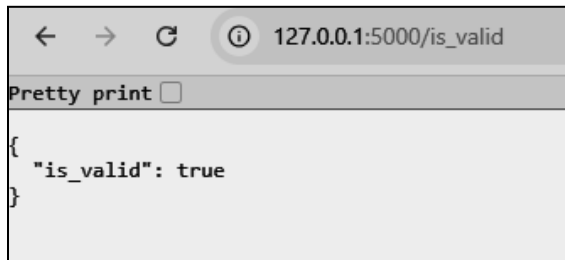
Output:

mine_block



```
{
  "block": {
    "data": "Some transaction data",
    "hash": "00002bbdd893c67f29a8d2ec5e4fe64e4c8cbc836084bb3ca6944b2a60f29e69",
    "index": 4,
    "nonce": 58439,
    "previous_hash": "00007957030d4e8b2432e5037939f03d9b16d8612364ce8c6cefd2b3e2a0d5f6",
    "timestamp": "2026-01-22 12:06:09.082474"
  },
  "message": "\u26cf\u2013 Block mined successfully!"
}
```

is_valid



```
{
  "is_valid": true
}
```

get_chain

```
127.0.0.1:5000/get_chain

Pretty print ☐

{
  "chain": [
    {
      "data": "Genesis Block",
      "hash": "d1c793d584710b37ed90d397f2f7a59cd9f741b1f424553cae1948dfeed661c4",
      "index": 1,
      "nonce": 0,
      "previous_hash": "0",
      "timestamp": "2026-01-22 12:05:29.544918"
    },
    {
      "data": "Some transaction data",
      "hash": "000030b5874b1e81cad31a1eb57cbcf69db993bbb0af36834c9742c4bea27ab5",
      "index": 2,
      "nonce": 215962,
      "previous_hash": "d1c793d584710b37ed90d397f2f7a59cd9f741b1f424553cae1948dfeed661c4",
      "timestamp": "2026-01-22 12:05:49.656127"
    },
    {
      "data": "Some transaction data",
      "hash": "00007957030d4e8b2432e5037939f03d9b16d8612364ce8c6cefd2b3e2a0d5f6",
      "index": 3,
      "nonce": 30433,
      "previous_hash": "000030b5874b1e81cad31a1eb57cbcf69db993bbb0af36834c9742c4bea27ab5",
      "timestamp": "2026-01-22 12:06:04.610851"
    },
    {
      "data": "Some transaction data",
      "hash": "00002bbdd893c67f29a8d2ec5e4fe64e4c8cbc836084bb3ca6944b2a60f29e69",
      "index": 4,
      "nonce": 58439,
      "previous_hash": "00007957030d4e8b2432e5037939f03d9b16d8612364ce8c6cefd2b3e2a0d5f6",
      "timestamp": "2026-01-22 12:06:09.082474"
    }
  ]
}
```

Conclusion: This experiment demonstrated the basic working of blockchain using Python. It showed how blocks are linked using cryptographic hashes, how mining with Proof-of-Work secures the data, and how validation ensures immutability. Overall, the experiment helped in understanding how blockchain maintains security, transparency, and data integrity without a central authority.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 03

Aim: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Roll No.	33
Name	Rujuta Vinit Medhi
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3: Implement smart contracts in Ethereum using different development frameworks. CO5: Interpret different Crypto assets and Crypto currencies

Experiment: 03

Aim: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Theory:

1. Blockchain Overview

Blockchain is a distributed and decentralized ledger that stores information in a series of linked blocks.

Each block contains:

- Transaction data
- Timestamp
- Previous block's hash
- Its own unique hash (digital fingerprint)

Once data is recorded in a blockchain, it becomes immutable because altering one block would require recalculating all subsequent blocks.

2. Mining

Mining is the process of:

1. Collecting pending transactions into a block.
2. Performing a computational puzzle (Proof-of-Work) to find a valid hash.
3. Adding the new block to the blockchain.
Broadcasting it to all connected peers.

Miners are rewarded with cryptocurrency for successfully mining a block.

3. Multi-Node Blockchain Network

In this lab, we simulate three independent blockchain nodes (5001, 5002, 5003).

Each node:

- Runs on a separate port.
- Maintains its own copy of the blockchain.
- Can connect with peers to share and validate blocks.

4. Consensus Mechanism

We use the **Longest Chain Rule**:

- If multiple versions of the chain exist, the longest valid chain is chosen.
- This ensures all nodes agree on a single transaction history.

5. Transactions & Mining Reward

Each transaction has:

- Sender
- Receiver
- Amount

When mining a block:

- Pending transactions are added to the block.
- A reward transaction is added automatically to pay the miner.

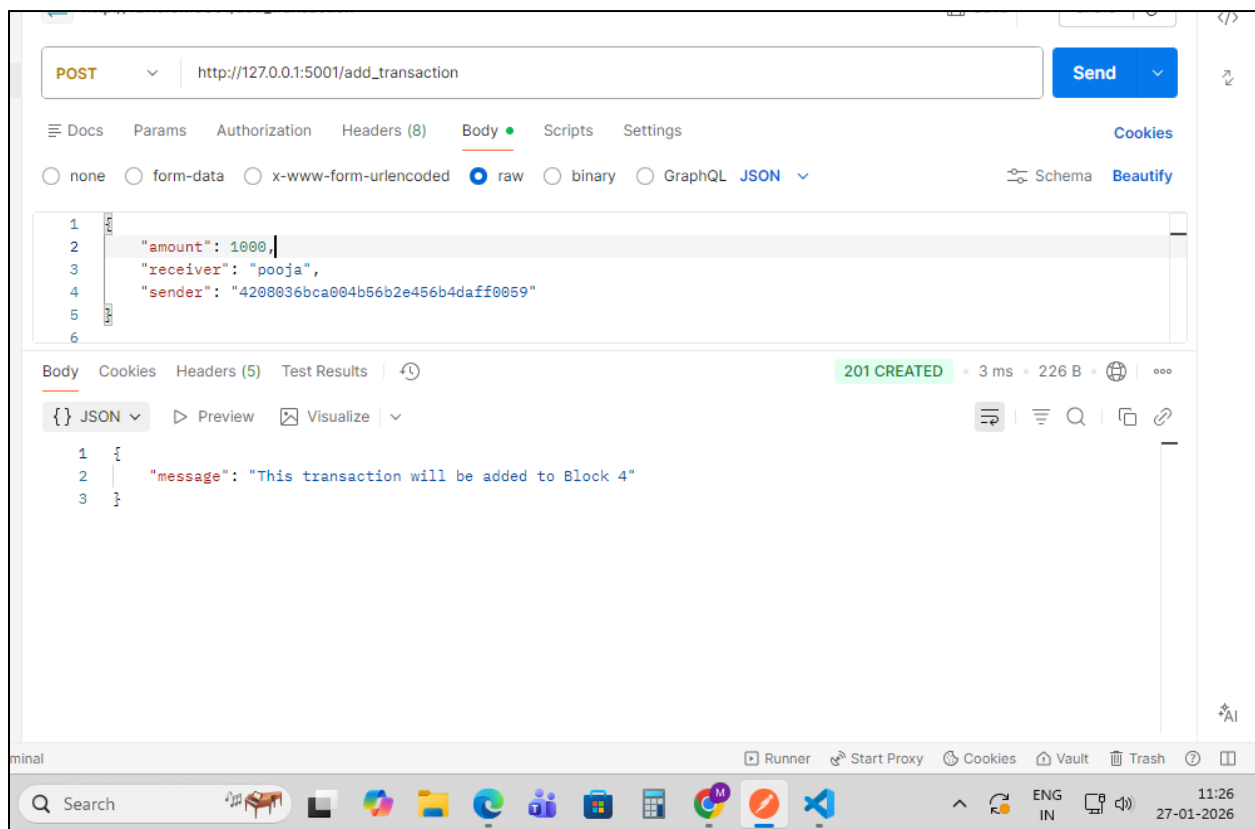
6. Chain Replacement

When /replace_chain is called:

1. Node requests chains from peers.
2. If it finds a longer and valid chain, it replaces its own.
3. This keeps the blockchain consistent across all nodes.

Implementation:

1. Add Transactions - invoke add_transactions() as a POST request.



2. mining - mine_block()

GET `http://127.0.0.1:5001/mine_block` Send

Docs Params Authorization Headers (6) Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers (5) Test Results (1/1) 200 OK 25 ms 621 B Save Response

JSON Preview Visualize

```
1 {
2   "index": 4,
3   "message": "Congratulations, you just mined a block!",
4   "previous_hash": "ed3cc6f96be22e405a6ed26538e3c0a71a79779d682eff6e7e713be9ec690104",
5   "proof": 21391,
6   "timestamp": "2026-01-27 11:26:58.840324",
7   "transactions": [
8     {
9       "amount": 1,
10      "receiver": "Richard",
11      "sender": "4208036bca004b56b2e456b4daff0059"
12    },
13    {
14      "amount": 1000,
15      "receiver": "pooja",
16      "sender": "4208036bca004b56b2e456b4daff0059"
17    }
18  ]
19 }
```

3. fetch the chain - get_chain(),

GET `http://127.0.0.1:5001/get_chain` Save Share Send

Docs Params Authorization Headers (6) Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers (5) Test Results 200 OK 4 ms 533 B

JSON Preview Visualize

```
1 {
2   "chain": [
3     {
4       "index": 1,
5       "previous_hash": "0",
6       "proof": 1,
7       "timestamp": "2026-01-27 11:09:25.029105",
8       "transactions": []
9     },
10    {
11      "index": 2,
12      "previous_hash": "6d617bf1d8543b614960a1afcdbe1e1bb5f00e0e840c15c815761faffe79dec0",
13      "proof": 533,
14      "timestamp": "2026-01-27 11:12:17.003830",
15      "transactions": []
16    }
17  ]
18 }
```

4. node - invoke connect_node()

HTTP http://127.0.0.1:5001/connect_node Save Share

POST http://127.0.0.1:5001/connect_node Send

Docs Params Authorization Headers (8) **Body** Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON ☐ Schema Beautify

```
1 {
2   "nodes": [
3     "http://127.0.0.1:5000",
4     "http://127.0.0.1:5002"
5   ]
6 }
```

Body Cookies Headers (5) Test Results 201 CREATED • 5 ms • 325 B

{ } JSON Preview Visualize

```
1 {
2   "message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:",
3   "total_nodes": [
4     "127.0.0.1:5002",
5     "127.0.0.1:5000"
6   ]
7 }
```

5. replace the longest chain - replace_chain()

HTTP http://127.0.0.1:5001/replace_chain Save Share

GET http://127.0.0.1:5001/replace_chain Send

Docs Params Authorization Headers (6) **Body** Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
-----	-------	-------------	-----------

Body Cookies Headers (5) Test Results 200 OK • 3 ms • 1.2 KB

{ } JSON Preview Visualize

```
1 {
2   "actual_chain": [
3     {
4       "index": 1,
5       "previous_hash": "0",
6       "proof": 1,
7       "timestamp": "2026-01-27 11:09:25.029105",
8       "transactions": []
9     },
10    {
11      "index": 2,
12      "previous_hash": "6d617bf1d8543b614960a1afcdbe1e1bb5f00e0e840c15c815761faffe79dec0",
13      "proof": 633,
14      "timestamp": "2026-01-27 11:12:17.003830",
15      "transactions": [
16        {
17          "amount": 1,
18          "receiver": "Richard",

```

```

19         "sender": "4208036bca004b56b2e456b4daff0059"
20     }
21 ]
22 },
23 {
24     "index": 3,
25     "previous_hash": "1af95c07c0d568f49621eb2fd8759ff3c729423d818dae19a05b2083fd",
26     "proof": 45293,
27     "timestamp": "2026-01-27 11:19:00.005438",
28     "transactions": [
29         {
30             "amount": 1,
31             "receiver": "Richard",
32             "sender": "4208036bca004b56b2e456b4daff0059"
33         }
34     ],
35     "index": 4,
36     "previous_hash": "ed3cc6f96be22e405a6ed26538e3c0a71a79779d682eff6e7e713be9ec690104",
37     "proof": 21391,
38     "timestamp": "2026-01-27 11:26:58.840324",
39     "transactions": [
40         {
41             "amount": 1,
42             "receiver": "Richard",
43             "sender": "4208036bca004b56b2e456b4daff0059"
44         },
45         {
46             "amount": 1000,
47             "receiver": "pooja",
48             "sender": "4208036bca004b56b2e456b4daff0059"
49         }
50     ],
51     "amount": 1,
52     "receiver": "Richard",
53     "sender": "4208036bca004b56b2e456b4daff0059"
54 }
55 ],
56 ],
57 ],
58 },
59 ],
60 "message": "All good. The chain is the largest one."
61 }

```

Conclusion: This experiment demonstrates the practical application of blockchain principles through the implementation of a simulated cryptocurrency. By modeling transactions, mining, and consensus across multiple nodes, it provides clear insight into how decentralized blockchain systems function. The experiment emphasizes the role of security, consensus mechanisms, and decentralization in maintaining a reliable and distributed ledger.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 04

Aim: Hands-on Solidity Programming Assignments for creating Smart Contracts

Roll No.	33
Name	Rujuta Vinit Medhi
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO3:Implement smart contracts in Ethereum using different development frameworks.

Experiment: 04

Aim: Hands on Solidity Programming Assignments for creating Smart Contracts

Theory:

1. Primitive Data Types, Variables, Functions – pure, view

In Solidity, primitive data types form the foundation of smart contract development. Commonly used types include:

- **uint / int:** unsigned and signed integers of different sizes (e.g., uint256, int128).
- **bool:** represents logical values (true or false).
- **address:** holds a 20-byte Ethereum account address, often used for storing user accounts or contract addresses.
- **bytes / string:** store binary data or textual data.

Variables in Solidity can be state variables (stored on the blockchain permanently), local variables (temporary, created during function execution), or global variables (special predefined variables such as `msg.sender`, `msg.value`, and `block.timestamp`).

Functions allow execution of contract logic. Special types of functions include:

- **pure:** cannot read or modify blockchain state; they work only with inputs and internal computations.
- **view:** can read state variables but cannot alter them. This classification helps optimize gas usage and enforces function integrity.

2. Inputs and Outputs to Functions

Functions in Solidity can accept input arguments and return one or more output values. Inputs enable users or other contracts to pass data into the contract, while outputs make it possible to return results after computation. For example, a function can accept an amount in Ether and return whether the transfer was successful. Solidity also allows named return variables, which improve readability and debugging.

3. Visibility, Modifiers and Constructors

- Function Visibility defines who can access a function:
 - o **public:** available both inside and outside the contract.
 - o **private:** only accessible within the same contract.
 - o **internal:** accessible within the contract and its child contracts.

- **Modifiers** are reusable code blocks that change the behavior of functions. They are often used for access control, such as restricting sensitive functions to the contract owner (onlyOwner).
- **Constructors** are special functions executed only once during contract deployment. They initialize important values, such as setting the deploying account as the owner of the contract.

4. Control Flow: if-else, loops

Control flow in Solidity is similar to traditional programming languages:

- **if-else** allows conditional decision-making in contract logic, e.g., checking if a balance is sufficient before transferring funds.
- **Loops** (for, while, do-while) enable repeated execution of code. For example, iterating through an array of users. However, loops must be used carefully, as excessive iterations increase gas consumption, potentially making the contract expensive to execute.

5. Data Structures: Arrays, Mappings, Structs, Enums

- **Arrays**: Can be fixed or dynamic and are used to store ordered lists of elements. Example: an array of addresses for registered users.
- **Mappings**: Key-value pairs that allow quick lookups. Example: mapping(address => uint) for storing balances. Unlike arrays, mappings do not support iteration.
- **Structs**: Allow grouping of related properties into a single data type, such as creating a struct Player {string name; uint score;}.
- **Enums**: Used to define a set of predefined constants, making code more readable. Example: enum Status { Pending, Active, Closed }.

6. Data Locations

Solidity uses three primary data locations for storing variables:

- **storage**: Data stored permanently on the blockchain. Examples: state variables.
- **memory**: Temporary data storage that exists only while a function is executing. Used for local variables and function inputs.
- **calldata**: A non-modifiable and non-persistent location used for external function parameters. It is gas-efficient compared to memory. Understanding data locations is essential, as they directly impact gas costs and performance.

7. Transactions: Ether and Wei, Gas and Gas Price, Sending Transactions

- **Ether and Wei**: Ether is the main currency in Ethereum. All values are measured in Wei, the smallest unit (1 Ether = 10^{18} Wei). This ensures high precision in financial transactions.

- **Gas and Gas Price:** Every transaction consumes gas, which represents computational effort. The gas price determines how much Ether is paid per unit of gas. A higher gas price incentivizes miners to prioritize the transaction.
- **Sending Transactions:** Transactions are used for transferring Ether or interacting with contracts. Functions like `transfer()` and `send()` are commonly used, while `call()` provides more flexibility. Each transaction requires gas, making efficiency in contract design very important.

Implementation:

1. Introduction

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.3;
3
4  contract Counter {
5      //Rujuta Medhi
6      uint public count;
7
8      // Function to get the current count
9      function get() public view returns (uint) {    2453 gas
10         |     return count;
11         | }
12
13         // Function to increment count by 1
14         function inc() public {    infinite gas
15         |     count += 1;
16         | }
17
18         // Function to decrement count by 1
19         function dec() public {    infinite gas
20         |     count -= 1;
21         | }

```

The screenshot displays the Remix Ethereum IDE interface. The top section shows the Solidity code for a `Counter` contract with functions `get()`, `inc()`, and `dec()`. The bottom section shows the deployment and transaction details.

Deployment and Transaction Details:

- Transaction 1:** `Counter.inc()` from `0x5B3...eddC4` to `Counter.inc()` at `0xd8b...33fa8`. Value: `0 wei`, data: `0x371...303c0`, logs: `0 hash: 0xf7c...92a7a`. Status: Transaction mined and execution succeed.
- Transaction 2:** `Counter.inc()` from `0x5B3...eddC4` to `Counter.inc()` at `0xd8b...33fa8`. Value: `0 wei`, data: `0x371...303c0`, logs: `0 hash: 0xd63...f3030`. Status: Transaction mined and execution succeed.

Contract State:

- Contract: `COUNTER AT 0xD8B...33FA8`
- Balance: `0 ETH`
- Count: `uint256: 2`

Low level interactions:

- CALLDATA: `0x371...303c0`

2. Basic Syntax

```
pragma solidity ^0.8.3;

contract MyContract{
    string public name = "Rujuta Medhi";
}
```

The screenshot displays the Remix IDE interface. On the left, the 'DEPLOY & RUN TRANSACTIONS' sidebar shows two contracts: 'HELLOWORLD AT 0xA60...96B' and 'MYCONTRACT AT 0X30B...A56'. The 'MYCONTRACT' is selected, showing its 'name' variable and a 'Transact' button. The main workspace shows the Solidity code for 'MyContract'. The right sidebar displays the transaction details for a call to 'MyContract.name', including the transaction hash, gas cost, and the decoded output 'Rujuta Medhi'.

3. Primitive Data Types

The screenshot displays the Remix IDE interface with a more complex contract. The 'DEPLOY & RUN TRANSACTIONS' sidebar shows a list of variables: 'i', 'i256', 'is', 'neg', 'newAddr', 'newU', 'u', 'u256', and 'u8'. The main workspace shows the Solidity code for a contract with various primitive data types. The right sidebar displays the transaction details for a call to 'MyContract.name', including the transaction hash, gas cost, and the decoded output 'Rujuta Medhi'.

4. Variables

The screenshot shows the Remix Ethereum IDE interface. The left sidebar displays the 'DEPLOY & RUN TRANSACTIONS' panel with a 'Transact' button. Below it, the 'VARIABLES AT 0x7B9...B6ACE' section shows the state of the contract: 'Balance: 0 ETH', 'doSomething' button, 'blockNumber' button, 'num' button (value: 123), 'text' button (value: 'Rujuta Medhi'), and 'Low level interactions' section with a 'CALLDATA' input and 'Transact' button.

The main editor displays the Solidity code for the 'Variables' contract:

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Variables {
5     // State variables are stored on the blockchain.
6     string public text = "Rujuta Medhi";
7     uint public num = 123;
8     uint public blockNumber;
9
10    function doSomething() public { 22334 gas
11        // Local variables are not saved to the blockchain.
12        uint i = 456;
13
14        // Here are some global variables
15        uint timestamp = block.timestamp; // Current block timestamp
16        address sender = msg.sender; // address of the caller
17        blockNumber = block.number;
18    }
19 }
```

The bottom status bar shows 'Scam Alert', 'Initialize as git repo', 'Did you know?', 'RemixAI Copilot (enabled)', and system information (28°C Sunny, 11:18 03-02-2026).

5. Functions - Reading and Writing to a State Variable

The screenshot shows the Remix Ethereum IDE interface. The left sidebar displays the 'DEPLOY & RUN TRANSACTIONS' panel with a 'Transact' button. Below it, the 'SIMPLESTORAGE AT 0x5E1...4f' section shows the state of the contract: 'Balance: 0 ETH', 'set' button (value: 'uint256 _num'), 'b' button (value: true), 'get' button, 'get_b' button (value: true), and 'num' button.

The main editor displays the Solidity code for the 'SimpleStorage' contract:

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract SimpleStorage {
5     // State variable to store a number
6     uint public num;
7     bool public b = true;
8     // Rujuta Medhi
9     function get_b() public view returns (bool) { 2539 gas
10         return b;
11     }
12     // You need to send a transaction to write to a state variable.
13     function set(uint _num) public { 22536 gas
14         num = _num;
15     }
16
17     // You can read from a state variable without sending a transaction.
18     function get() public view returns (uint) { 2475 gas
19         return num;
20     }
21 }
```

The bottom status bar shows 'Scam Alert', 'Initialize as git repo', 'Did you know?', 'RemixAI Copilot (enabled)', and system information (28°C Sunny, 11:22 03-02-2026).

6. Functions - View and Pure

The screenshot shows the Remix IDE interface. The left sidebar contains the 'DEPLOY & RUN TRANSACTIONS' panel, which includes a 'Transact' button and a 'VIEWANDPURE AT 0XEC2...CF1' section. The main editor displays the Solidity code for the 'ViewAndPure' contract. The code includes a 'view' function 'addToX2' and a 'pure' function 'add'. The bottom status bar shows 'Scam Alert', 'Initialize as git repo', and 'RemixAI Copilot (enabled)'.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract ViewAndPure {
5     uint public x = 1;
6     //Rujuta Medhi
7     // Promise not to modify the state.
8     function addToX(uint y) public view returns (uint) {
9         return x + y;
10    }
11
12    function addToX2(uint y) public {
13        x = x+y;
14    }
15
16    // Promise not to modify or read from the state.
17    function add(uint i, uint j) public pure returns (uint) {
18        return i + j;
19    }
20 }
21
```

7. Functions - Modifiers and Constructors

The screenshot shows the Remix IDE interface. The left sidebar contains the 'DEPLOY & RUN TRANSACTIONS' panel, which includes a 'Transact' button and a 'FUNCTIONMODIFIER AT 0XB5' section. The main editor displays the Solidity code for the 'FunctionModifier' contract. The code includes a 'constructor' and a 'modifier' 'onlyOwner'. The bottom status bar shows 'Scam Alert', 'Initialize as git repo', and 'RemixAI Copilot (enabled)'.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract FunctionModifier {
5     //Rujuta
6     // We will use these variables to demonstrate how to use
7     // modifiers.
8     address public owner;
9     uint public x = 10;
10    bool public locked;
11
12    constructor() {
13        // Set the transaction sender as the owner of the contract.
14        owner = msg.sender;
15    }
16
17    // Modifier to check that the caller is the owner of
18    // the contract.
19    modifier onlyOwner() {
20        require(msg.sender == owner, "Not owner");
21        // Underscore is a special character only used inside
22    }
23 }
24
```

Blockchain and its Application | D20A-ITC801-Blockchain | KJ BC_01_02_03_10 - Google | Remix - Ethereum IDE | Untitled document - Google | +

remix.ethereum.org/?#activate=udapp,solidity,LearnEth&lang=en&optimize&runs=200&evmVersion&version=soljson-v0.8.31+commit.f3a2265js

REMIX 1.5.1 learneth tutorials Login with GitHub Theme

DEPLOY & RUN TRANSACTIONS

Balance: 0 ETH

changeOwner address_newOwner

decrement uint256 i

INCREASEX

y: 33

Calldata Parameters transact

locked

owner

x

0: uint256: 142

Low level interactions

block hash 0xf964d9b35dd779e8ae37f32f0b5a7ac4ed85089776b63658912fe74ee0ce379

block number 42

from 0x58380a6a701c568545dcfc883fc8875f56beddC4

to FunctionModifier.IncreaseX(uint256) 0x857ee0797c3fc0205714a577c02f7285b889d9f30

transaction cost 29048 gas

execution cost 7844 gas

output 0x

decoded input { "uint256 y": "33" }

decoded output {}

logs []

raw logs []

call to FunctionModifier.x

CALL [call] from: 0x58380a6a701c568545dcfc883fc8875f56beddC4 to: FunctionModifier.x() data: 0x0c5...5699c

Debug

Activate Windows Go to Settings to activate Windows.

Scam Alert Initialize as git repo Did you know? To learn new contract patterns and prototype, you can activate and try the cookbook plugin! RemixAI Copilot (enabled)

30°C Sunny

Search

ENG US 11:49 03-02-2026

8. Functions - Inputs and Outputs

REMIX 1.5.1 learneth tutorials Login with GitHub Theme

DEPLOY & RUN TRANSACTIONS

arrayInput uint256[]_arr

arr uint256

arrayOutput

assigned

destructingAss...

named

returnMany

returnTwo

0: int256: i - 2

1: bool: b true

Low level interactions

CALLDATA

Transact

Compile

```
//  
78  
79  
80  
81  
82  
83  
84  
85  
86  
87  
88  
89  
90  
91  
92
```

```
return arr;  
  
function returnTwo() 472 gas  
//Rujuta Medhi  
public  
pure  
returns (  
    int i,  
    bool b  
)  
{  
    i = -2;  
    b = true;  
}
```

Explain contract

AI copilot

0 Listen on all transactions Filter with transaction hash or address

Activate Windows Go to Settings to activate Windows.

Scam Alert Initialize as git repo Did you know? To learn new contract patterns and prototype, you can activate and try the cookbook plugin! RemixAI Copilot (enabled)

30°C Sunny

Search

ENG US 12:01 03-02-2026

9. Visibility

The screenshot shows the Remix IDE interface. On the left, the 'DEPLOY & RUN TRANSACTIONS' panel is visible, showing a balance of 0 ETH and several buttons for interacting with the contract: `externalFunc`, `publicFunc`, `publicVar`, `testInternalFunc`, and `testPrivateFunc`. The main editor displays a Solidity contract with the following code:

```
56 // Inherited contracts do not have access to private functions
57 // and state variables.
58 // function testPrivateFunc() public pure returns (string memory) {
59 //     return privateFunc();
60 // }
61
62 // Internal function call be called inside child contracts.
63 function testInternalFunc() public pure override returns (string memory) { infinite gas
64     return internalFunc();
65 }
66 // Rujuta Medhi
67 function testInternalVar() public view returns (string memory, string memory) { infinite gas
68     return (internalVar, publicVar);
69 }
70 }
```

The bottom status bar shows the system tray with a weather widget (30°C Sunny), search bar, and system clock (12:06, 03-02-2026).

10. Control Flow - If/Else

The screenshot shows the Remix IDE interface. On the left, the 'DEPLOY & RUN TRANSACTIONS' panel is visible, showing a balance of 0 ETH and several buttons for interacting with the contract: `foo` (uint256 x) and `ternary` (uint256 x). The main editor displays a Solidity contract with the following code:

```
9     return 1;
10 } else {
11     return 2;
12 }
13 }
14
15 function ternary(uint _x) public pure returns (uint) { infinite gas
16     // if (_x < 10) {
17     //     return 1;
18     // }
19     // shorthand way to write if / else statement
20     return _x < 10 ? 1 : 2;
21 }
22
23 //Rujuta Medhi
24 function evenCheck(uint y) public pure returns (bool) { infinite gas
25     return y%2 == 0 ? true : false;
26 }
27 }
28 }
```

The bottom status bar shows the system tray with a weather widget (30°C Sunny), search bar, and system clock (12:11, 03-02-2026).

11. Control Flow - Loops

The screenshot shows the Remix IDE interface. On the left, the 'DEPLOY & RUN TRANSACTIONS' panel is visible, showing a deployed contract named 'LOOP' at address 0x56A...5E318. The contract's state is displayed as 'count: 0'. The main editor shows a Solidity contract named 'Loop' with the following code:

```
4 contract Loop {
5   uint public count;
6   function loop() public {
7     // for loop
8     for (uint i = 0; i < 10; i++) {
9       if (i == 5) {
10        // Skip to next iteration with continue
11        continue;
12      }
13      if (i == 5) {
14        // Exit loop with break
15        break;
16      }
17      count++;
18    }
19    // Rujuta Medhi
20    // while loop
21    uint j;
22    while (j < 10) {
23      j++;
24    }
25  }
26 }
```

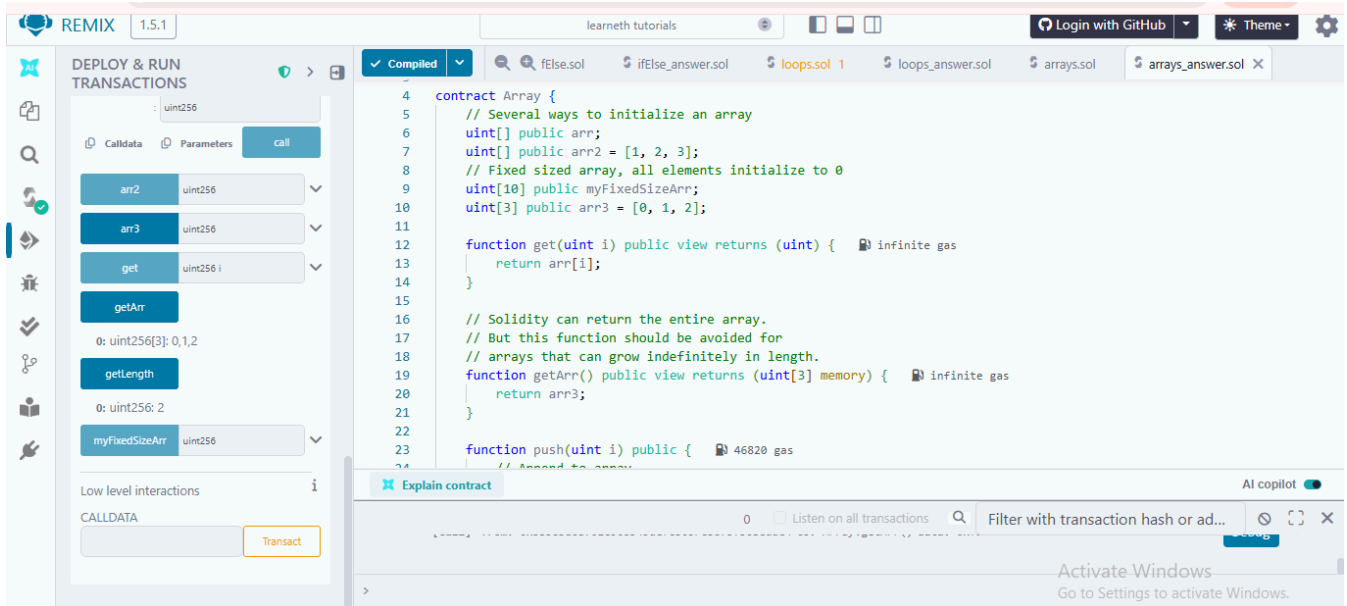
The bottom status bar shows 'Scam Alert', 'Initialize as git repo', and 'RemixAI Copilot (enabled)'.

12. Data Structures - Arrays

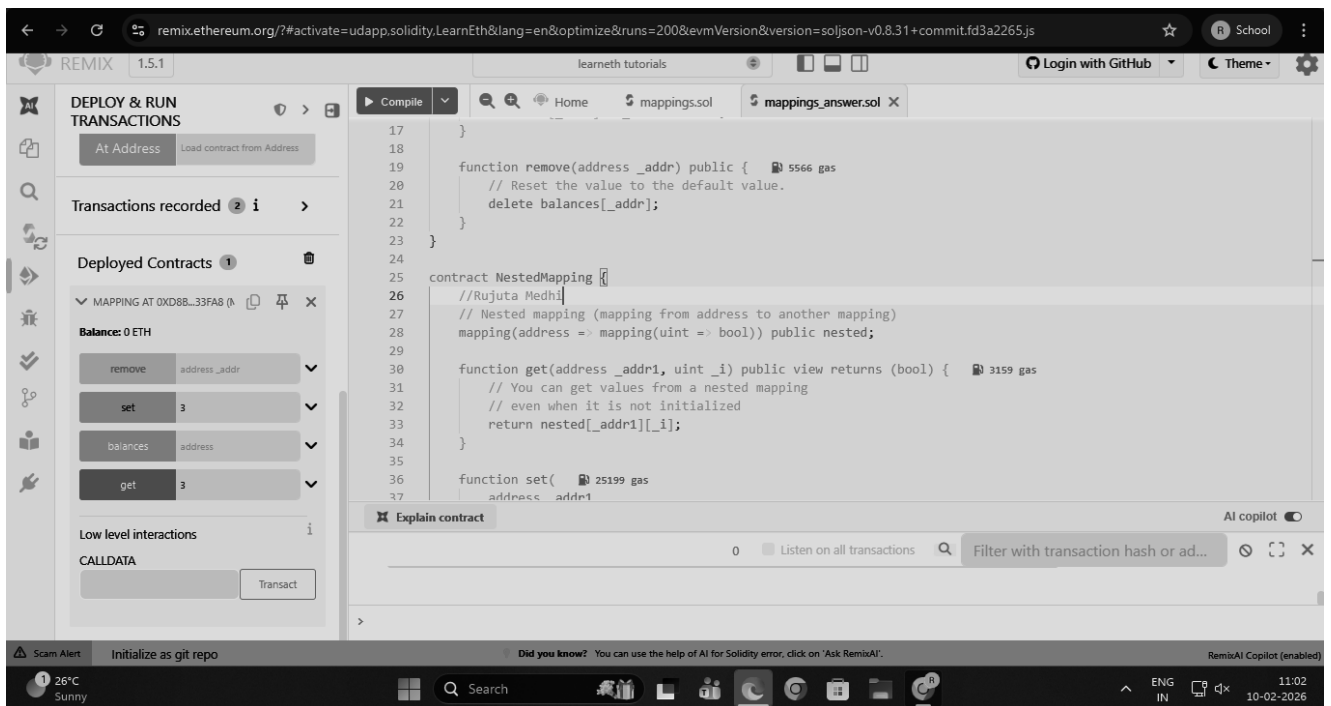
The screenshot shows the Remix IDE interface. On the left, the 'DEPLOY & RUN TRANSACTIONS' panel is visible, showing a deployed contract named 'POP' at address 0x56A...5E318. The contract's state is displayed as 'arr: uint256[]'. The main editor shows a Solidity contract named 'POP' with the following code:

```
25 // This will increase the array length by 1.
26 arr.push(i);
27
28
29 function pop() public {
30   // Remove last element from array
31   // This will decrease the array length by 1
32   arr.pop();
33 }
34
35 function getLength() public view returns (uint) {
36   return arr.length;
37 }
38
39 function remove(uint index) public {
40   // Delete does not change the array length.
41   // It resets the value at index to its default value,
42   // in this case 0
43   delete arr[index];
44 }
45 }
```

The bottom status bar shows 'Scam Alert', 'Initialize as git repo', and 'RemixAI Copilot (enabled)'.



13. Data Structures - Mappings



14. Data Structures - Structs

REMIX 1.5.1 learneth tutorials Login with GitHub Theme

LEARNETH

Tutorials list Syllabus

8.3 Data Structures - Structs 14 / 19

Another way to update the value is using the dot operator by providing the name of the enum and its member (line 35).

Accessing structs

To access a member of a struct we can use the dot operator (line 33).

Updating structs

To update a structs' member we also use the dot operator and assign it a new value (lines 39 and 45).

[Watch a video tutorial on Structs.](#)

Assignment

Create a function `remove` that takes a `uint` as a parameter and deletes a struct member with the given index in the `todos` mapping.

Check Answer Show answer

Next

Well done! No errors.

```
32 Todo storage todo = todos[_index];
33 return (todo.text, todo.completed);
34 }
35
36 // update text
37 function update(uint _index, string memory _text) public {
38     Todo storage todo = todos[_index];
39     todo.text = _text;
40 }
41
42 // update completed
43 function toggleCompleted(uint _index) public {
44     Todo storage todo = todos[_index];
45     todo.completed = !todo.completed;
46 }
47 //Rujuta Medhi -33
48 function remove(uint _index) public {
49     delete todos[_index];
50 }
51 }
```

Explain contract AI copilot

0 Listen on all transactions Filter with transaction hash or ad...

[vm] from: 0x583...eddC4 to: Todos.(constructor) value: 0 wei data: 0x608...f0033

15. Data Structures - Enums

REMIX 1.5.1 learneth tutorials Login with GitHub Theme

LEARNETH

Tutorials list Syllabus

8.4 Data Structures - Enums 15 / 19

Another way to update the value is using the dot operator by providing the name of the enum and its member (line 35).

Removing an enum value

We can use the delete operator to delete the enum value of the variable, which means as for arrays and mappings, to set the default value to 0.

[Watch a video tutorial on Enums.](#)

Assignment

1. Define an enum type called `Size` with the members `S`, `M`, and `L`.
2. Initialize the variable `sizes` of the enum type `Size`.
3. Create a getter function `getSize()` that returns the value of the variable `sizes`.

Check Answer Show answer

Next

Well done! No errors.

```
28
29
29 function getSize() public view returns (Size) {
30     return sizes;
31 }
32
33 // Update status by passing uint into input
34 function set(Status _status) public {
35     status = _status;
36 }
37
38 // You can update to a specific enum like this
39 function cancel() public {
40     status = Status.Canceled;
41 }
42 // Rujuta Medhi
43 // delete resets the enum to its first value, 0
44 function reset() public {
45     delete status;
46 }
47 }
```

Explain contract AI copilot

0 Listen on all transactions Filter with transaction hash or ad...

[vm] from: 0x583...eddC4 to: Todos.(constructor) value: 0 wei data: 0x608...f0033

16. Data Locations

LEARNETH 9. Data Locations 16 / 19

Assignment

1. Change the value of the `myStruct` member `foo`, inside the function `f`, to 4.
2. Create a new struct `myMemStruct2` with the data location `memory` inside the function `f` and assign it the value of `myMemStruct`. Change the value of the `myMemStruct2` member `foo` to 1.
3. Create a new struct `myMemStruct3` with the data location `memory` inside the function `f` and assign it the value of `myStruct`. Change the value of the `myMemStruct3` member `foo` to 3.
4. Let the function `f` return `myStruct`, `myMemStruct2`, and `myMemStruct3`.

Tip: Make sure to create the correct return types for the function `f`.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

```
28 function f( /* undefined gas
29     uint[] storage _arr,
30     mapping(uint => address) storage _map,
31     MyStruct storage _myStruct
32 ) internal {
33     // do something with storage variables
34 }
35
36 // You can return memory variables
37 function g(uint[] memory _arr) public returns (uint[] memory) { /* infinite gas
38     // do something with memory array
39     _arr[0] = 1;
40 }
41
42 function h(uint[] calldata _arr) external { /* 468 gas
43     // do something with calldata array
44     _arr[0] = 1;
45 }
46 }
47
```

Explain contract AI copilot

0 ☐ Listen on all transactions

Welcome to Remix 1.5.1

Scam Alert Initialize as git repo Did you know? You can use the help of AI for Solidity error, click on 'Ask RemixAI'. RemixAI Copilot (enabled)

17. Transactions - Ether and Wei

LEARNETH 10.1 Transactions - Ether and Wei 17 / 19

Assignment

1. Create a `public uint` called `oneGwei` and set it to 1 `gwei`.
2. Create a `public bool` called `isOneGwei` and set it to the result of a comparison operation between 1 `gwei` and 10^9 .

Tip: Look at how this is written for `gwei` and `ether` in the contract.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract EtherUnits {
5     uint public oneWei = 1 wei;
6     // 1 wei is equal to 1
7     bool public isOneWei = 1 wei == 1;
8
9     uint public oneEther = 1 ether;
10    // 1 ether is equal to 10^18 wei
11    bool public isOneEther = 1 ether == 1e18;
12
13    uint public oneGwei = 1 gwei;
14    // 1 ether is equal to 10^9 wei
15    bool public isOneGwei = 1 gwei == 1e9;
16 }
```

Explain contract AI copilot

0 ☐ Listen on all transactions

Welcome to Remix 1.5.1

Scam Alert Initialize as git repo Did you know? You can use the help of AI for Solidity error, click on 'Ask RemixAI'. RemixAI Copilot (enabled)

18. Transactions - Gas and Gas Price

REMIX 1.5.1 learneth tutorials Login with GitHub Theme

LEARNETH

Tutorials list Syllabus

10.2 Transactions - Gas and Gas Price 18 / 19

run out of *gas* before being completed, reverting any changes being made. In this case, the *gas* was consumed and can't be refunded.

Learn more about *gas* on ethereum.org.

Watch a video tutorial on Gas and Gas Price.

★ Assignment

Create a new **public** state variable in the **Gas** contract called **cost** of the type **uint**. Store the value of the gas cost for deploying the contract in the new variable, including the cost for the value you are storing.

Tip: You can check in the Remix terminal the details of a transaction, including the gas cost. You can also use the Remix plugin *Gas Profiler* to check for the gas cost of transactions.

Check Answer Show answer

Next

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Gas {
5     //Rujuta
6     uint public i = 0;
7     uint public cost = 170367;
8
9     // Using up all of the gas that you send causes your transaction to fail.
10    // State changes are undone
11    // Gas s uint256 public i
12    function
13    // H
14    // a remix-project-org/remix-workshops/10.2 Transactions - Gas and Gas Price/gasA
15    while
16        i += 1;
17    }
18 }
19 }
```

Explain contract AI copilot

0 Listen on all transactions Filter with transaction hash or address

Welcome to Remix 1.5.1

19. Transactions - Sending Ether

REMIX 1.5.1 learneth tutorials Login with GitHub Theme

LEARNETH

Tutorials list Syllabus

10.3 Transactions - Sending Ether 19 / 19

★ Assignment

Build a charity contract that receives Ether that can be withdrawn by a beneficiary.

1. Create a contract called **Charity**.
2. Add a public state variable called **owner** of the type **address**.
3. Create a donate function that is public and payable without any parameters or function code.
4. Create a withdraw function that is public and sends the total balance of the contract to the **owner** address.

Tip: Test your contract by deploying it from one account and then sending Ether to it from another account. Then execute the withdraw function.

Check Answer Show answer

Next

Well done! No errors.

```
49 require(sent, "Failed to send Ether");
50 }
51 }
52
53 contract Charity {
54     address public owner;
55
56     constructor() { 165452 gas 141000 gas
57         owner = msg.sender;
58     }
59
60     function donate() public payable {} 141 gas
61
62     function withdraw() public { infinite gas
63         uint amount = address(this).balance;
64
65         (bool sent, bytes memory data) = owner.call(value: amount)("");
66         require(sent, "Failed to send Ether");
67     }
68 }
```

Explain contract AI copilot

0 Listen on all transactions Filter with transaction hash or address

Welcome to Remix 1.5.1

Conclusion: This experiment provided practical exposure to Solidity programming and the process of creating smart contracts on a blockchain platform. Through hands-on implementation, the fundamental concepts of blockchain such as decentralization, immutability, and trustless execution were clearly understood. Designing and deploying smart contracts helped in learning Solidity syntax, data types, functions, modifiers, and transaction handling.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 05

Aim: Deploying a Voting/Ballot Smart Contract

Roll No.	33
Name	Rujuta Vinit Medhi
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks.

Aim: Deploying a Voting/Ballot Smart Contract

Theory:

1. Relevance of require Statements in Solidity Programs

In Solidity, the require statement acts as a **guard condition** within functions. It ensures that only valid inputs or authorized users can execute certain parts of the code. If the condition inside require is not satisfied, the function execution stops immediately, and all state changes made during that transaction are reverted to their original state. This rollback mechanism ensures that invalid transactions do not corrupt the blockchain data.

For example, in a **Voting Smart Contract**, require can be used to check:

- Whether the person calling the function has the right to vote (require(voters[msg.sender].weight > 0, "Has no right to vote");).
- Whether a voter has already voted before allowing them to vote again.
- Whether the function caller is the **chairperson** before granting voting rights.

Thus, require statements enforce **security, correctness, and reliability** in smart contracts. They also allow developers to attach error messages, making debugging and contract interaction easier for users.

2. Keywords: mapping, storage, and memory

- **mapping:**

A mapping is a special data structure in Solidity that links keys to values, similar to a hash table. Its syntax is mapping(keyType => valueType). For example:

```
mapping(address => Voter) public voters;
```

Here, each address (Ethereum account) is mapped to a Voter structure. Mappings are very useful for contracts like **Ballot**, where you need to associate voters with their data (whether they voted, which proposal they chose, etc.). Unlike arrays, mappings do not have a length property and cannot be iterated over directly, making them **gas efficient** for lookups but

limited for enumeration.

- **storage:**

In Solidity, storage refers to the **permanent memory** of the contract, stored on the Ethereum blockchain. Variables declared at the contract level are stored in storage by default. Data stored in storage is persistent across transactions, which means once written, it remains available unless explicitly modified. However, because writing to blockchain storage consumes gas, it is more expensive. For example, a voter's information saved in the voters mapping remains available throughout the contract's lifecycle.

- **memory:**

In contrast, memory is **temporary storage**, used only for the lifetime of a function call. When the function execution ends, the data stored in memory is discarded. Memory is mainly used for temporary variables, function arguments, or computations that don't need to be permanently stored on the blockchain. It is cheaper than storage in terms of gas cost. For instance, when handling proposal names or temporary string manipulations, memory is often used.

Thus, a smart contract developer must **balance between storage and memory** to ensure efficiency and cost-effectiveness.

3. Why bytes32 Instead of string?

In earlier implementations of the Ballot contract, bytes32 was used for proposal names instead of string. The reason lies in **efficiency and gas optimization**.

- **bytes32** is a **fixed-size type**, meaning it always stores exactly 32 bytes of data. This makes storage simple, comparison operations faster, and gas costs lower. However, it limits proposal names to 32 characters, which is not very flexible for user-friendly names.
- **string** is a **dynamically sized type**, meaning it can store text of variable length. While it is easier for developers and users (since names can be written normally), it requires more complex handling inside the Ethereum Virtual Machine (EVM). This increases gas usage and may slow down comparisons or manipulations.

To make the system more user-friendly, modern implementations of the Ballot contract often convert from bytes32 to string. Tools like the **Web3 Type Converter** help developers easily switch between these two types for deployment and testing.

In summary, bytes32 is used when performance and gas efficiency are priorities, while

string is preferred for readability and ease of use.

Code:

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.20;

/**
 * @title Ballot
 * @dev Implements voting process along with vote delegation
 */
contract Ballot {

    struct Voter {
        uint weight;
        bool voted;
        address delegate;
        uint vote;
    }

    struct Proposal {
        string name;    //  store full string
        uint voteCount;
    }

    address public chairperson;

    mapping(address => Voter) public voters;

    Proposal[] public proposals;

    constructor(string[] memory proposalNames) {
        chairperson = msg.sender;
        voters[chairperson].weight = 1;

        for (uint i = 0; i < proposalNames.length; i++) {
            proposals.push(
                Proposal({
                    name: proposalNames[i],
                    voteCount: 0
                })
            )
        }
    }
}
```

```

    );
}
}

```

```

function giveRightToVote(address voter) external {
    require(msg.sender == chairperson, "Only chairperson can give right to vote.");
    require(!voters[voter].voted, "The voter already voted.");
    require(voters[voter].weight == 0, "Voter already has the right to vote.");

    voters[voter].weight = 1;
}

```

```

function delegate(address to) external {
    Voter storage sender = voters[msg.sender];

    require(sender.weight != 0, "You have no right to vote");
    require(!sender.voted, "You already voted.");
    require(to != msg.sender, "Self-delegation is disallowed.");

    while (voters[to].delegate != address(0)) {
        to = voters[to].delegate;
        require(to != msg.sender, "Found loop in delegation.");
    }

    Voter storage delegate_ = voters[to];
    require(delegate_.weight >= 1, "Delegate has no right to vote.");

    sender.voted = true;
    sender.delegate = to;

    if (delegate_.voted) {
        proposals[delegate_.vote].voteCount += sender.weight;
    } else {
        delegate_.weight += sender.weight;
    }
}

```

```

function vote(uint proposal) external {
    Voter storage sender = voters[msg.sender];

```

```

require(sender.weight != 0, "Has no right to vote");
require(!sender.voted, "Already voted.");
require(proposal < proposals.length, "Invalid proposal.");

sender.voted = true;
sender.vote = proposal;

proposals[proposal].voteCount += sender.weight;
}

function winningProposal() public view returns (uint winningProposal_) {
    uint winningVoteCount = 0;

    for (uint p = 0; p < proposals.length; p++) {
        if (proposals[p].voteCount > winningVoteCount) {
            winningVoteCount = proposals[p].voteCount;
            winningProposal_ = p;
        }
    }
}

function winnerName() external view returns (string memory) {
    return proposals[winningProposal()].name;
}
}

```

Implementation:

Deployed

Chairperson gave right to vote to 0x4B20993Bc481177ec7E8f571ceCaE8A9e22C02db

DEPLOY & RUN
TRANSACTIONS

▼ BALLOT AT 0XD4F...2CBEE (MEM)

Balance: 0 ETH

delegate

address to

▼

giveRightToVote

0x4B20993Bc481177ec7E8f5

▼

vote

0

▼

chairperson

proposals

uint256

▼

voters

address

▼

winnerName

0: string: Rujuta

winningProposal

0

☐ Listen on all transactions

🔍

🔊

⌵

✕

✓ [vm] from: 0x5B3...eddC4
to: Ballot.giveRightToVote(address) 0xD4F...2cbee value: 0 wei
data: 0x9e7...c02db logs: 0 hash: 0x4e4...774ea

Debug

⌵

status

1 Transaction mined and execution succeed

transaction hash

0x4e4a9075b24dfb3cadb44eb43911c717e85d545f4c78a22b0c897de13c9774ea

🔗

block hash

0x1470f032e9286b23fed1fa5a0f5fe06cba1d44427fb6c8ceb7108d6e9c07fcab

🔗

block number

29

🔗

from

0x5B38Da6a701c568545dCfcB03Fc8875f56beddC4

🔗

to

Ballot.giveRightToVote(address)
0xD4Fc541236927E2EAF8F27606b07309C1Fc2cbee

🔗

transaction cost

48642 gas

🔗

execution cost

27210 gas

🔗

output

0x

🔗

decoded input

{
 "address voter": "0x4B209938c481177ec7E8F571ceCaE8A9e22C02db"
}

🔗

Gave vote to person at index 1

DEPLOY & RUN TRANSACTIONS

BALLOT AT 0XD4F...2CBEE (MEV)

Balance: 0 ETH

delegate

address to

giveRightToVote

0x4B209938c481177ec7E8f5

vote

1

chairperson

proposals

uint256

voters

address

winnerName

0: string: Rujuta

winningProposal

Low level interactions

CALLDATA

0 ☐ Listen on all transactions

Filter with transaction hash or ad...

transact to Ballot.vote pending ...

✓

[vm] from: 0x5B3...eddC4 to: Ballot.vote(uint256) 0xD4F...2cbee value: 0 wei data: 0x012...00001 logs: 0 hash: 0x4d6...d164a

Debug

status

1 Transaction mined and execution succeed

transaction hash

0x4d621d2881c22a17ab6a29a360c0837fdad96f7eb1b21e354c4316af5b9d164a

block hash

0x235420947c64ac48179e799be2779a663f3d4eab5af10a84dd0dd57c9b08e7c

block number

30

from

0x5B38Da6a701c568545dCfc8B3Fcb875f56beddC4

to

Ballot.vote(uint256) 0xD4Fc541236927E2EAF8F27606b07309C1Fc2cbee

transaction cost

93027 gas

execution cost

71823 gas

output

0x

decoded input

{ "uint256 proposal": "1" }

decoded output

{}

Chairman gave right to vote to 0x78731D3Ca6b7E34aC0F824c42a7cC18A495cabaB

Deployed Contracts

BALLOT AT 0XD4F...2CBEE (MEV)

Balance: 0 ETH

delegate

address to

giveRightToVote

0x78731D3Ca6b7E34aC0F82

vote

1

chairperson

proposals

uint256

voters

address

winnerName

0: string: Rujuta

winningProposal

transact to Ballot.giveRightToVote pending ...

✓

[vm] from: 0x5B3...eddC4 to: Ballot.giveRightToVote(address) 0xD4F...2cbee value: 0 wei data: 0x9e7...cabab logs: 0 hash: 0x1c6...38563

Debug

status

1 Transaction mined and execution succeed

transaction hash

0x1c66389537780983d242e4df719Fdbd7a71911684f64ba33fa5a62dd31f38563

block hash

0xf37a77ace0680d51c7f56dea83b931012eb5d025f8037Fecca22bdb604014bd

block number

32

from

0x5B38Da6a701c568545dCfc8B3Fcb875f56beddC4

to

Ballot.giveRightToVote(address) 0xD4Fc541236927E2EAF8F27606b07309C1Fc2cbee

transaction cost

48642 gas

execution cost

27210 gas

output

0x

Gave vote to person at index 1

delegate address to

giveRightToVote 0x78731D3Ca6b7E34aC0F82

vote 1

chairperson

proposals uint256

voters address

winnerName

0: string: Rujata

winningProposal

```

logs
raw logs
[]

transaction to Ballot.vote pending ...

[vm] from: 0x787...cabaB to: Ballot.vote(uint256) 0xD4F...2cbee
value: 0 wei data: 0x012...0001 logs: 0 hash: 0x94f...81b20

status 1 Transaction mined and execution succeed
transaction hash 0x94fb2c75b57e29dcff2e911d82096d0b9bcad778d8174609b7ec2cf581481b20

block hash 0x34e79f4ebe4d2b7fba94e1c3393bf9db3fd379d2f4684d39f0b46e22d51b9c0b

block number 33

from 0x78731D3Ca6b7E34aC0F824c42a7c18A495cabaB

to Ballot.vote(uint256) 0xD4Fc541236927E2EAf8F27606bD7309C1Fc2cbee

```

Now winner is person at index 1

[illegible]

Conclusion:

In this experiment, a Voting/Ballot Smart Contract was successfully deployed and tested. The chairperson granted voting rights, voters cast their votes, and the contract correctly determined the winner based on vote count. The use of `require` statements ensured security and prevented invalid actions such as double voting. This experiment demonstrated how blockchain technology enables secure, transparent, and reliable voting using smart contracts.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 06

Aim: Creating Smart Contracts and performing transactions using Solidity and Remix IDE

Roll No.	33
Name	Rujuta Vinit Medhi
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5:Interpret different Crypto assets and Crypto currencies



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

**Rujuta Medhi
D20A - 33**

Experiment: 06

Aim: Creating Smart Contracts and performing transactions using Solidity and Remix IDE

Theory:

A Smart Contract is a self-executing program stored on the blockchain that automatically enforces the terms of an agreement when predefined conditions are met. It eliminates the need for intermediaries, ensuring transparency, security, and trust between parties. Smart contracts are written in Solidity (for Ethereum-based systems) and executed on the Ethereum Virtual Machine (EVM).

Significance of Smart Contracts in the Ethereum Blockchain:

- Automation: Executes actions automatically when conditions are satisfied.
- Transparency: The contract code and transactions are visible on the blockchain.
- Security: Once deployed, contracts are immutable and tamper-proof.
- Cost-effective: Removes intermediaries and reduces transaction costs.
- Trustless System: Participants can interact without needing to trust each other directly.

Smart contracts form the backbone of decentralized applications (DApps) and decentralized finance (DeFi) systems on Ethereum.

Code:

TrustScore.sol

```
// SPDX-License-Identifier: MIT
```

```
pragma solidity ^0.8.20;
```

```
contract TrustScore {  
    // Store trust scores for each user  
    mapping(address => uint256) public trustScores;  
    address public admin;  
    address public loanManagerAddress;  
    uint256 public initialScore = 50; // Initial score for new users  
  
    constructor() {  
        admin = msg.sender;  
    }  
}
```



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

```
// Set LoanManager address (admin only)
function setLoanManagerAddress(address _loanManagerAddress) public {
    require(msg.sender == admin, "Only admin");
    loanManagerAddress = _loanManagerAddress;
}

// Admin or LoanManager can update scores
function updateScore(address user, uint256 score) public {
    require(msg.sender == admin || msg.sender == loanManagerAddress, "Only admin or
LoanManager");
    require(score <= 100, "Score must be 0-100");
    trustScores[user] = score;
}

// Set initial score for new users (admin only)
function setInitialScore(uint256 _initialScore) public {
    require(msg.sender == admin, "Only admin");
    require(_initialScore <= 100, "Score must be 0-100");
    initialScore = _initialScore;
}

// Anyone can check a score - returns initialScore if not set
function getScore(address user) public view returns (uint256) {
    uint256 score = trustScores[user];
    // If score is 0, return initial score (user hasn't been explicitly set)
    return score > 0 ? score : initialScore;
}
}
```

Deployed TrustScore.sol



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26



LoanManager.sol

// SPDX-License-Identifier: MIT

pragma solidity ^0.8.20;

```
interface ITrustScore {  
    function getScore(address user) external view returns (uint256);  
    function updateScore(address user, uint256 score) external;  
}
```

```
contract LoanManager {
```

```
    ITrustScore public trustScoreContract;  
    address public admin;  
    uint256 public minTrustScore;
```

```
    // Trust score rewards/penalties
```

```
    uint256 public repaymentBonus = 5;    // +5 points for successful repayment
```

```
    uint256 public defaultPenalty = 10;    // -10 points for defaulted loan
```

```
    // Safety pool address for collecting fees
```

```
    address public safetyPoolAddress;
```

```
    constructor(address _trustScoreAddress) {  
        trustScoreContract = ITrustScore(_trustScoreAddress);  
    }
```




**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

```
admin = msg.sender;
minTrustScore = 40; // Default minimum trust score
}

// Set safety pool address (admin only)
function setSafetyPoolAddress(address _safetyPoolAddress) public {
    require(msg.sender == admin, "Only admin");
    safetyPoolAddress = _safetyPoolAddress;
}

struct Loan {
    address borrower;
    address lender;
    uint256 amount;
    uint256 interestRate; // Interest rate in basis points (e.g., 500 = 5%)
    uint256 safetyFee;    // Safety fee in basis points
    uint256 duration;
    uint256 createdAt;
    bool funded;
    bool repaid;
    bool withdrawn;
    bool defaulted;
}

Loan[] public loans;

event LoanRequested(uint256 loanId, address borrower, uint256 amount, uint256
interestRate);
event LoanFunded(uint256 loanId, address lender);
event LoanWithdrawn(uint256 loanId);
event LoanRepaid(uint256 loanId, uint256 totalRepaid, uint256 interest);
event LoanDefaulted(uint256 loanId, address borrower);
event TrustScoreIncreased(address borrower, uint256 amount);
event TrustScoreDecreased(address borrower, uint256 amount);
event MinTrustScoreUpdated(uint256 newScore);

// Calculate interest rate based on trust score
// Higher trust score = lower interest rate
```



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

```
function getInterestRate(uint256 trustScore) public pure returns (uint256) {
    if (trustScore >= 90) return 300;    // 3% for excellent credit
    if (trustScore >= 75) return 500;    // 5% for good credit
    if (trustScore >= 60) return 800;    // 8% for fair credit
    if (trustScore >= 50) return 1200;   // 12% for poor credit
    return 1500;                        // 15% for very poor credit
}

// Get maximum borrow limit based on trust score (in wei)
// Higher trust = larger loans allowed
function getMaxBorrowLimit(uint256 trustScore) public pure returns (uint256) {
    if (trustScore >= 90) return 10 ether; // Excellent: up to 10 ETH
    if (trustScore >= 75) return 5 ether;  // Good: up to 5 ETH
    if (trustScore >= 60) return 2 ether;  // Fair: up to 2 ETH
    if (trustScore >= 50) return 0.5 ether; // Poor: up to 0.5 ETH
    return 0.1 ether;                     // Very poor: up to 0.1 ETH
}

// Get safety fee based on trust score (in basis points)
// Lower trust = higher safety fee to protect lenders
function getSafetyFee(uint256 trustScore) public pure returns (uint256) {
    if (trustScore >= 90) return 50;    // 0.5% for excellent credit
    if (trustScore >= 75) return 100;    // 1% for good credit
    if (trustScore >= 60) return 200;    // 2% for fair credit
    if (trustScore >= 50) return 400;    // 4% for poor credit
    return 600;                        // 6% for very poor credit
}

// Get borrower's current limit info
function getBorrowerLimits(address borrower) public view returns (
    uint256 trustScore,
    uint256 maxBorrowLimit,
    uint256 interestRate,
    uint256 safetyFee
) {
    trustScore = trustScoreContract.getScore(borrower);
    maxBorrowLimit = getMaxBorrowLimit(trustScore);
    interestRate = getInterestRate(trustScore);
}
```



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

```
safetyFee = getSafetyFee(trustScore);
}

// Calculate total repayment amount (principal + interest + safety fee)
function calculateRepayment(uint256 loanId) public view returns (uint256) {
    Loan memory loan = loans[loanId];
    uint256 interest = (loan.amount * loan.interestRate) / 10000;
    uint256 safetyFeeAmount = (loan.amount * loan.safetyFee) / 10000;
    return loan.amount + interest + safetyFeeAmount;
}

// Get breakdown of repayment components
function getRepaymentBreakdown(uint256 loanId) public view returns (
    uint256 principal,
    uint256 interest,
    uint256 safetyFeeAmount,
    uint256 total
) {
    Loan memory loan = loans[loanId];
    principal = loan.amount;
    interest = (loan.amount * loan.interestRate) / 10000;
    safetyFeeAmount = (loan.amount * loan.safetyFee) / 10000;
    total = principal + interest + safetyFeeAmount;
}

// Admin function to update minimum trust score
function setMinTrustScore(uint256 _minScore) public {
    require(msg.sender == admin, "Only admin can set minimum trust score");
    require(_minScore <= 100, "Score must be 0-100");
    minTrustScore = _minScore;
    emit MinTrustScoreUpdated(_minScore);
}

// Set trust score reward for repayment
function setRepaymentBonus(uint256 _bonus) public {
    require(msg.sender == admin, "Only admin");
    repaymentBonus = _bonus;
}
```



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

```
// Set trust score penalty for default
function setDefaultPenalty(uint256 _penalty) public {
    require(msg.sender == admin, "Only admin");
    defaultPenalty = _penalty;
}

// Check if a loan is overdue
function isLoanOverdue(uint256 loanId) public view returns (bool) {
    Loan memory loan = loans[loanId];
    if (!loan.withdrawn || loan.repaid || loan.defaulted) {
        return false;
    }
    return block.timestamp > loan.createdAt + loan.duration;
}

// Mark loan as defaulted (called by admin or lender)
function markLoanDefaulted(uint256 loanId) public {
    Loan storage loan = loans[loanId];
    require(!loan.defaulted && !loan.repaid, "Loan already finalized");
    require(isLoanOverdue(loanId), "Loan is not overdue");
    require(msg.sender == loan.lender || msg.sender == admin, "Only lender or admin can mark
default");

    loan.defaulted = true;

    // Decrease borrower's trust score
    uint256 currentScore = trustScoreContract.getScore(loan.borrower);
    uint256 newScore = currentScore > defaultPenalty ? currentScore - defaultPenalty : 0;
    trustScoreContract.updateScore(loan.borrower, newScore);

    emit LoanDefaulted(loanId, loan.borrower);
    emit TrustScoreDecreased(loan.borrower, defaultPenalty);
}

// -----
// BORROWER CREATES LOAN REQUEST
// -----
```



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

```
function createLoan(uint256 amount, uint256 duration) public {

    uint256 score = trustScoreContract.getScore(msg.sender);
    require(score >= minTrustScore, "Trust score too low");

    // Enforce maximum borrow limit based on trust score
    uint256 maxLimit = getMaxBorrowLimit(score);
    require(amount <= maxLimit, "Amount exceeds your borrow limit");

    uint256 interestRate = getInterestRate(score);
    uint256 safetyFee = getSafetyFee(score);

    loans.push(
        Loan({
            borrower: msg.sender,
            lender: address(0),
            amount: amount,
            interestRate: interestRate,
            safetyFee: safetyFee,
            duration: duration,
            createdAt: block.timestamp,
            funded: false,
            repaid: false,
            withdrawn: false,
            defaulted: false
        })
    );

    emit LoanRequested(loans.length - 1, msg.sender, amount, interestRate);
}

// -----
// LENDER FUNDS LOAN
// -----
function fundLoan(uint256 loanId) public payable {

    Loan storage loan = loans[loanId];
```



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

```
require(!loan.funded, "Already funded");
require(msg.value == loan.amount, "Incorrect ETH amount");

loan.lender = msg.sender;
loan.funded = true;

emit LoanFunded(loanId, msg.sender);
}

// -----
// BORROWER WITHDRAWS FUNDS
// -----
function withdrawLoan(uint256 loanId) public {

    Loan storage loan = loans[loanId];

    require(msg.sender == loan.borrower, "Not borrower");
    require(loan.funded, "Loan not funded");
    require(!loan.withdrawn, "Already withdrawn");

    loan.withdrawn = true;

    payable(msg.sender).transfer(loan.amount);

    emit LoanWithdrawn(loanId);
}

// -----
// BORROWER REPAYS LOAN
// -----
function repayLoan(uint256 loanId) public payable {

    Loan storage loan = loans[loanId];

    require(msg.sender == loan.borrower, "Not borrower");
    require(loan.withdrawn, "Loan not withdrawn");
    require(!loan.repaid, "Already repaid");
    require(!loan.defaulted, "Loan is defaulted");
```



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

```
require(loan.lender != address(0), "Invalid lender address");

// Get repayment breakdown
(uint256 principal, uint256 interest, uint256 safetyFeeAmount, uint256 totalRepayment) =
getRepaymentBreakdown(loanId);
require(msg.value >= totalRepayment, "Insufficient repayment amount");

// Transfer principal + interest to lender
uint256 lenderAmount = principal + interest;
payable(loan.lender).transfer(lenderAmount);

// Transfer safety fee to safety pool (if set)
if (safetyPoolAddress != address(0) && safetyFeeAmount > 0) {
    payable(safetyPoolAddress).transfer(safetyFeeAmount);
}

loan.repaid = true;

// Increase borrower's trust score for successful repayment
uint256 currentScore = trustScoreContract.getScore(loan.borrower);
uint256 newScore = currentScore + repaymentBonus;
// Cap at 100
if (newScore > 100) newScore = 100;
trustScoreContract.updateScore(loan.borrower, newScore);

emit LoanRepaid(loanId, totalRepayment, interest);
emit TrustScoreIncreased(loan.borrower, repaymentBonus);

// Return excess payment if any
if (msg.value > totalRepayment) {
    payable(msg.sender).transfer(msg.value - totalRepayment);
}
}

// -----
// GET ALL LOANS
// -----
function getLoans() public view returns (Loan[] memory) {
```



VIVEKANAND EDUCATION SOCIETY'S INSTITUTE OF TECHNOLOGY

Department of Information Technology

Academic Year: 2025-26

return loans;

```
}  
}  
}
```

REMIX 1.5.1

default_workspace

Login with GitHub Theme

DEPLOY & RUN TRANSACTIONS

ACCOUNT +

0xAb8...35cb2 (99.9999999999)

+ Authorize Delegation

GAS LIMIT

Estimated Gas

Custom 3000000

VALUE

0 Wei

CONTRACT

LoanManager - contracts/LoanMan...

evm version: osaka

Deploy 0xAb84B3F6d9C5d1EcF9b849f

At Address Load contract from Address

Welcome to Remix 1.5.1

Your files are stored in indexedDB, 2.64 KB / 142.58 GB used

You can use this terminal to:

- Check transactions details and start debugging.
- Execute JavaScript scripts:
 - Input a script directly in the command line interface
 - Select a Javascript file in the file explorer and then run 'remix.execute()' or 'remix.executeCurrent()' in the command line interface
 - Right-click on a JavaScript file in the file explorer and then click 'Run'

The following libraries are accessible:

- ethers.js

Type the library name to see available commands.

creation of TrustScore pending...

[vm] from: 0x583...eddC4 to: TrustScore.(constructor) value: 0 wei data: 0x608...f0033 logs: 0 hash: 0xc13...5053a Debug

creation of TrustScore pending...

[vm] from: 0x583...eddC4 to: TrustScore.(constructor) value: 0 wei data: 0x608...f0033 logs: 0 hash: 0xf1b...f3325 Debug

creation of LoanManager pending...

[vm] from: 0xAb8...35cb2 to: LoanManager.(constructor) value: 0 wei data: 0x608...35cb2 logs: 0 hash: 0x4b4...1ff34 Debug

Activate Windows
Go to Settings to activate Windows.

Scam Alert Initialize as git repo

Did you know? You can use 'Explain this function' in the right-click menu to receive an explanation from AI.

RemixAI Copilot (enabled)

USD/INR +0.24%

Search

ENG US 11:29 17-03-2026

TrustScore.sol

REMIX 1.5.1

default_workspace

Login with GitHub Theme

DEPLOY & RUN TRANSACTIONS

TRUSTSCORE AT 0XD8B...33FA

Balance: 0 ETH

setInitialScore 30

setLoanManag... 0xa131AD247055FD2e2aA...

updateScore updateScore - transact (not payable)

admin

0: address: 0x5838Da6a701c568545dCfC803FcB875f56beddC4

getScore address user

initialScore

0: uint256: 30

loanManagerA...

0: address: 0xa131AD247055FD2e2aA8b156A11bdEc81b9eAD95

trustScores address

transact to TrustScore.setInitialScore pending ...

[vm] from: 0x583...eddC4 to: TrustScore.setInitialScore(uint256) 0xd8b...33fa8 value: 0 wei data: 0x914...0001e logs: 0 hash: 0xdfb...06353 Debug

call to TrustScore.initialScore

CALL [call] from: 0x5838Da6a701c568545dCfC803FcB875f56beddC4 to: TrustScore.initialScore() data: 0x541...e14cd Debug

call to TrustScore.admin

CALL [call] from: 0x5838Da6a701c568545dCfC803FcB875f56beddC4 to: TrustScore.admin() data: 0xf85...1a440 Debug

transact to TrustScore.setLoanManagerAddress pending ...

[vm] from: 0x583...eddC4 to: TrustScore.setLoanManagerAddress(address) 0xd8b...33fa8 value: 0 wei data: 0xda0...ead95 logs: 0 hash: 0xce2...5d780 Debug

call to TrustScore.loanManagerAddress

CALL [call] from: 0x5838Da6a701c568545dCfC803FcB875f56beddC4 to: TrustScore.loanManagerAddress() data: 0x40e...efcbf Debug

transact to TrustScore.updateScore pending ...

[vm] from: 0x583...eddC4 to: TrustScore.updateScore(address,uint256) 0xd8b...33fa8 value: 0 wei data: 0x65d...00032 logs: 0 hash: 0x466...9d426 Debug

Activate Windows
Go to Settings to activate Windows.

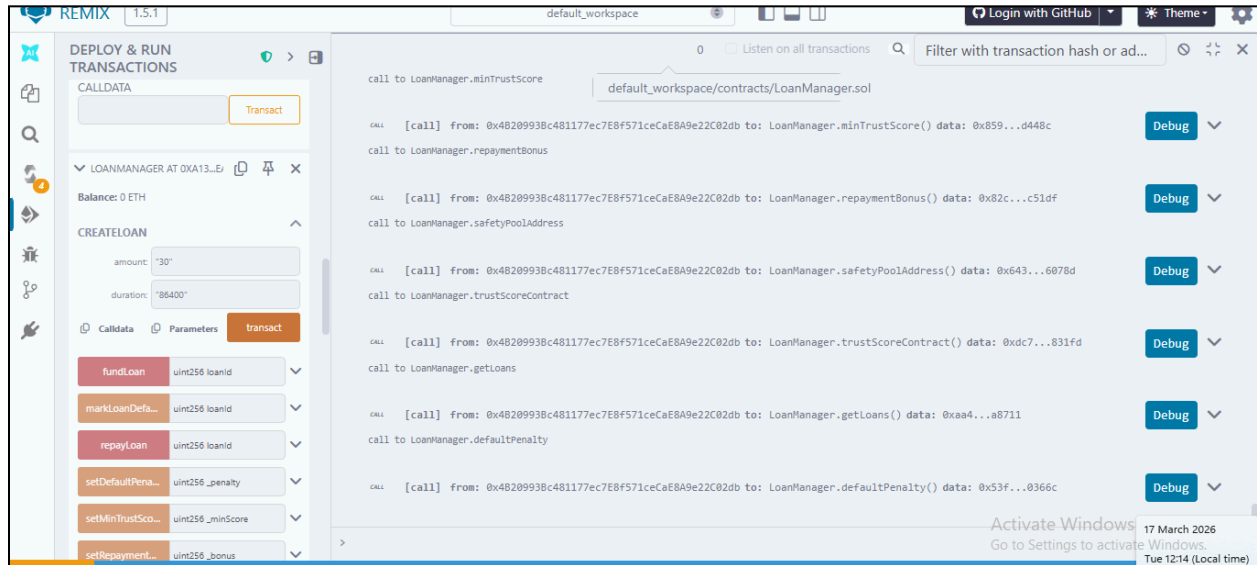
Scam Alert Initialize as git repo

Did you know? You can use 'Explain this function' in the right-click menu to receive an explanation from AI.

RemixAI Copilot (enabled)



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26



Conclusion:

This experiment helped in understanding how smart contracts automate processes and execute securely on the Ethereum blockchain. Using Solidity and Remix IDE, the development, deployment, and interaction with smart contracts become simple and efficient, forming the foundation for decentralized applications.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26
ITC801 : Blockchain & DLT Lab

Experiment 07

Aim: Implement the embedding wallet (Metamask) and transaction using Solidity

Roll No.	33
Name	Rujuta Vinit Medhi
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5:Interpret different Crypto assets and Crypto currencies

Experiment: 07

Aim: Implement the embedding wallet (Metamask) and transaction using Solidity

Theory:

MetaMask is a **cryptocurrency wallet and gateway** to blockchain applications. It allows users to store, send, and receive Ether or tokens, and connect to decentralized applications (DApps) directly from a web browser. MetaMask acts as a **bridge** between a user's browser and the Ethereum blockchain (or other compatible networks).

Features:

- Easy management of multiple blockchain accounts
- Secure private key storage
- Integration with test networks like **Sepolia**, **Linea**, or **RSK**
- Interaction with DApps and smart contracts via **Remix IDE**

Testnet:

A Testnet is a blockchain network used for testing and development without using real cryptocurrency. It mimics the main Ethereum network but uses test Ether (fake ETH). Common testnets include Sepolia Testnet, Linea Testnet, and RSK Testnet.

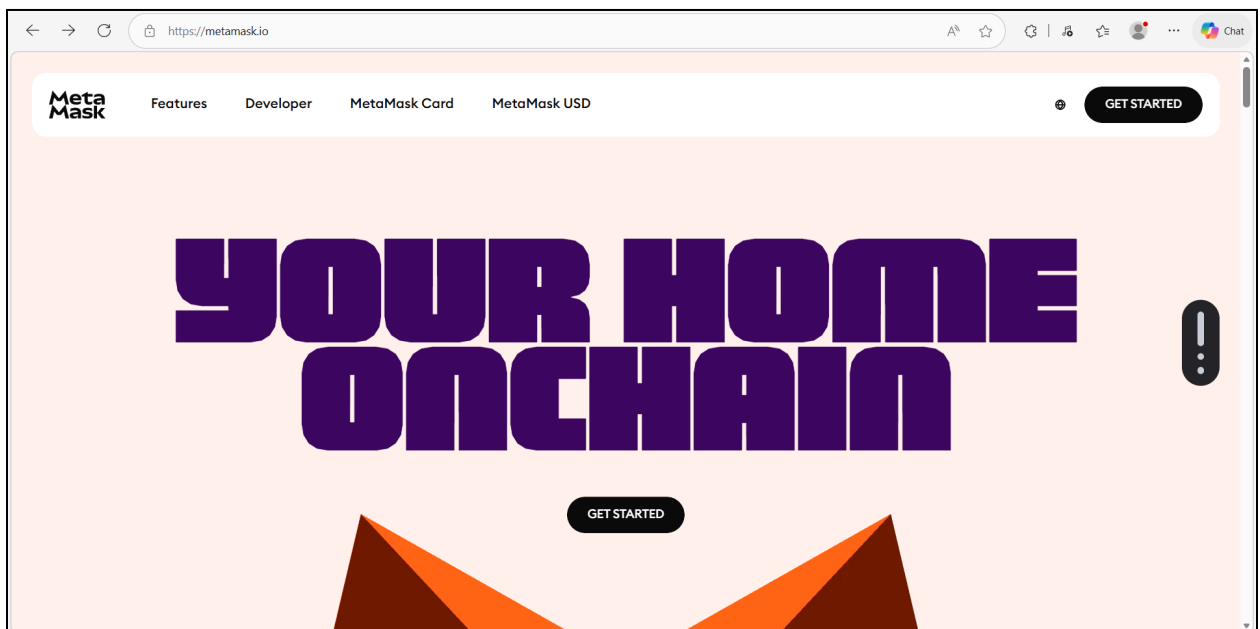
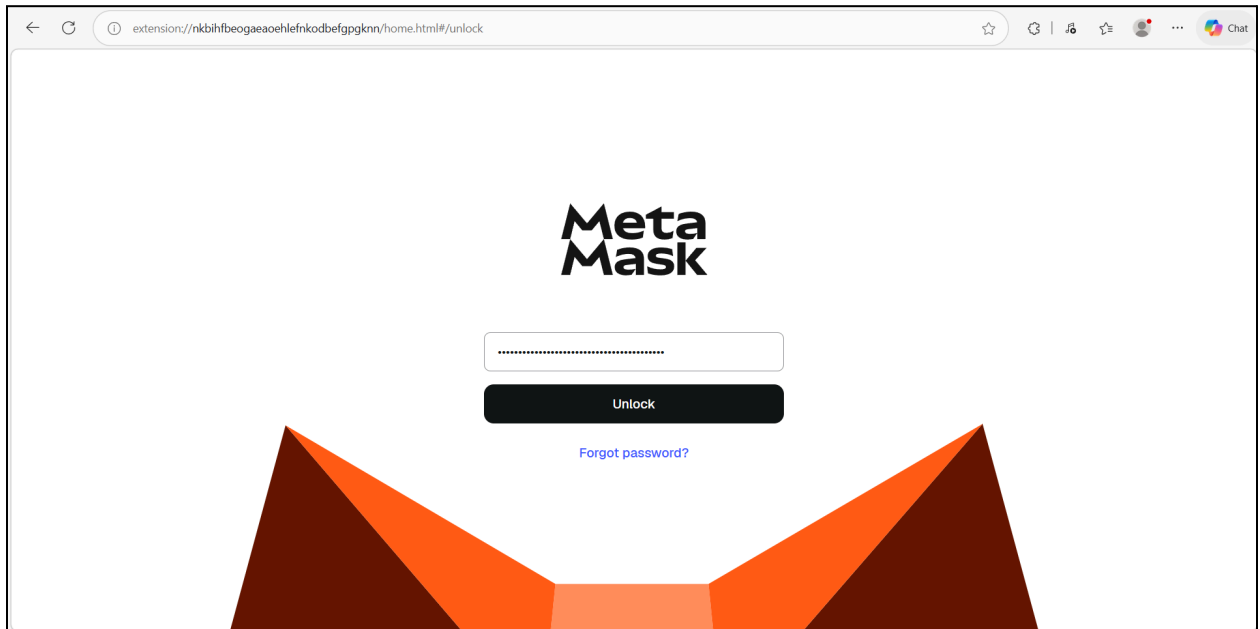
Advantages:

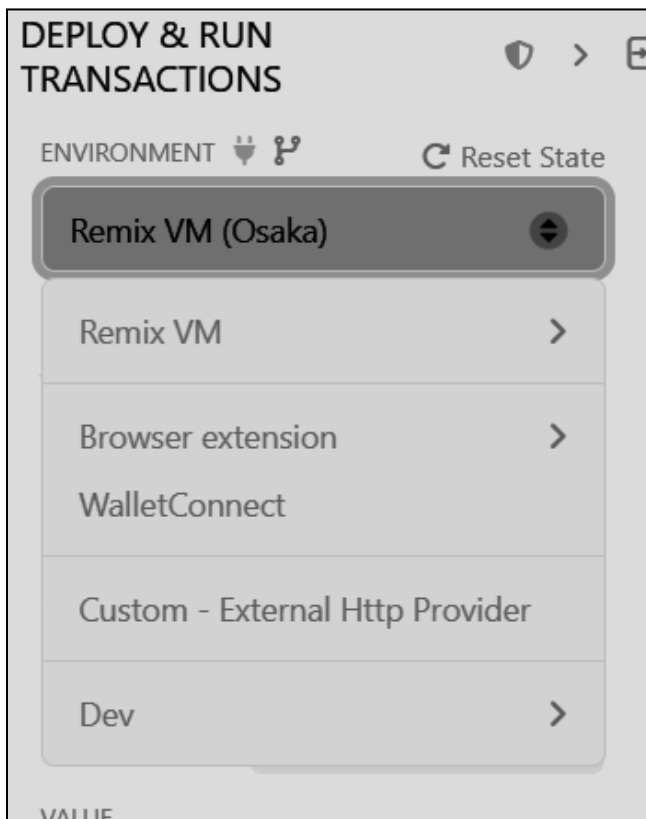
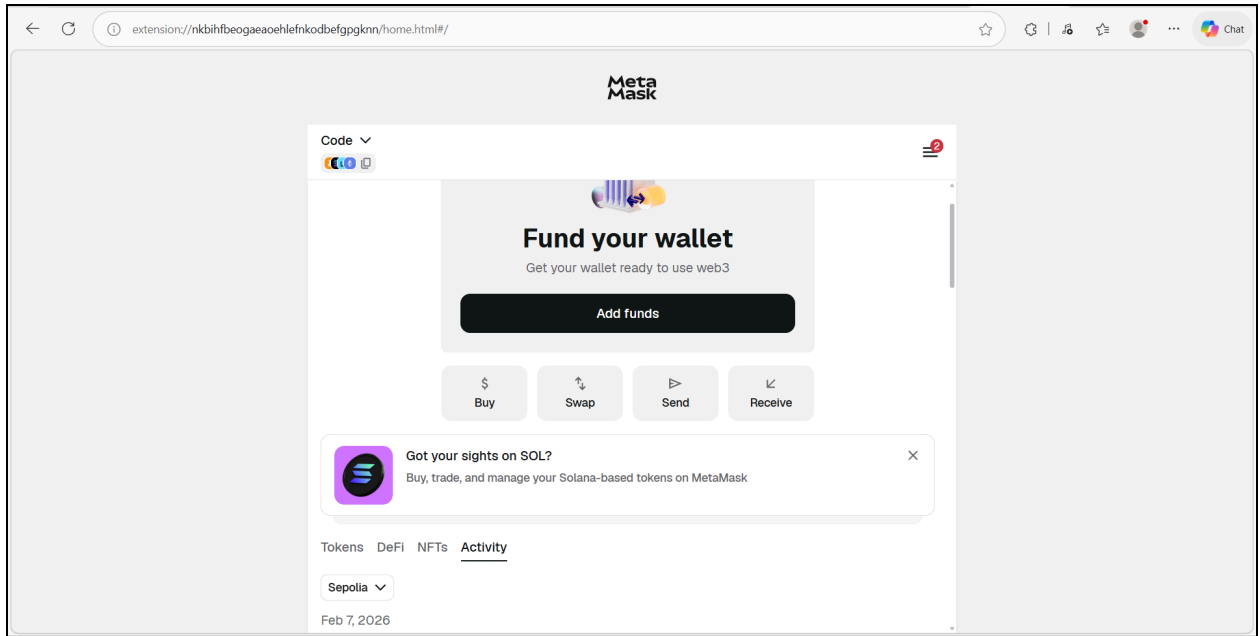
- No real money loss during testing
- Safe environment for developers
- Helps test smart contracts and transactions before mainnet deployment

Steps to Connect MetaMask with Remix IDE for Performing Transactions:

1. Set Up MetaMask:
 - Install the MetaMask extension on your browser.
 - Create or import a wallet account using a recovery phrase.
2. Add a Test Network:
 - Open MetaMask → Settings → Networks → Add Network.
 - Enter details for the desired Testnet (e.g., Sepolia or RSK).
 - Example RPC URL: `https://sepolia.infura.io/v3/<your_project_id>`
3. Fund the Wallet:
 - Use a faucet to receive free test Ether (e.g., 0.5 ETH/day).
 - Check the balance in MetaMask.
4. Connect MetaMask with Remix IDE:
 - Open Remix IDE.

- In the Deploy & Run Transactions tab, choose Injected Web3 as the environment.
- Approve connection in MetaMask.





DEPLOY & RUN
TRANSACTIONS

ENVIRONMENT 

WalletConnect  

Sepolia (11155111) network

ACCOUNT   

0x1b4...0f039 (0.16127144765€) 

+ Create Smart Account

GAS LIMIT

☒ Estimated Gas

☐ Custom 3000000

VALUE

0 Wei

Sepolia test network connected to RemixIDE using metamask

Sepolia Faucet Mining

←

↻

🔒

https://sepolia-faucet.pk910.de/#/mine/38f0cf0d-1619-40dc-8516-f509dfca9ebb

☆

PoWFaucet

Hoodi ▾

Holešky ▾

Sepolia ▾

Ephemery ▾

Devconnect 2025 ▾

Sepolia PoW Faucet

✕



Target Address:

0x1b46C1954361A8cB698711426ffa4f0B1b0f039

Your Mining Reward:

0.012 SepETH

Current Hashrate:

85.67 H/s

Number of Workers:

8 / 8

+

-

Minimum Claim Reward:

0.05 SepETH

Maximum Claim Reward:

2.5 SepETH

Remaining Session Time:

11h 58min

Total Shares:

1

Avg. Reward per Hour:

2.562 SepETH/h

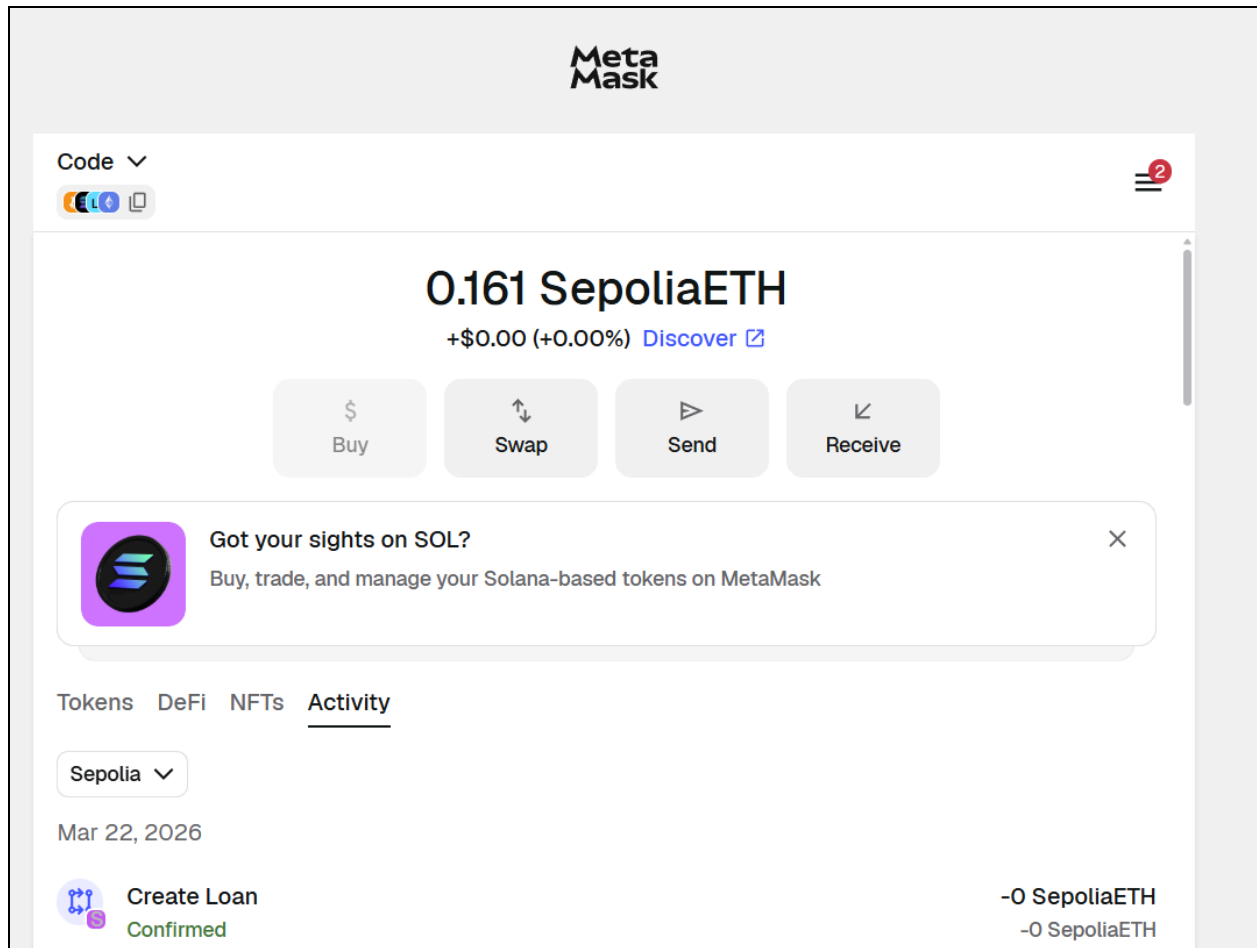
Reward Boost:

+ 0%

Boost

Stop Mining

Account balance



TrustScore.sol

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

```
contract TrustScore {  
    // Store trust scores for each user  
    mapping(address => uint256) public trustScores;  
    address public admin;  
    address public loanManagerAddress;  
    uint256 public initialScore = 50; // Initial score for new users  
  
    constructor() {  
        admin = msg.sender;  
    }  
  
    // Set LoanManager address (admin only)
```



```

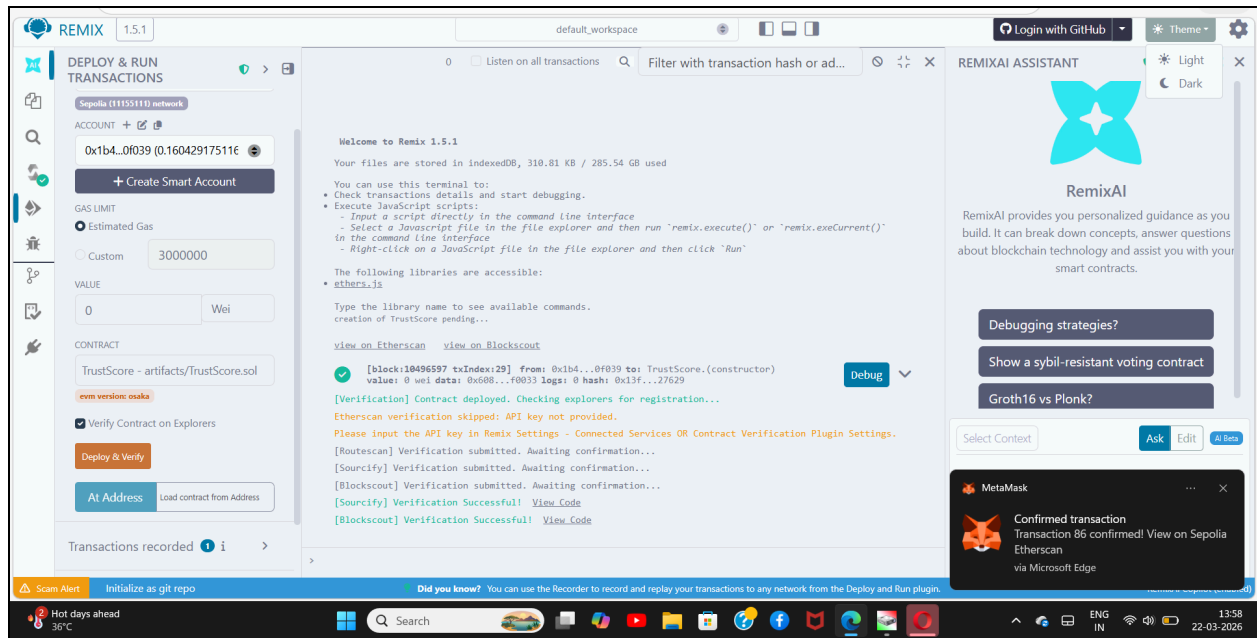
function setLoanManagerAddress(address _loanManagerAddress) public {
    require(msg.sender == admin, "Only admin");
    loanManagerAddress = _loanManagerAddress;
}

// Admin or LoanManager can update scores
function updateScore(address user, uint256 score) public {
    require(msg.sender == admin || msg.sender == loanManagerAddress, "Only admin or
LoanManager");
    require(score <= 100, "Score must be 0-100");
    trustScores[user] = score;
}

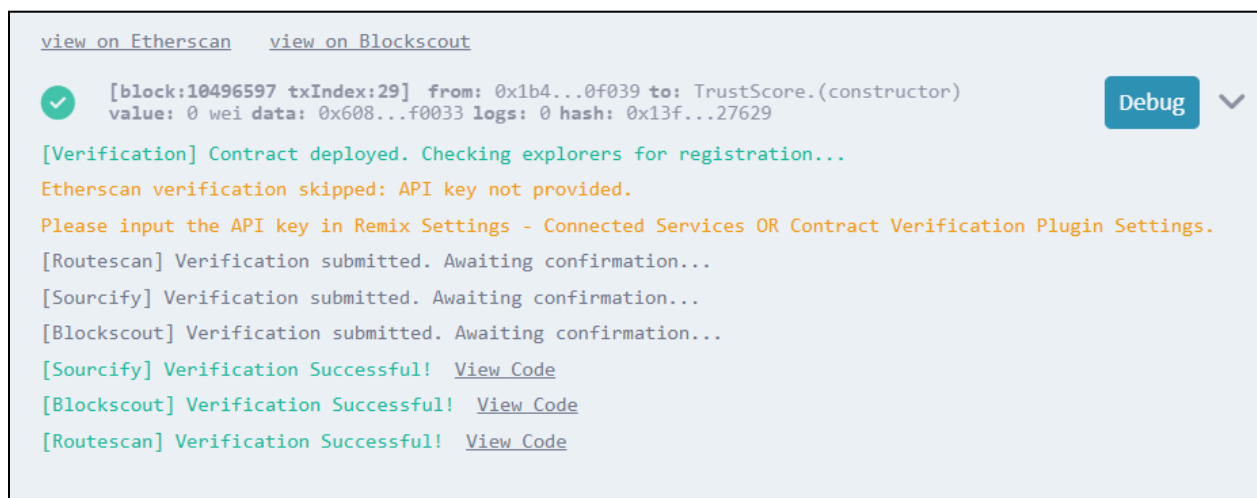
// Set initial score for new users (admin only)
function setInitialScore(uint256 _initialScore) public {
    require(msg.sender == admin, "Only admin");
    require(_initialScore <= 100, "Score must be 0-100");
    initialScore = _initialScore;
}

// Anyone can check a score - returns initialScore if not set
function getScore(address user) public view returns (uint256) {
    uint256 score = trustScores[user];
    // If score is 0, return initial score (user hasn't been explicitly set)
    return score > 0 ? score : initialScore;
}
}

```



Contract Deployed:



Transaction request

REMIX 1.5.1

default_workspace

DEPLOY & RUN TRANSACTIONS

Transactions recorded 2 i

Deployed Contracts 1

TRUSTSCORE AT 0XBBD...ESE25

Balance: 0 ETH

SETINITIALSCORE

initialScore: 10

Calldata Parameters transact

setLoanManag... address_loanManagerAddr

updateScore address user, uint256 score

admin

getScore address user

inituScore

loanManagerA...

trustScores address

Welcome to Remix 1.5.1

Your files are stored in indexedDB, 310.81 KB / 285.54 GB used

You can use this terminal to:

- Check transactions details and start debugging.
- Execute JavaScript scripts:
 - Input a script directly in the command line interface
 - Select a Javascript file in the file explorer and then run "remix.execute()" or "remix.executeCurrent()" in the command line interface
 - Right-click on a JavaScript file in the file explorer and then click "Run"
- The following libraries are accessible:
 - ethers.js
- Type the library name to see available commands.
creation of TrustScore pending...

view_on_Etherscan view_on_Blockscout

[block:10496597 txIndex:29] from: 0x1b4...0f039 to: TrustScore.(constructor)
value: 0 wei data: 0x6088...f0033 logs: 0 hash: 0x13f...27629

Debug

[Verification] Contract deployed. Checking explorers for registration...

Etherscan verification skipped: API key not provided.

Please input the API key in Remix Settings - Connected Services OR Contract Verification Plugin Settings.

[Routescan] Verification submitted. Awaiting confirmation...

[Sourcify] Verification submitted. Awaiting confirmation...

[Blockscout] Verification submitted. Awaiting confirmation...

[Sourcify] Verification Successful! View Code

[Blockscout] Verification Successful! View Code

[Routescan] Verification Successful! View Code

transact to TrustScore.setInitialScore pending ...

Transaction request

Estimated changes No changes

Network s Sepolia

Request from remix.ethereum.org

Interacting with 0xbD4b...e5e29

Network fee 0 s SepoliaETH

Speed Market ~12 sec

Cancel Confirm

Select Context Ask Edit AI Bes

Select context and ask me anything!

MistralAI Audio Prompt Create new workspace with AI

Scam Alert Initialize as git repo

Did you know? You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin.

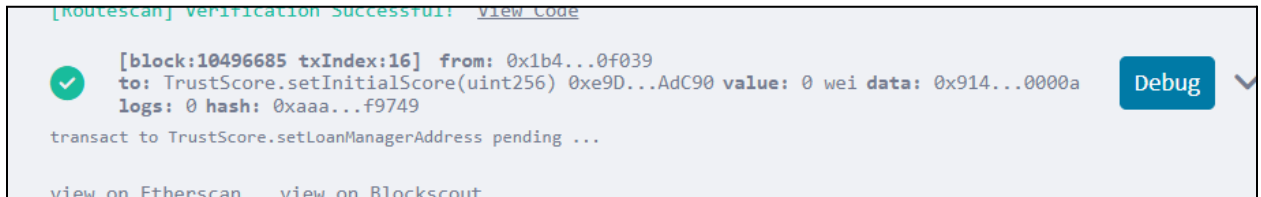
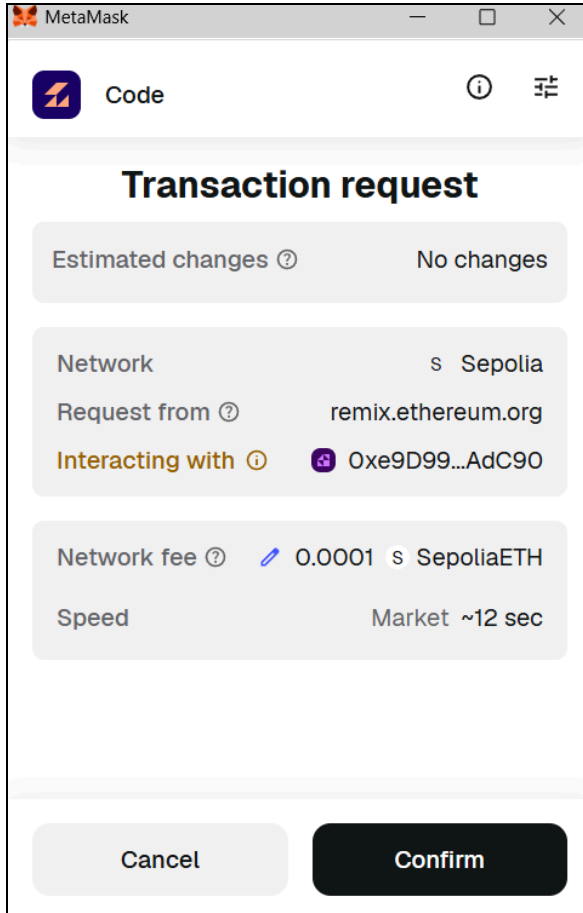
RemixAI Copilot (enabled)

36°C Sunny

Search

14:02 22-03-2026

Confirmation on Metamask



Contract details on Metmask:

Mar 22, 2026



Update Score

Confirmed

-0 SepoliaETH

-0 SepoliaETH



Contract interaction

Confirmed

-0 SepoliaETH

-0 SepoliaETH



Contract interaction

Confirmed

-0 SepoliaETH

-0 SepoliaETH



Contract deployment

Confirmed

-0 SepoliaETH

-0 SepoliaETH

Functions in contract executed :

Deployed Contracts 1

✓ TRUSTSCORE AT 0xE9D...ADC9

Balance: 0 ETH

setInitialScore 10

SETLOANMANAGERADDRESS

_loanManagerAddress: 0x1b46C1954361A8cB698711

Calldata Parameters transact

UPDATESCORE

user: 0xe9D99e9E73ca9fD72fF52f090Da8

score: 20

Calldata Parameters transact

DEPLOY & RUN TRANSACTIONS

SETLOANMANAGERADDRESS

_loanManagerAddress: 0x1b46C1954361A8cB698711

Calldata Parameters transact

UPDATESCORE

user: 0xe9D99e9E73ca9fD72fF52f090Da8

score: 20

Calldata Parameters transact

admin

GETSCORE

user: 0xe9D99e9E73ca9fD72fF52f090Da8

Calldata Parameters call

0: uint256: 20

Conclusion:

In this experiment, we successfully connected **Remix IDE** to the **Sepolia Testnet** using **MetaMask** as the wallet provider. After configuring the correct network and linking the wallet through the Injected Provider, Remix was able to interact directly with the blockchain.

Smart contracts were then compiled, deployed, and executed on the Sepolia network. The test ETH was used to pay gas fees, and all transactions were recorded on the blockchain, demonstrating a real-time decentralized execution environment.

This experiment provided practical exposure to blockchain development workflows, including wallet integration, testnet usage, contract deployment, and transaction validation, helping us understand how decentralized applications are developed and tested before mainnet deployment.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 08

Aim: To implement the Blockchain platform Ganache and deploy a simple Solidity smart contract using Metamask and Remix IDE.

Roll No.	33
Name	Rujuta Vinit Medhi
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5:Interpret different Crypto assets and Crypto currencies

Experiment No. 8

Aim: To implement the Blockchain platform Ganache and deploy a simple Solidity smart contract using Metamask and Remix IDE.

Theory:

Ganache is a personal Ethereum blockchain used for testing and development. It allows developers to deploy contracts, develop decentralized applications (DApps), and run tests in a safe and deterministic environment.

Ganache provides:

- Instant mining of blocks
- Pre-funded test accounts
- Transaction history view
- Graphical interface to monitor blockchain activities

It is part of the Truffle Suite and is commonly used with Remix and Metamask for smart contract deployment.

Connection between Ganache, MetaMask, and Remix IDE

These three tools work together to create a complete blockchain workflow:

1. Ganache → MetaMask

- Ganache provides **local blockchain + accounts**
- MetaMask connects using:
 - RPC URL (e.g., 127.0.0.1:7545)
- Ganache accounts are imported into MetaMask using private keys
- MetaMask can now use Ganache accounts

2. MetaMask → Remix IDE

- In Remix, select:
👉 **Injected Provider - MetaMask**
- Remix connects to MetaMask wallet
- Remix uses MetaMask account to perform transactions

3. Remix IDE → Ganache

- When you deploy a contract:

- Remix → sends request to MetaMask
- MetaMask → signs transaction
- Ganache → executes transaction

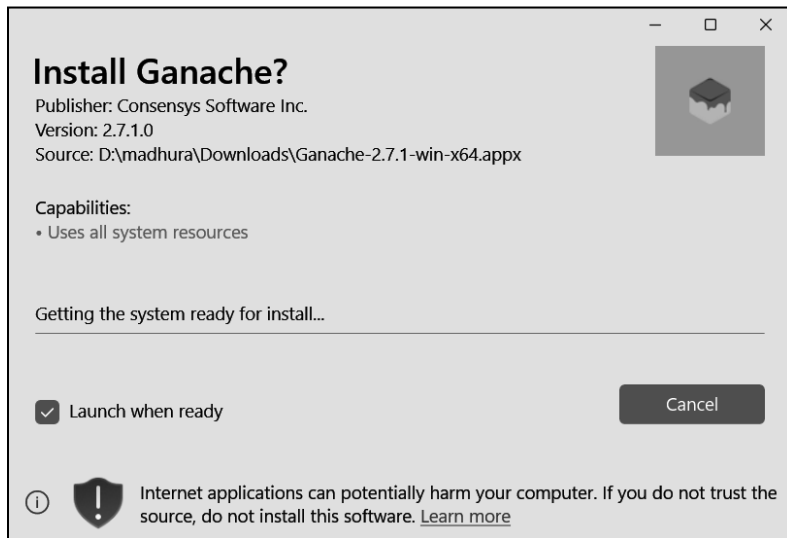
The contract gets deployed on the local blockchain.

Advantages of this setup

- No real money required
- Fast execution
- Easy debugging
- Safe testing environment

Implementation:

Installing Ganache in system-



After installing Ganache, clicking on quickstart we get 10 accounts with 100ETH each-

Ganache

ACCOUNTS

BLOCKS

TRANSACTIONS

CONTRACTS

EVENTS

LOGS

SEARCH FOR BLOCK NUMBERS OR TX HASHES

CURRENT BLOCK
0

GAS PRICE
20000000000

GAS LIMIT
6721975

HARDFORK
MERGE

NETWORK ID
5777

RPC SERVER
HTTP://127.0.0.1:7545

MINING STATUS
AUTOMINING

WORKSPACE
QUICKSTART

SAVE

SWITCH

MNEMONIC

call cluster rescue pulse picture wage pill exit exercise increase eyebrow clog

HD PATH

m44'60'0'0account_index

ADDRESS	BALANCE	TX COUNT	INDEX	
0x5b179E6F7Bc02d32846C97cc1162A3Be2949d9B9	100.00 ETH	0	0	
ADDRESS	BALANCE	TX COUNT	INDEX	
0xc6b2C102a15645697A05a4DEb5Df0c962b6ae7Cb	100.00 ETH	0	1	
ADDRESS	BALANCE	TX COUNT	INDEX	
0xDAdAd9c585AcB448e3525e2793cd92211f9eE9E	100.00 ETH	0	2	
ADDRESS	BALANCE	TX COUNT	INDEX	
0xEE040B87fdc42A5Af45997db300fE4f68EbEA2fA	100.00 ETH	0	3	
ADDRESS	BALANCE	TX COUNT	INDEX	
0x53CbcE93C9103e9E29A418F3C0ee711634b53C34	100.00 ETH	0	4	
ADDRESS	BALANCE	TX COUNT	INDEX	
0x47E76daC2CdCf4981c78F9759dECdeb0B199b8DA	100.00 ETH	0	5	
ADDRESS	BALANCE	TX COUNT	INDEX	
0xF5Dd69936b84e2BE9CCE3847B97fd5aAB340D3F8	100.00 ETH	0	6	

In Matamask Adding Ganache network-

MetaMask

< Add network ×

Network name

Ganache

According to our records, the network name may not correctly match this chain ID.
Suggested name: Localhost 8545

Default RPC URL

127.0.0.1:7545 ▾

Chain ID

1337

Currency symbol

ETH

Block explorer URL

Add a URL ▾

Save

Copying the private key of account 0 and importing that account in metamask-

MetaMask

< Add wallet ×

Private key

Imported private keys are backed up to your account and sync automatically when you sign in with the same Google or Apple login.
[Learn more about how imported keys work](#)

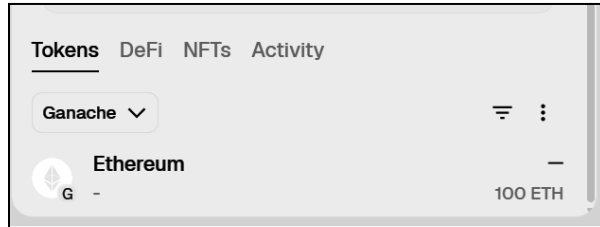
Select type Private key ▾

Enter your private key string here:

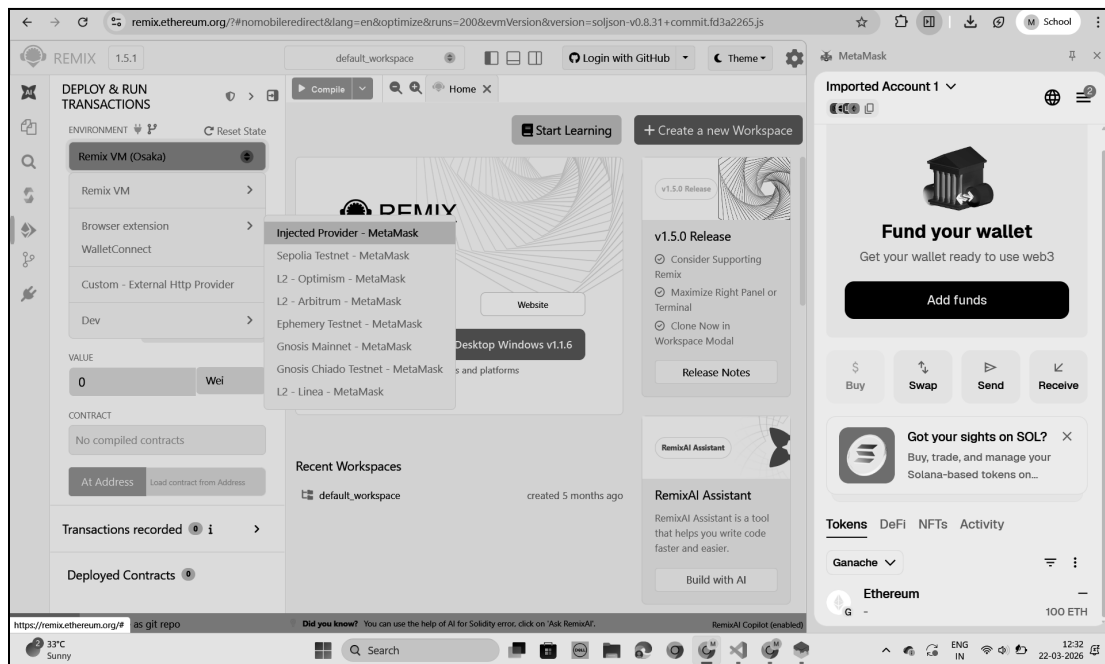
..... 🔒

Cancel Import

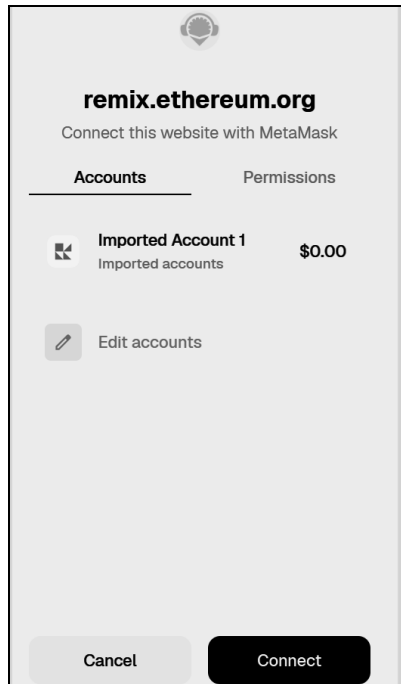
Account imported successfully balance is 100 ETH-



Selecting injected provider - metamask environment in remix ide-



Connection with remix ide-



Deploying contracts-

Contract 1- TrustScore.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.17;
```

```
contract TrustScore {
    // Store trust scores for each user
    mapping(address => uint256) public trustScores;
    address public admin;
    address public loanManagerAddress;
    uint256 public initialScore = 50; // Initial score for
    new users
```

```
    constructor() {
        admin = msg.sender;
    }
```

```
    // Set LoanManager address (admin only)
    function setLoanManagerAddress(address
    _loanManagerAddress) public {
        require(msg.sender == admin, "Only admin");
        loanManagerAddress = _loanManagerAddress;
    }
```

```
    // Admin or LoanManager can update scores
```

```
        function updateScore(address user, uint256 score)
        public {
            require(msg.sender == admin || msg.sender ==
```

```
            loanManagerAddress, "Only admin or
            LoanManager");
            require(score <= 100, "Score must be 0-100");
            trustScores[user] = score;
        }
```

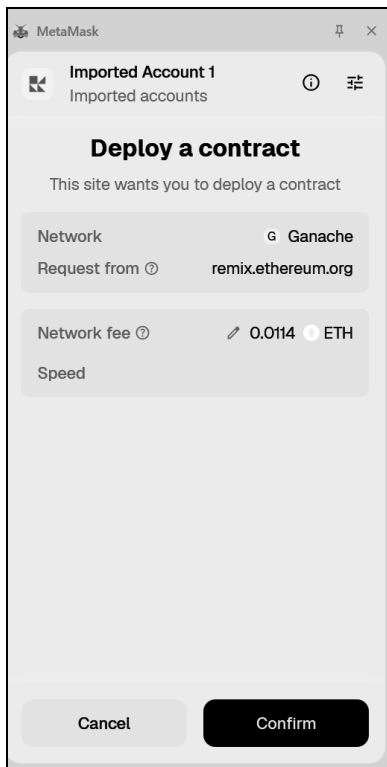
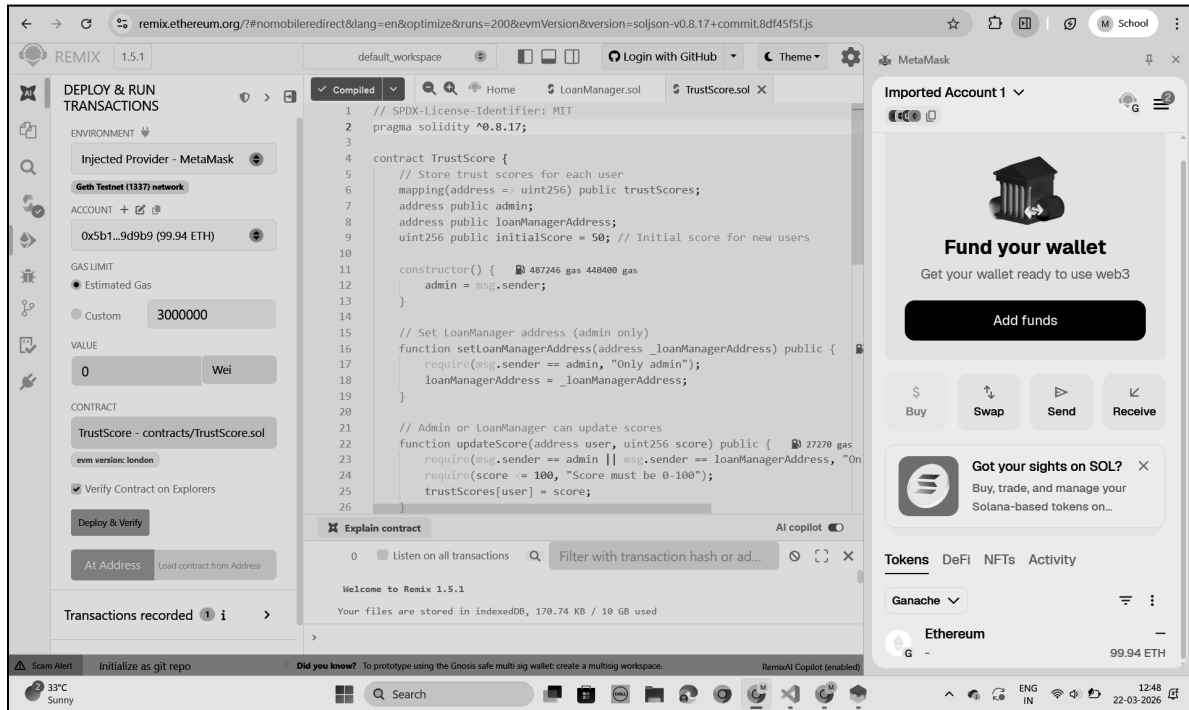
```
    // Set initial score for new users (admin only)
    function setInitialScore(uint256 _initialScore)
    public {
        require(msg.sender == admin, "Only admin");
        require(_initialScore <= 100, "Score must be
        0-100");
        initialScore = _initialScore;
    }
```

```
    // Anyone can check a score - returns initialScore if
    not set
    function getScore(address user) public view
    returns (uint256) {
        uint256 score = trustScores[user];
```

// If score is 0, return initial score (user hasn't been explicitly set)

return score > 0 ? score : initialScore;

}



Contract 2- LoanManager.sol

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.17;
import "./TrustScore.sol";

interface ITrustScore {
    function getScore(address user) external view
    returns (uint256);
    function updateScore(address user, uint256 score)
    external;
}

contract LoanManager {

    TrustScore public trustScoreContract;
    address public admin;
    uint256 public minTrustScore;

    // Trust score rewards/penalties
    uint256 public repaymentBonus = 5;    // +5
    points for successful repayment
    uint256 public defaultPenalty = 10;    // -10 points
    for defaulted loan

    // Safety pool address for collecting fees
    address public safetyPoolAddress;

    constructor(address _trustScoreAddress) {
        trustScoreContract =
        TrustScore(_trustScoreAddress);
        admin = msg.sender;
        minTrustScore = 40; // Default minimum trust
        score
    }

    // Set safety pool address (admin only)
    function setSafetyPoolAddress(address
    _safetyPoolAddress) public {
        require(msg.sender == admin, "Only admin");
        safetyPoolAddress = _safetyPoolAddress;
    }

    struct Loan {
        address borrower;
        address lender;
        uint256 amount;
        uint256 interestRate; // Interest rate in basis
        points (e.g., 500 = 5%)
        uint256 safetyFee;    // Safety fee in basis points

```

```

        uint256 duration;
        uint256 createdAt;
        bool funded;
        bool repaid;
        bool withdrawn;
        bool defaulted;
    }

    Loan[] public loans;

    event LoanRequested(uint256 loanId, address
    borrower, uint256 amount, uint256 interestRate);
    event LoanFunded(uint256 loanId, address lender);
    event LoanWithdrawn(uint256 loanId);
    event LoanRepaid(uint256 loanId, uint256
    totalRepaid, uint256 interest);
    event LoanDefaulted(uint256 loanId, address
    borrower);
    event TrustScoreIncreased(address borrower,
    uint256 amount);
    event TrustScoreDecreased(address borrower,
    uint256 amount);
    event MinTrustScoreUpdated(uint256 newScore);

    // Calculate interest rate based on trust score
    // Higher trust score = lower interest rate
    function getInterestRate(uint256 trustScore) public
    pure returns (uint256) {
        if (trustScore >= 90) return 300;    // 3% for
        excellent credit
        if (trustScore >= 75) return 500;    // 5% for
        good credit
        if (trustScore >= 60) return 800;    // 8% for fair
        credit
        if (trustScore >= 50) return 1200;    // 12% for
        poor credit
        return 1500;                        // 15% for very
        poor credit
    }

    // Get maximum borrow limit based on trust score
    (in wei)
    // Higher trust = larger loans allowed
    function getMaxBorrowLimit(uint256 trustScore)
    public pure returns (uint256) {
        if (trustScore >= 90) return 10 ether;    //
        Excellent: up to 10 ETH

```

```

        if (trustScore >= 75) return 5 ether;    // Good:
up to 5 ETH
        if (trustScore >= 60) return 2 ether;    // Fair: up
to 2 ETH
        if (trustScore >= 50) return 0.5 ether; // Poor:
up to 0.5 ETH
        return 0.1 ether;                      // Very poor: up
to 0.1 ETH
    }

```

```

// Get safety fee based on trust score (in basis
points)
// Lower trust = higher safety fee to protect lenders
function getSafetyFee(uint256 trustScore) public
pure returns (uint256) {
    if (trustScore >= 90) return 50;    // 0.5% for
excellent credit
    if (trustScore >= 75) return 100;   // 1% for
good credit
    if (trustScore >= 60) return 200;   // 2% for fair
credit
    if (trustScore >= 50) return 400;   // 4% for
poor credit
    return 600;                        // 6% for very poor
credit
}

```

```

// Get borrower's current limit info
function getBorrowerLimits(address borrower)
public view returns (
    uint256 trustScore,
    uint256 maxBorrowLimit,
    uint256 interestRate,
    uint256 safetyFee
) {
    trustScore =
trustScoreContract.getScore(borrower);
    maxBorrowLimit =
getMaxBorrowLimit(trustScore);
    interestRate = getInterestRate(trustScore);
    safetyFee = getSafetyFee(trustScore);
}

```

```

// Calculate total repayment amount (principal +
interest + safety fee)
function calculateRepayment(uint256 loanId)
public view returns (uint256) {
    Loan memory loan = loans[loanId];

```

```

    uint256 interest = (loan.amount *
loan.interestRate) / 10000;
    uint256 safetyFeeAmount = (loan.amount *
loan.safetyFee) / 10000;
    return loan.amount + interest +
safetyFeeAmount;
}

```

```

// Get breakdown of repayment components
function getRepaymentBreakdown(uint256 loanId)
public view returns (
    uint256 principal,
    uint256 interest,
    uint256 safetyFeeAmount,
    uint256 total
) {
    Loan memory loan = loans[loanId];
    principal = loan.amount;
    interest = (loan.amount * loan.interestRate) /
10000;
    safetyFeeAmount = (loan.amount *
loan.safetyFee) / 10000;
    total = principal + interest + safetyFeeAmount;
}

```

```

// Admin function to update minimum trust score
function setMinTrustScore(uint256 _minScore)
public {
    require(msg.sender == admin, "Only admin can
set minimum trust score");
    require(_minScore <= 100, "Score must be
0-100");
    minTrustScore = _minScore;
    emit MinTrustScoreUpdated(_minScore);
}

```

```

// Set trust score reward for repayment
function setRepaymentBonus(uint256 _bonus)
public {
    require(msg.sender == admin, "Only admin");
    repaymentBonus = _bonus;
}

```

```

// Set trust score penalty for default
function setDefaultPenalty(uint256 _penalty)
public {
    require(msg.sender == admin, "Only admin");
    defaultPenalty = _penalty;
}

```



```

    // Check if a loan is overdue
    function isLoanOverdue(uint256 loanId) public
view returns (bool) {
    Loan memory loan = loans[loanId];
    if (!loan.withdrawn || loan.repaid ||
loan.defaulted) {
        return false;
    }
    return block.timestamp > loan.createdAt +
loan.duration;
}

    // Mark loan as defaulted (called by admin or
lender)
    function markLoanDefaulted(uint256 loanId)
public {
        Loan storage loan = loans[loanId];
        require(!loan.defaulted && !loan.repaid, "Loan
already finalized");
        require(isLoanOverdue(loanId), "Loan is not
overdue");
        require(msg.sender == loan.lender || msg.sender
== admin, "Only lender or admin can mark default");

        loan.defaulted = true;

        // Decrease borrower's trust score
        uint256 currentScore =
trustScoreContract.getScore(loan.borrower);
        uint256 newScore = currentScore >
defaultPenalty ? currentScore - defaultPenalty : 0;
        trustScoreContract.updateScore(loan.borrower,
newScore);

        emit LoanDefaulted(loanId, loan.borrower);
        emit TrustScoreDecreased(loan.borrower,
defaultPenalty);
    }

    // -----
    // BORROWER CREATES LOAN REQUEST
    // -----
    function createLoan(uint256 amount, uint256
duration) public {

        uint256 score =
trustScoreContract.getScore(msg.sender);

```

```

        require(score >= minTrustScore, "Trust score
too low");

        // Enforce maximum borrow limit based on trust
score
        uint256 maxLimit =
getMaxBorrowLimit(score);
        require(amount <= maxLimit, "Amount exceeds
your borrow limit");

        uint256 interestRate = getInterestRate(score);
        uint256 safetyFee = getSafetyFee(score);

        loans.push(
            Loan({
                borrower: msg.sender,
                lender: address(0),
                amount: amount,
                interestRate: interestRate,
                safetyFee: safetyFee,
                duration: duration,
                createdAt: block.timestamp,
                funded: false,
                repaid: false,
                withdrawn: false,
                defaulted: false
            })
        );

        emit LoanRequested(loans.length - 1,
msg.sender, amount, interestRate);
    }

    // -----
    // LENDER FUNDS LOAN
    // -----
    function fundLoan(uint256 loanId) public payable
{

        Loan storage loan = loans[loanId];

        require(!loan.funded, "Already funded");
        require(msg.value == loan.amount, "Incorrect
ETH amount");

        loan.lender = msg.sender;
        loan.funded = true;

        emit LoanFunded(loanId, msg.sender);

```

```

    }

    // -----
    // BORROWER WITHDRAWS FUNDS
    // -----
    function withdrawLoan(uint256 loanId) public {

        Loan storage loan = loans[loanId];

        require(msg.sender == loan.borrower, "Not
borrower");
        require(loan.funded, "Loan not funded");
        require(!loan.withdrawn, "Already withdrawn");

        loan.withdrawn = true;

        payable(msg.sender).transfer(loan.amount);

        emit LoanWithdrawn(loanId);
    }

    // -----
    // BORROWER REPAYS LOAN
    // -----
    function repayLoan(uint256 loanId) public payable
    {

        Loan storage loan = loans[loanId];

        require(msg.sender == loan.borrower, "Not
borrower");
        require(loan.withdrawn, "Loan not withdrawn");
        require(!loan.repaid, "Already repaid");
        require(!loan.defaulted, "Loan is defaulted");
        require(loan.lender != address(0), "Invalid
lender address");

        // Get repayment breakdown
        (uint256 principal, uint256 interest, uint256
safetyFeeAmount, uint256 totalRepayment) =
getRepaymentBreakdown(loanId);
        require(msg.value >= totalRepayment,
"Insufficient repayment amount");

        // Transfer principal + interest to lender

```

```

        uint256 lenderAmount = principal + interest;
        payable(loan.lender).transfer(lenderAmount);

        // Transfer safety fee to safety pool (if set)
        if (safetyPoolAddress != address(0) &&
safetyFeeAmount > 0) {

            payable(safetyPoolAddress).transfer(safetyFeeAmou
nt);
        }

        loan.repaid = true;

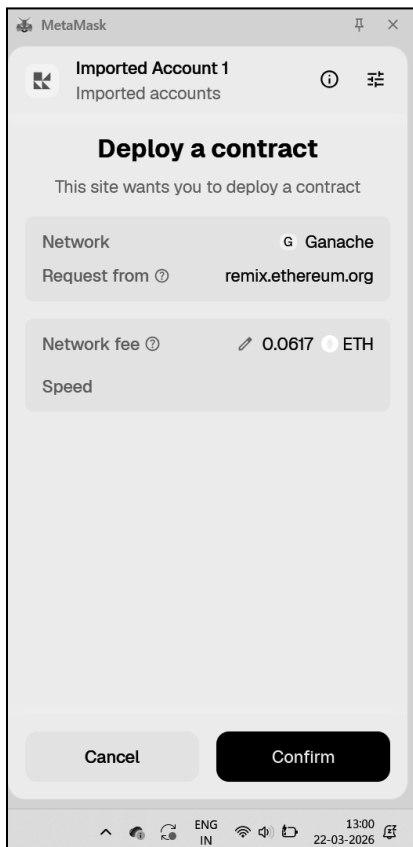
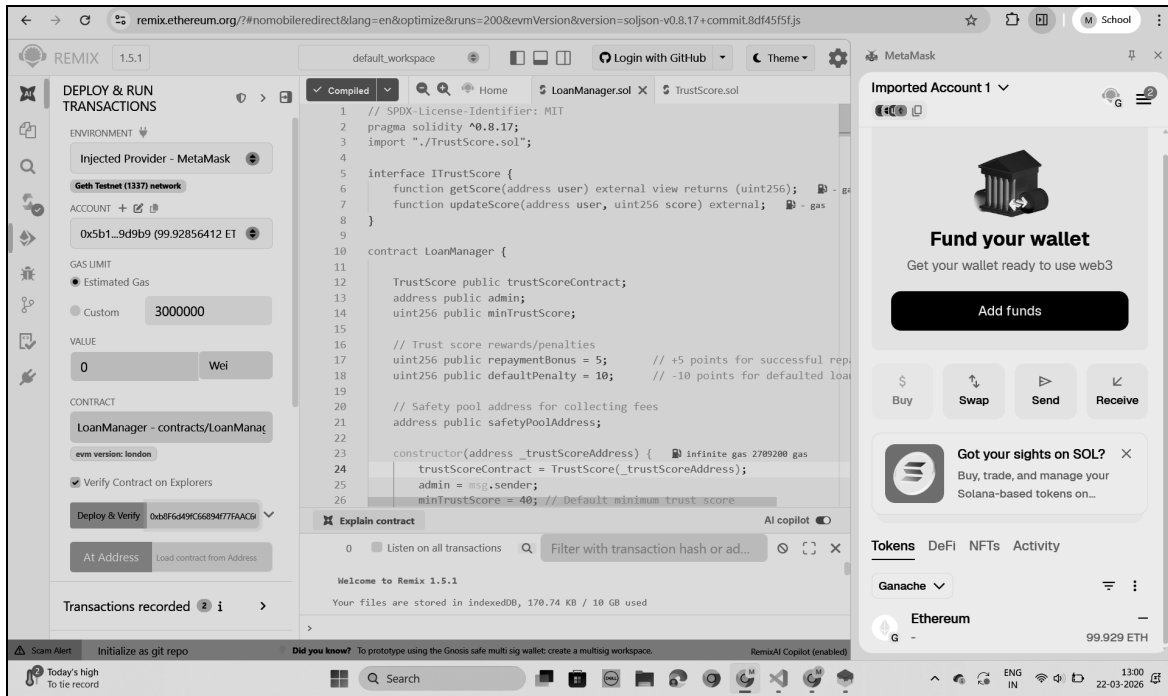
        // Increase borrower's trust score for successful
repayment
        uint256 currentScore =
trustScoreContract.getScore(loan.borrower);
        uint256 newScore = currentScore +
repaymentBonus;
        // Cap at 100
        if (newScore > 100) newScore = 100;
        trustScoreContract.updateScore(loan.borrower,
newScore);

        emit LoanRepaid(loanId, totalRepayment,
interest);
        emit TrustScoreIncreased(loan.borrower,
repaymentBonus);

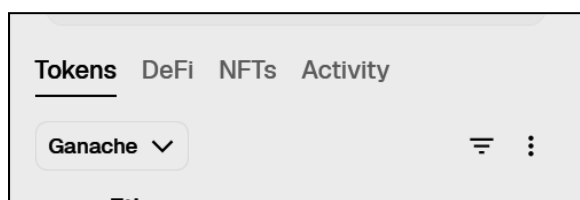
        // Return excess payment if any
        if (msg.value > totalRepayment) {
            payable(msg.sender).transfer(msg.value -
totalRepayment);
        }
    }

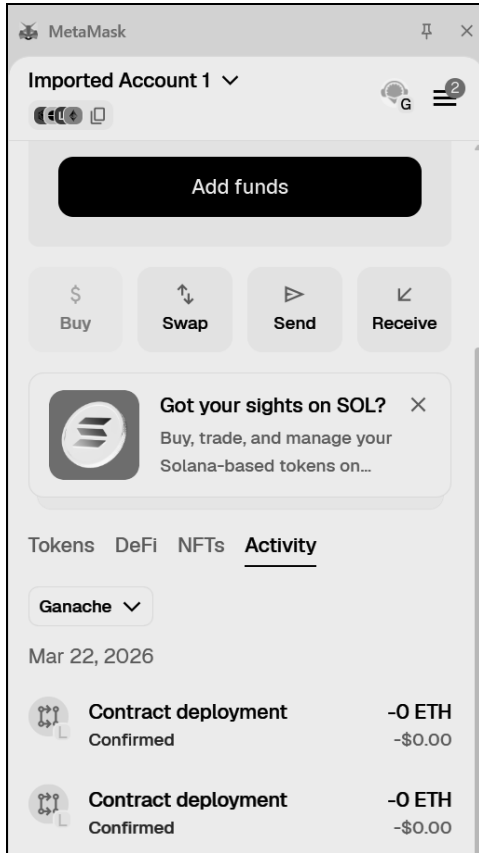
    // -----
    // GET ALL LOANS
    // -----
    function getLoans() public view returns (Loan[]
memory) {
        return loans;
    }
}

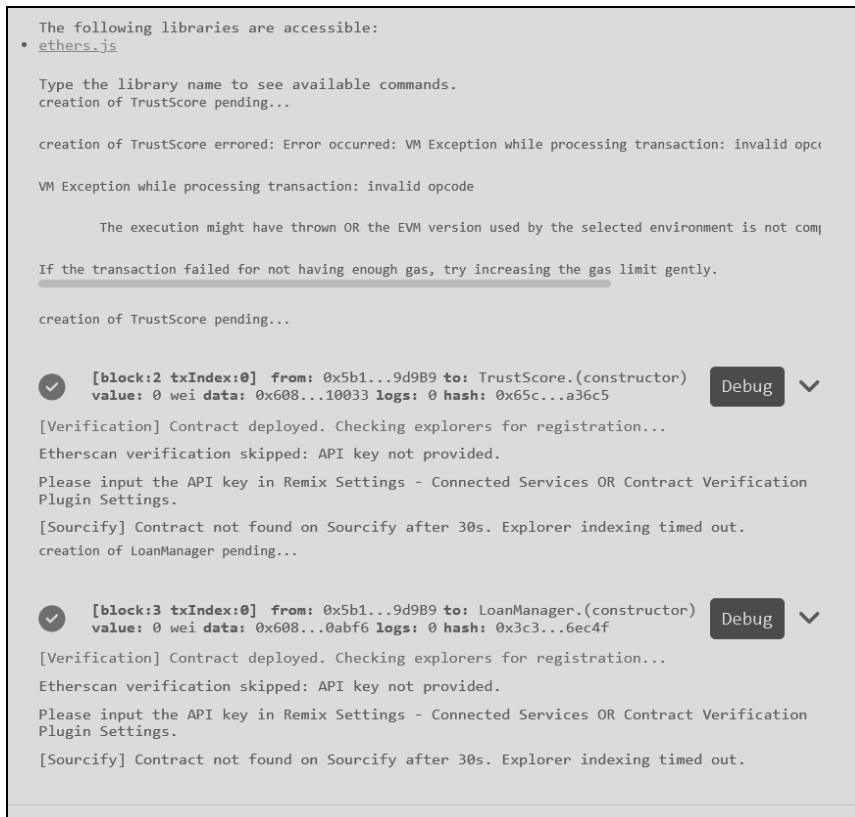
```



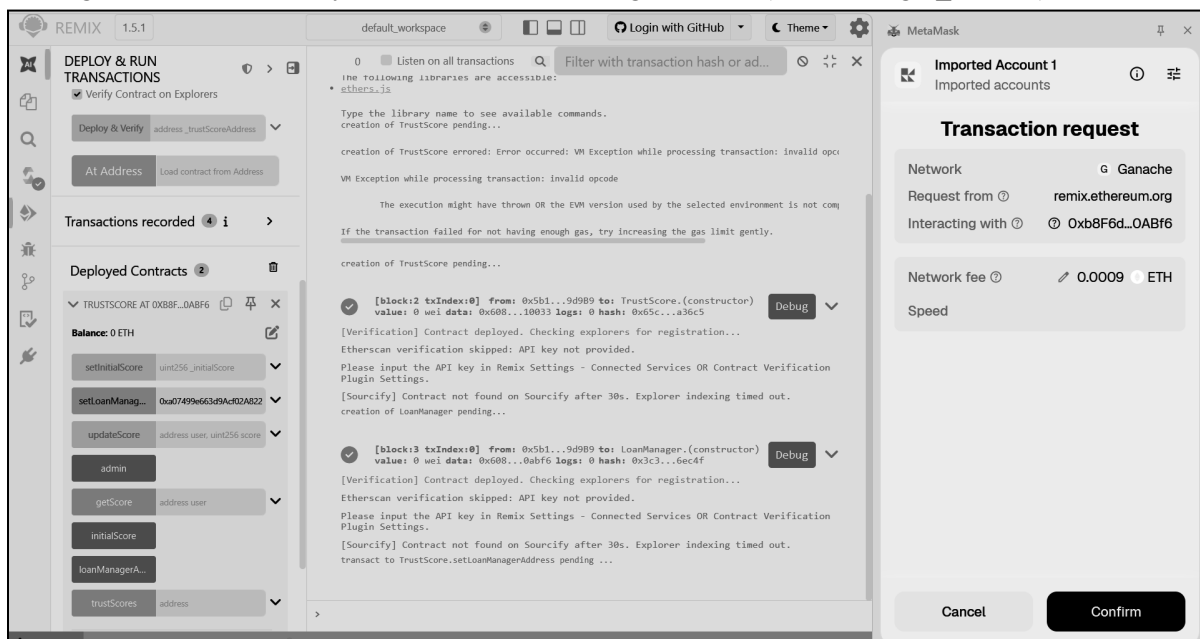
Contracts Deployed successfully-



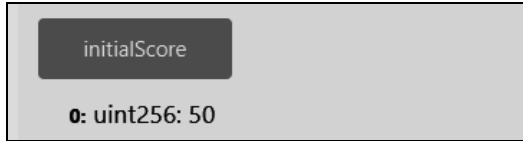




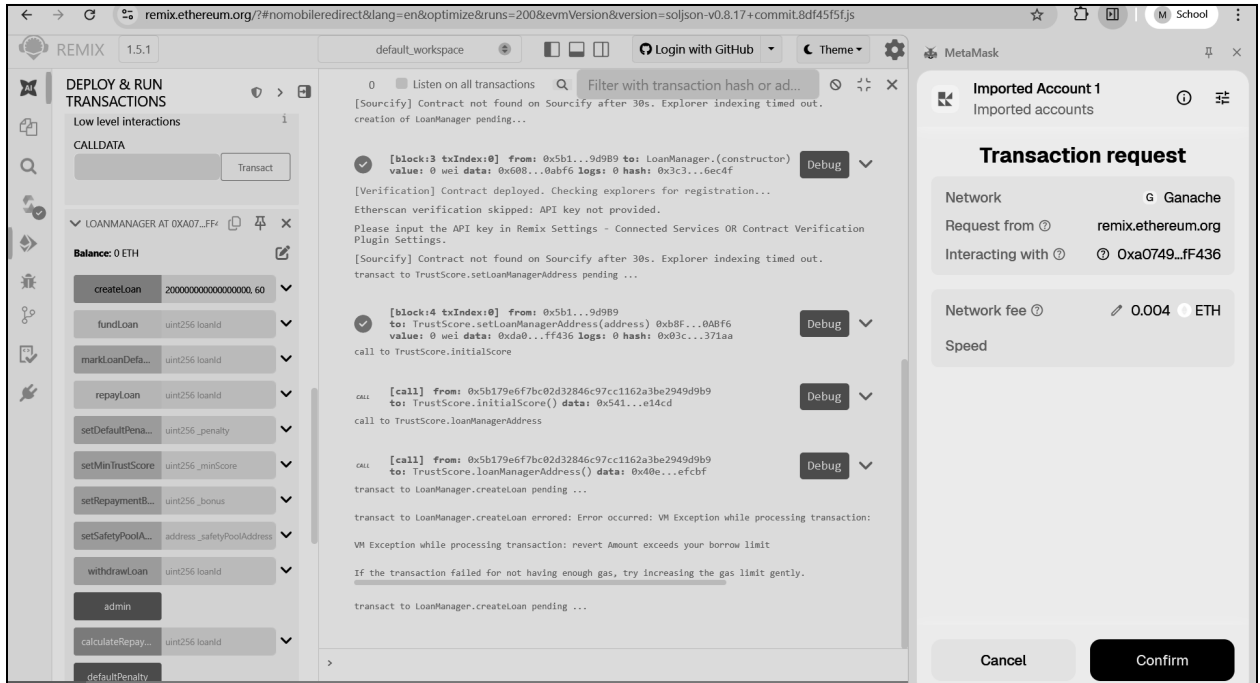
Linking both the contracts by function- setLoanManagerAddress(LoanManager_address)



getScore(your_address) - is 50 which is default



Function createLoan - Loan of 2 Eth created



Balance became 99.86 ETH after all the transactions-

Ganache									
ACCOUNTS		BLOCKS	TRANSACTIONS	CONTRACTS	EVENTS	LOGS	SEARCH FOR BLOCK NUMBERS OR TX HASHES		
CURRENT BLOCK 6	GAS PRICE 20000000000	GAS LIMIT 6721975	HARDFORK MERGE	NETWORK ID 5777	RPC SERVER HTTP://127.0.0.1:7545	MINING STATUS AUTOMINING	WORKSPACE QUICKSTART	SAVE	SWITCH
MNEMONIC call cluster rescue pulse picture wage pill exit exercise increase eyebrow clog							HD PATH m44'60'0'0account_index		
ADDRESS	BALANCE				TX COUNT	INDEX			
0x5b179E6F7Bc02d32846C97cc1162A3Be2949d9B9	99.86 ETH				6	0			
ADDRESS	BALANCE				TX COUNT	INDEX			
0xc6b2C102a15645697A05a4DEb5Df0c962b6ae7Cb	100.00 ETH				0	1			
ADDRESS	BALANCE				TX COUNT	INDEX			
0xDAAdD9c585AcB448e3525e2793cd92211f9eE9E	100.00 ETH				0	2			
ADDRESS	BALANCE				TX COUNT	INDEX			
0xEE040B87fdc42A5Af45997db300fE4f68EbEA2fA	100.00 ETH				0	3			
ADDRESS	BALANCE				TX COUNT	INDEX			
0x53CbcE93C9103e9E29A418F3C0ee711634b53C34	100.00 ETH				0	4			
ADDRESS	BALANCE				TX COUNT	INDEX			
0x47E76daC2CdCf4981c78F9759dECdeb0B199b8DA	100.00 ETH				0	5			
ADDRESS	BALANCE				TX COUNT	INDEX			
0xF5Dd69936b84e2BE9CCE3847B97fd5aAB340D3F8	100.00 ETH				0	6			
ADDRESS	BALANCE				TX COUNT	INDEX			
0x42CBc877791d28138204C48494b9CDc833f1DB56	100.00 ETH				0	7			

Transactions in Ganache-

Ganache

ACCOUNTS

BLOCKS

TRANSACTIONS

CONTRACTS

EVENTS

LOGS

SEARCH FOR BLOCK NUMBERS OR TX HASHES

CURRENT BLOCK
6

GAS PRICE
20000000000

GAS LIMIT
6721975

HARDFORK
MERGE

NETWORK ID
5777

RPC SERVER
HTTP://127.0.0.1:7545

MINING STATUS
AUTOMINING

WORKSPACE
QUICKSTART

SAVE

SWITCH

TX HASH

0xc590512e98401b6ed38a7e789653476039a3505e91ede413d79532091c2bf03

CONTRACT CALL

FROM ADDRESS

TO CONTRACT ADDRESS

GAS USED

VALUE

0x5b179E6F7Bc02d32846C97cc1162A3Be2949d9B9

0xa07499e663d9Acf02A8220a26C5F55eE11FFf436

198478

0

TX HASH

0xaa9f9182d9ffeee1639d11b7e6b2d7fd541ef40bd7ba0f891609daf79d11a0c0

CONTRACT CALL

FROM ADDRESS

TO CONTRACT ADDRESS

GAS USED

VALUE

0x5b179E6F7Bc02d32846C97cc1162A3Be2949d9B9

0xa07499e663d9Acf02A8220a26C5F55eE11FFf436

34995

0

TX HASH

0x484fe9c01e12f6a7780571ca44915add62f5930676d3283d6cc2ef630e20f529

CONTRACT CALL

FROM ADDRESS

TO CONTRACT ADDRESS

GAS USED

VALUE

0x5b179E6F7Bc02d32846C97cc1162A3Be2949d9B9

0xb8F6d49fC66894f77FAAC60b4A8Ba8680770ABf6

46268

0

TX HASH

0x4c1cf3ecbccf018807619e2647fac2116cf5d39bfe8532d5ab9b993faede0692

CONTRACT CREATION

FROM ADDRESS

CREATED CONTRACT ADDRESS

GAS USED

VALUE

0x5b179E6F7Bc02d32846C97cc1162A3Be2949d9B9

0xa07499e663d9Acf02A8220a26C5F55eE11FFf436

3082525

0

TX HASH

0x67e386b411dd100a3299018e28593892fb51be5a1ecca8c5beea46680e7c71b3

CONTRACT CREATION

FROM ADDRESS

CREATED CONTRACT ADDRESS

GAS USED

VALUE

0x5b179E6F7Bc02d32846C97cc1162A3Be2949d9B9

0xb8F6d49fC66894f77FAAC60b4A8Ba8680770ABf6

571794

0

No. of Blocks Mined-

ACCOUNTS

BLOCKS

TRANSACTIONS

CONTRACTS

EVENTS

LOGS

SEARCH FOR BLOCK NUMBERS OR TX HASHES

CURRENT BLOCK
6

GAS PRICE
20000000000

GAS LIMIT
6721975

HARDFORK
MERGE

NETWORK ID
5777

RPC SERVER
HTTP://127.0.0.1:7545

MINING STATUS
AUTOMINING

WORKSPACE
QUICKSTART

SAVE

SWITCH

BLOCK
6

MINED ON
2026-03-22 13:08:06

GAS USED
198478

1 TRANSACTION

BLOCK
5

MINED ON
2026-03-22 13:06:15

GAS USED
34995

1 TRANSACTION

BLOCK
4

MINED ON
2026-03-22 13:04:00

GAS USED
46268

1 TRANSACTION

BLOCK
3

MINED ON
2026-03-22 13:00:50

GAS USED
3082525

1 TRANSACTION

BLOCK
2

MINED ON
2026-03-22 12:49:22

GAS USED
571794

1 TRANSACTION

BLOCK
1

MINED ON
2026-03-22 12:44:36

GAS USED
3000000

1 TRANSACTION

BLOCK
0

MINED ON
2026-03-22 11:52:02

GAS USED
0

NO TRANSACTIONS

Conclusion:

Ganache provides a simple and powerful environment to simulate a local Ethereum blockchain. By integrating it with **Metamask** and **Remix**, developers can deploy, test, and debug smart contracts efficiently before moving to a public network.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 09

Aim: To implement a Private Ethereum Blockchain using Geth.

Roll No.	33
Name	Rujuta Vinit Medhi
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO3:Implement smart contracts in Ethereum using different development frameworks.

Experiment: 09

Aim: To implement a Private Ethereum Blockchain using Geth.

Theory:

Ethereum is an open-source, decentralized blockchain platform that enables developers to build and deploy smart contracts and decentralized applications (DApps). Unlike public blockchains such as the Ethereum mainnet, a Private Ethereum Blockchain is restricted to specific participants, making it ideal for development, testing, and enterprise applications where confidentiality, control, and performance are essential.

Geth (Go-Ethereum) is one of the most widely used Ethereum clients, written in the Go programming language. It allows users to run an Ethereum node, mine Ether, and interact with the blockchain through JSON-RPC, command-line, or JavaScript consoles.

Setting up a **Private Ethereum Network** involves creating a customized blockchain environment isolated from the public Ethereum network. It enables developers to test smart contracts, consensus mechanisms, and network behavior without incurring gas costs or depending on public validators.

Steps Involved:

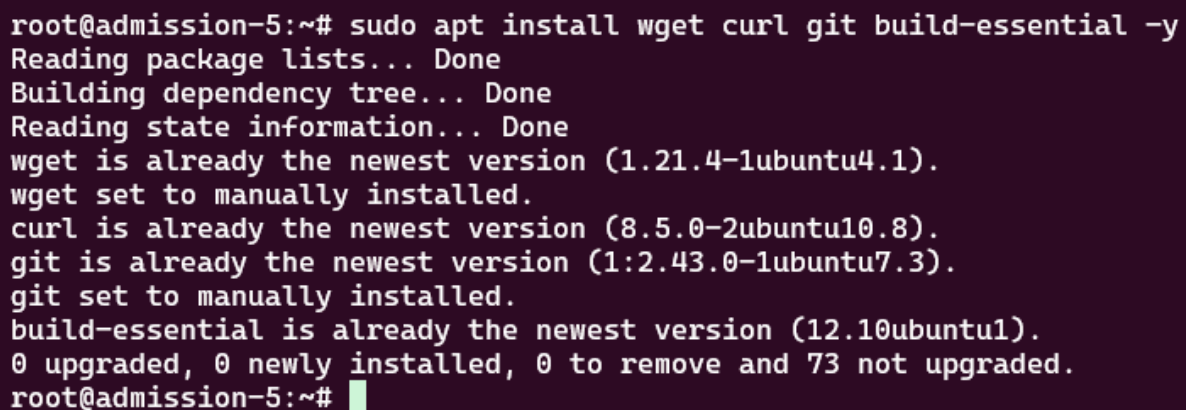
1. **Choosing a Network ID:**
2. Each private blockchain must have a unique network ID to differentiate it from public or other private networks.
3. **Choosing a Consensus Algorithm:**
4. Ethereum supports different consensus mechanisms such as **Proof of Work (PoW)** and **Proof of Authority (PoA)**. In private networks, **Clique (PoA)** is often used for faster block confirmation.
5. **Creating a Genesis Block:**
6. The genesis block is the first block of the blockchain, defining network parameters such as difficulty, gas limit, initial account balances, and consensus settings.
7. **Initializing the Geth Database:**
8. The `geth init` command initializes the blockchain using the `genesis.json` file, preparing the data directory for node operation.
9. **Setting up Networking:**
10. Nodes communicate using **enodes** (Ethereum Node URLs) that contain IP addresses and public keys for peer discovery and connection.
11. **Running the Member Nodes:**

12. Each participant runs a Geth node that connects to the private network and participates in block validation and transaction processing.
13. **Running a Signer (in Clique):**
14. In the **Clique consensus**, designated **signers** validate and sign blocks. These accounts are pre-configured in the genesis file to ensure trusted block production.

Implementation:

1. First install all the necessary dependencies like curl, git, build-essential.

`sudo apt install wget curl git build-essential -y`

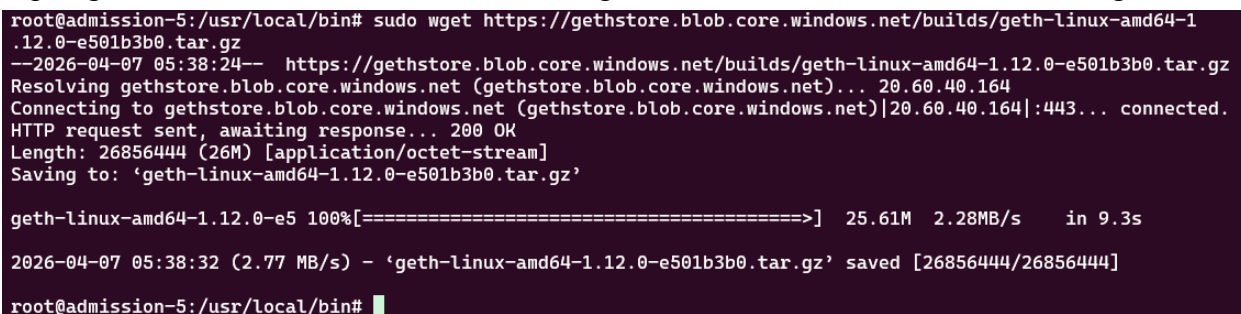


```
root@admission-5:~# sudo apt install wget curl git build-essential -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
wget is already the newest version (1.21.4-1ubuntu4.1).
wget set to manually installed.
curl is already the newest version (8.5.0-2ubuntu10.8).
git is already the newest version (1:2.43.0-1ubuntu7.3).
git set to manually installed.
build-essential is already the newest version (12.10ubuntu1).
0 upgraded, 0 newly installed, 0 to remove and 73 not upgraded.
root@admission-5:~#
```

2. Install geth using wget

`cd /usr/local/bin && sudo wget`

`https://gethstore.blob.core.windows.net/builds/geth-linux-amd64-1.12.0-e501b3b0.tar.gz`



```
root@admission-5:/usr/local/bin# sudo wget https://gethstore.blob.core.windows.net/builds/geth-linux-amd64-1.12.0-e501b3b0.tar.gz
--2026-04-07 05:38:24-- https://gethstore.blob.core.windows.net/builds/geth-linux-amd64-1.12.0-e501b3b0.tar.gz
Resolving gethstore.blob.core.windows.net (gethstore.blob.core.windows.net)... 20.60.40.164
Connecting to gethstore.blob.core.windows.net (gethstore.blob.core.windows.net)|20.60.40.164|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 26856444 (26M) [application/octet-stream]
Saving to: 'geth-linux-amd64-1.12.0-e501b3b0.tar.gz'

geth-linux-amd64-1.12.0-e5 100%[=====] 25.61M 2.28MB/s in 9.3s

2026-04-07 05:38:32 (2.77 MB/s) - 'geth-linux-amd64-1.12.0-e501b3b0.tar.gz' saved [26856444/26856444]

root@admission-5:/usr/local/bin#
```

3. Extract the tar file and move the geth into /usr/local/bin/geth path & check the geth version

`sudo tar -xvf geth-linux-amd64-1.12.0-e501b3b0.tar.gz sudo mv`

`geth-linux-amd64-1.12.0-e501b3b0/geth /usr/local/bin/geth geth version`

```

root@admission-5:/usr/local/bin# sudo tar -xvf geth-linux-amd64-1.12.0-e501b3b0.tar.gz
sudo mv geth-linux-amd64-1.12.0-e501b3b0/geth /usr/local/bin/geth
geth version
geth-linux-amd64-1.12.0-e501b3b0/
geth-linux-amd64-1.12.0-e501b3b0/COPYING
geth-linux-amd64-1.12.0-e501b3b0/geth
Geth
Version: 1.12.0-stable
Git Commit: e501b3b05db8e169f67dc78b7b59bc352b3c638d
Git Commit Date: 20230525
Architecture: amd64
Go Version: go1.20.3
Operating System: linux
GOPATH=
GOROOT=
root@admission-5:/usr/local/bin# █

```

4. Create a folder named private_ethereum_setup and create another 2 directories naming node1 and node2 individually inside private_ethereum_setup directory.

mkdir private_ethereum_setup cd private_ethereum_setup && mkdir node1 node2

```

root@admission-5:/usr/local/bin# mkdir private_ethereum_setup
root@admission-5:/usr/local/bin# cd private_ethereum_setup && mkdir node1 node2
root@admission-5:/usr/local/bin/private_ethereum_setup# ls
node1  node2
root@admission-5:/usr/local/bin/private_ethereum_setup# █

```

5. Create account for node1

geth --datadir "node1" account new

```

root@admission-5:/usr/local/bin/private_ethereum_setup# geth --datadir "node1" account new
INFO [04-07|05:42:29.002] Maximum peer count          ETH=50 LES=0 total=50
INFO [04-07|05:42:29.003] Smartcard socket not found, disabling  err="stat /run/pcscd/pcscd.comm: no such file or directory"
Your new account is locked with a password. Please give a password. Do not forget this password.
Password:
Repeat password:

Your new key was generated

Public address of the key:  0xB39388323dCD82eeb28F73B3448424c008FE7150
Path of the secret key file: node1/keystore/UTC--2026-04-07T05-42-42.824513266Z--b39388323dcd82eeb28f73b3448424c008fe7150

- You can share your public address with anyone. Others need it to interact with you.
- You must NEVER share the secret key with anyone! The key controls access to your funds!
- You must BACKUP your key file! Without the key, it's impossible to access account funds!
- You must REMEMBER your password! Without the password, it's impossible to decrypt the key!

root@admission-5:/usr/local/bin/private_ethereum_setup# █

```

6. Do the same for node2

geth --datadir "node2" account new


```

    },
    "0x126F82349979c6b3d2a52eE032B7ea11880D2dC9": {
      "balance": "1000000000000000000000000"
    }
  }
}

```

8. Cat genesis.json

```
root@admission-5: /usr/local/bin/private_ethereum_setup# cat genesis.json
{
  "config": {
    "chainId": 12345,
    "homesteadBlock": 0,
    "eip150Block": 0,
    "eip155Block": 0,
    "eip158Block": 0,
    "byzantiumBlock": 0,
    "constantinopleBlock": 0,
    "petersburgBlock": 0,
    "istanbulBlock": 0,
    "berlinBlock": 0,
    "londonBlock": 0,
    "clique": {
      "period": 5,
      "epoch": 30000
    }
  },
  "difficulty": "1",
  "gasLimit": "8000000",
  "extradata":
"0x0000000000000000000000000000000000000000000000000000000000000000
00000000c99cf64ef49425bf0e814c9d51364281a61747d2000000000000000
0000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000
000000",
  "alloc": {
    "0xB39388323dCD82eeb28F73B3448424c008FE7150": {
      "balance": "100000000000000000000"
    },
    "0x126F82349979c6b3d2a52eE032B7ea11880D2dC9": {
      "balance": "100000000000000000000"
    }
  }
}
```

9. Initializing geth for node1 using the genesis.json file

```
geth init --datadir node1 genesis.json
```

```

root@admission-5:/usr/local/bin/private_ethereum_setup# geth init --datadir node1 genesis.json
INFO [04-07|05:58:25.537] Maximum peer count          ETH=50 LES=0 total=50
INFO [04-07|05:58:25.538] Smartcard socket not found, disabling err="stat /run/pcscd/pcscd.comm: no such file or directory"
INFO [04-07|05:58:25.543] Set global gas cap          cap=50,000,000
INFO [04-07|05:58:25.544] Initializing the KZG library backend=gokzg
INFO [04-07|05:58:25.594] Defaulting to pebble as the backing database
INFO [04-07|05:58:25.595] Allocated cache and file handles database=/usr/local/bin/private_ethereum_setup/node1/geth/chaindata
a cache=16.00MiB handles=16
INFO [04-07|05:58:25.618] Opened ancient database      database=/usr/local/bin/private_ethereum_setup/node1/geth/chaindata
a/ancient/chain readonly=false
INFO [04-07|05:58:25.619] Writing custom genesis block
INFO [04-07|05:58:25.619] Persisted trie from memory database nodes=3 size=411.00B time="9.642µs" gcnodes=0 gcsize=0.00B gctime=
0s livenodes=0 livesize=0.00B
INFO [04-07|05:58:25.623] Successfully wrote genesis state database=chaindata hash=8c5dd7..24caed
INFO [04-07|05:58:25.623] Defaulting to pebble as the backing database
INFO [04-07|05:58:25.623] Allocated cache and file handles database=/usr/local/bin/private_ethereum_setup/node1/geth/lightcha
indata cache=16.00MiB handles=16
INFO [04-07|05:58:25.640] Opened ancient database      database=/usr/local/bin/private_ethereum_setup/node1/geth/lightcha
indata/ancient/chain readonly=false
INFO [04-07|05:58:25.641] Writing custom genesis block
INFO [04-07|05:58:25.641] Persisted trie from memory database nodes=3 size=411.00B time="10.121µs" gcnodes=0 gcsize=0.00B gctime
=0s livenodes=0 livesize=0.00B
INFO [04-07|05:58:25.644] Successfully wrote genesis state database=lightchaindata hash=8c5dd7..24caed
root@admission-5:/usr/local/bin/private_ethereum_setup#

```

10. Do the same for node2

geth init --datadir node2 genesis.json

```

root@admission-5:/usr/local/bin/private_ethereum_setup# geth init --datadir node2 genesis.json
INFO [04-07|05:59:40.725] Maximum peer count          ETH=50 LES=0 total=50
INFO [04-07|05:59:40.726] Smartcard socket not found, disabling err="stat /run/pcscd/pcscd.comm: no such file or directory"
INFO [04-07|05:59:40.729] Set global gas cap          cap=50,000,000
INFO [04-07|05:59:40.729] Initializing the KZG library backend=gokzg
INFO [04-07|05:59:40.760] Defaulting to pebble as the backing database
INFO [04-07|05:59:40.760] Allocated cache and file handles database=/usr/local/bin/private_ethereum_setup/node2/geth/chaindata
a cache=16.00MiB handles=16
INFO [04-07|05:59:40.780] Opened ancient database      database=/usr/local/bin/private_ethereum_setup/node2/geth/chaindata
a/ancient/chain readonly=false
INFO [04-07|05:59:40.780] Writing custom genesis block
INFO [04-07|05:59:40.780] Persisted trie from memory database nodes=3 size=411.00B time="10.186µs" gcnodes=0 gcsize=0.00B gctime
=0s livenodes=0 livesize=0.00B
INFO [04-07|05:59:40.784] Successfully wrote genesis state database=chaindata hash=8c5dd7..24caed
INFO [04-07|05:59:40.784] Defaulting to pebble as the backing database
INFO [04-07|05:59:40.784] Allocated cache and file handles database=/usr/local/bin/private_ethereum_setup/node2/geth/lightcha
indata cache=16.00MiB handles=16
INFO [04-07|05:59:40.805] Opened ancient database      database=/usr/local/bin/private_ethereum_setup/node2/geth/lightcha
indata/ancient/chain readonly=false
INFO [04-07|05:59:40.805] Writing custom genesis block
INFO [04-07|05:59:40.805] Persisted trie from memory database nodes=3 size=411.00B time="10.09µs" gcnodes=0 gcsize=0.00B gctime
=0s livenodes=0 livesize=0.00B
INFO [04-07|05:59:40.808] Successfully wrote genesis state database=lightchaindata hash=8c5dd7..24caed
root@admission-5:/usr/local/bin/private_ethereum_setup#

```

11. Now we will install bootnode for configuring both the nodes.

sudo add-apt-repository -y ppa:ethereum/ethereum

sudo apt-get update

sudo apt-get install bootnode

```

root@admission-5:/usr/local/bin/private_ethereum_setup# sudo add-apt-repository -y ppa:ethereum/ethereum
sudo apt-get update
sudo apt-get install bootnode
Repository: 'Types: deb
URIs: https://ppa.launchpadcontent.net/ethereum/ethereum/ubuntu/
Suites: noble
Components: main
'
More info: https://launchpad.net/~ethereum/+archive/ubuntu/ethereum
Adding repository.
Hit:1 https://deb.nodesource.com/node_18.x nodistro InRelease
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:3 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:4 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:5 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Get:6 https://ppa.launchpadcontent.net/ethereum/ethereum/ubuntu noble InRelease [18.1 kB]
Get:7 https://ppa.launchpadcontent.net/ethereum/ethereum/ubuntu noble/main amd64 Packages [2576 B]
Get:8 https://ppa.launchpadcontent.net/ethereum/ethereum/ubuntu noble/main Translation-en [772 B]
Fetched 21.4 kB in 1s (18.3 kB/s)
Reading package lists... Done
Hit:1 https://deb.nodesource.com/node_18.x nodistro InRelease
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:5 https://ppa.launchpadcontent.net/ethereum/ethereum/ubuntu noble InRelease
Hit:6 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Reading package lists... Done
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  bootnode
0 upgraded, 1 newly installed, 0 to remove and 73 not upgraded.

```

12. Create a key for the bootnode and save it to boot.key in the folder bootnode -genkey boot.key

```

root@admission-5:/usr/local/bin/private_ethereum_setup# bootnode -genkey boot.key
root@admission-5:/usr/local/bin/private_ethereum_setup# ls
boot.key genesis.json node1 node2
root@admission-5:/usr/local/bin/private_ethereum_setup# █

```

13. Copy this command and copy the enode://.... This will be required later
bootnode -nodekey boot.key -verbosity 9 -addr :30305

```

root@admission-5:/usr/local/bin/private_ethereum_setup# bootnode -nodekey boot.key -verbosity 9 -addr :30305
enode://7ff7a90d47bc73762802a4f924fdb403af4bffa08ded3c3b0b5c2efeb9f76553859f6edf9abdf20619c04c9408fc7ed606e097a26d4519d50f5afcc063e02120@127.0.0.1:30305
Note: you're using cmd/bootnode, a developer tool.
We recommend using a regular node as bootstrap node for production deployments.
INFO [04-07|06:03:30.800] New local node record      seq=1,775,541,810,798 id=f572303c2cd66d79 ip="invalid IP" udp=0 tcp=0

```

14. Now on another terminal we will start mining node1

```

geth --datadir node1 \
--port 30306 \
--bootnodes
"enode://7ff7a90d47bc73762802a4f924fdb403af4bffa08ded3c3b0b5c2efeb9f76553859f6edf9abdf20619c04c9408fc7ed606e097a26d4519d50f5afcc063e02120@127.0.0.1:30305" \
--networkid 123454321 \

```



```
root@admission-5: /usr/local/bin/private_ethereum_setup# rm -rf /usr/local/bin/private_ethereum_setup/node1/gets
root@admission-5: /usr/local/bin/private_ethereum_setup# geth --datadir node1 --port 30306 --bootnodes "enode://7fff7a90d47bc73762802a4
f924fcd3403af4bfff08ded3c3b0b5c2efeb9f76553859f6edf9abdf20619c04c9408fc7ed606e097a26d4519d50f5afcc063e02120127.0.0.1:30305" --network
id 123454321 --unlock 0xb39388323dCD82eeb28f73B3448424c008FE7150 --password node1/password.txt --authrpc.port 8551 --miner.etherbase
0xb39388323dCD82eeb28f73B3448424c008FE7150 --mine
INFO [04-07]06:38:47.164] Maximum peer count ETH=50 LES=0 total=50
INFO [04-07]06:38:47.166] Smartcard socket not found, disabling err="stat /run/pcscd/pcscd.comm: no such file or directory"
INFO [04-07]06:38:47.168] Set global gas cap cap=50,000,000
INFO [04-07]06:38:47.169] Initializing the KZG library backend=gokzg
INFO [04-07]06:38:47.199] Allocated trie memory caches clean=154.00MiB dirty=256.00MiB
INFO [04-07]06:38:47.199] Defaulting to pebble as the backing database
INFO [04-07]06:38:47.199] Allocated cache and file handles database=/usr/local/bin/private_ethereum_setup/node1/gets/chaindata
a cache=512.00MiB handles=524,288
INFO [04-07]06:38:47.225] Opened ancient database database=/usr/local/bin/private_ethereum_setup/node1/gets/chaindata
a/ancient/chain readonly=false
INFO [04-07]06:38:47.225] Initialising Ethereum protocol network=123,454,321 dbversion=<nil>
INFO [04-07]06:38:47.226] Writing default main-net genesis block
INFO [04-07]06:38:47.461] Persisted trie from memory database nodes=12356 size=1.79MiB time=48.92926ms gcnodes=0 gcsz=0.00B gc
time=0s livenodes=0 liveness=0.00B
INFO [04-07]06:38:47.476]
INFO [04-07]06:38:47.476] -----
INFO [04-07]06:38:47.476] Chain ID: 1 (mainnet)
INFO [04-07]06:38:47.476] Consensus: Beacon (proof-of-stake), merged from Ethash (proof-of-work)
INFO [04-07]06:38:47.476]
INFO [04-07]06:38:47.476] Pre-Merge hard forks (block based):
INFO [04-07]06:38:47.476] - Homestead: #1150000 (https://github.com/ethereum/execution-specs/blob/master/network-
upgrades/mainnet-upgrades/homestead.md)
INFO [04-07]06:38:47.476] - DAO Fork: #1920000 (https://github.com/ethereum/execution-specs/blob/master/network-
upgrades/mainnet-upgrades/dao-fork.md)
INFO [04-07]06:38:47.476] - Tangerine Whistle (EIP 150): #2463000 (https://github.com/ethereum/execution-specs/blob/master/network-
upgrades/mainnet-upgrades/tangerine-whistle.md)

root@admission-5: - X root@admission-5: /usr/local/ X root@admission-5: /usr/local, X + v - 0 X
WARN [04-07]06:53:41.791] Unclean shutdown detected booted=2026-04-07T06:38:47+0000 age=14m54s
WARN [04-07]06:53:41.791] Engine API enabled protocol=eth
INFO [04-07]06:53:41.791] Starting peer-to-peer node instance=Geth/v1.12.0-stable-e501b3b0/Linux-amd64/go1.20.3
INFO [04-07]06:53:41.799] Generated state snapshot accounts=8893 slots=0 storage=409.64KiB dangling=0 elapsed=9.332ms
INFO [04-07]06:53:41.806] New local node record seq=1,775,543,927,501 id=5fffab40659cdeaba ip=127.0.0.1 udp=30306 t
cp=30306
INFO [04-07]06:53:41.806] Started P2P networking self=enode://45d557b46f2495e3c7bdf4c01c5e3025b83bfbbfe5ef89badb57a
0a7d87a8d4c6ccab873cd8e40fb59e0d5b9ed3063d1a2e7b1c0042de2863090ad672c3b450127.0.0.1:30306
INFO [04-07]06:53:41.807] IPC endpoint opened url=/usr/local/bin/private_ethereum_setup/node1/gets_ipc
INFO [04-07]06:53:41.807] Loaded JWT secret file path=/usr/local/bin/private_ethereum_setup/node1/gets/jwtsecret cr
c32=0x21e9937f
INFO [04-07]06:53:41.808] WebSockets enabled url=ws://127.0.0.1:8551
INFO [04-07]06:53:41.808] HTTP server started endpoint=127.0.0.1:8551 auth=true prefix= cors=localhost vhosts=lo
calhost
INFO [04-07]06:53:42.426] Unlocked account address=0xb39388323dCD82eeb28f73B3448424c008FE7150
INFO [04-07]06:53:42.426] Transaction pool price threshold updated price=0
INFO [04-07]06:53:42.426] Transaction pool price threshold updated price=1,000,000,000
INFO [04-07]06:53:42.427] Commit new sealing work number=1 sealhash=0dc167.0b853c uncles=0 txs=0 gas=0 fees=0 elaps
ed="787.008µs"
panic: ethash (pow) sealing not supported any more

goroutine 46 [running]:
github.com/ethereum/go-ethereum/consensus/ethash.(*Ethash).Seal(0xc0002f6520?, {0x41aea6?, 0xc00044c2c0?}, 0xc00044c3c8?, 0xc00044c2c
0?, 0xf4a6bcad6d815a31?)
github.com/ethereum/go-ethereum/consensus/ethash/ethash.go:84 +0x27
github.com/ethereum/go-ethereum/consensus/beacon.(*Beacon).Seal(0xc00046a80a, {0x2066f20, 0xc000581800}, 0xc0002c6640, 0x9daddf69ca14
7c60?, 0x9e5c859464211001?)
github.com/ethereum/go-ethereum/consensus/beacon/consensus.go:387 +0x9e
github.com/ethereum/go-ethereum/miner.(*worker).taskLoop(0xc0001ce280)
github.com/ethereum/go-ethereum/miner/worker.go:706 +0x2fc
created by github.com/ethereum/go-ethereum/miner.newWorker
github.com/ethereum/go-ethereum/miner/worker.go:329 +0xa8d
root@admission-5: /usr/local/bin/private_ethereum_setup#
```

15. Now do same for node2

```

pranav@DESKTOP-0SHSLJ1: ~/private_ethereum_setup$ geth --datadir node2 \
--port 30307 \
--networkid 12345 \
--bootnodes "enode://65150fcbf2bac27c639de2d15c6265f8135751d09041a248806606ef4a44206a0bf6673458a5dcb1146bda9aa54d511cbcabfd26c807257191ec1c77b46472@127.0.0.1:30305" \
--http --http.port 8546 --http.api "eth,net,web3" \
--authrpc.port 8552
INFO [03-31|14:18:32.384] Maximum peer count                      ETH=50 LES=0 total=50
INFO [03-31|14:18:32.385] Smartcard socket not found, disabling   err="stat /run/pcscd/pcscd.comm: no such file or directory"
INFO [03-31|14:18:32.387] Set global gas cap                       cap=50,000,000
INFO [03-31|14:18:32.387] Initializing the KZG library             backend=gokzg
INFO [03-31|14:18:32.433] Allocated trie memory caches            clean=154.00MiB dirty=256.00MiB
INFO [03-31|14:18:32.433] Using pebble as the backing database
INFO [03-31|14:18:32.433] Allocated cache and file handles       database=/home/pranav/private_ethereum_setup/node2/geth/chaindata cache=512.00MiB handles=524,288
INFO [03-31|14:18:32.479] Opened ancient database                 database=/home/pranav/private_ethereum_setup/node2/geth/chaindata/ancient/chain_readonly=false
INFO [03-31|14:18:32.479] Initialising Ethereum protocol         network=12345 dbversion=nil>
INFO [03-31|14:18:32.480] -----
INFO [03-31|14:18:32.480] Chain ID: 12345 (unknown)
INFO [03-31|14:18:32.480] Consensus: Clique (proof-of-authority)
INFO [03-31|14:18:32.480]
INFO [03-31|14:18:32.480] Pre-Merge hard forks (block based):
INFO [03-31|14:18:32.480]   - Homestead: #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/homestead.md)
INFO [03-31|14:18:32.480]   - Tangerine Whistle (EIP 150): #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/tangerine-whistle.md)
INFO [03-31|14:18:32.480]   - Spurious Dragon/1 (EIP 155): #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/spurious-dragon.md)
INFO [03-31|14:18:32.480]   - Spurious Dragon/2 (EIP 158): #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/spurious-dragon.md)
INFO [03-31|14:18:32.480]   - Byzantium: #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/byzantium.md)
INFO [03-31|14:18:32.480]   - Constantinople: #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/constantinople.md)
INFO [03-31|14:18:32.515] Started P2P networking
INFO [03-31|14:18:32.516] IPC endpoint opened
INFO [03-31|14:18:32.517] Generated JWT secret
INFO [03-31|14:18:32.517] HTTP server started
INFO [03-31|14:18:32.518] WebSocket enabled
INFO [03-31|14:18:32.518] HTTP server started
INFO [03-31|14:18:43.349] Block synchronisation started
INFO [03-31|14:18:43.365] Syncing: state download in progress
INFO [03-31|14:18:43.365] Imported new chain segment
INFO [03-31|14:18:43.365] Syncing: chain download in progress
INFO [03-31|14:18:43.365] Syncing: state download in progress
WARN [03-31|14:18:43.366] Synchronisation failed, retrying
WARN [03-31|14:18:43.366] Unexpected account range packet
INFO [03-31|14:18:43.373] Looking for peers
WARN [03-31|14:18:45.003] Snap syncing, discarded propagated block
INFO [03-31|14:18:53.368] Syncing: state download in progress
INFO [03-31|14:18:53.370] Syncing: chain download in progress
INFO [03-31|14:18:53.371] Syncing: state download in progress
WARN [03-31|14:18:53.371] Unexpected account range packet
INFO [03-31|14:18:53.374] Syncing: state download in progress
INFO [03-31|14:18:53.374] Syncing: state healing in progress
INFO [03-31|14:18:53.385] Rebuilding state snapshot
INFO [03-31|14:18:53.385] Committed new head block
INFO [03-31|14:18:53.385] Resuming state snapshot generation
INFO [03-31|14:18:53.386] Generated state snapshot
INFO [03-31|14:18:53.385] Imported new chain segment
INFO [03-31|14:18:53.392] Looking for peers

```

16. Now we will interact with the blockchain using javascript console
geth attach node1/geth.ipc

```

pranav@DESKTOP-0SHSLJ1:~/private_ethereum_setup$ geth attach node2/geth.ipc
Welcome to the Geth JavaScript console!

instance: Geth/v1.12.0-stable-e501b3b0/linux-amd64/go1.20.3
at block: 32 (Tue Mar 31 2026 14:20:10 GMT+0000 (UTC))
datadir: /home/pranav/private_ethereum_setup/node2
modules: admin:1.0 clique:1.0 debug:1.0 engine:1.0 eth:1.0 miner:1.0 net:1.0 rpc:1.0 txpool:1.0 web3:1.0

To exit, press ctrl-d or type exit
> net.peerCount
1
>

```

17. Interacting with the blockchain



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 10

Aim: Explore the Cryptocurrency Landscape

Roll No.	33
Name	Rujuta Vinit Medhi
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO5:Interpret different Crypto assets and Crypto currencies CO6:Analyze the use of Blockchain with AI, IoT and Cyber Security using case studies.

Experiment: 10

Aim: Explore the Cryptocurrency Landscape

Theory:

Cryptocurrency Landscape

1. Definition – Cryptocurrencies are decentralized digital assets built on blockchain networks, enabling secure, transparent, and immutable transactions without central authority.
2. Market Diversity – Thousands of cryptocurrencies exist, each serving unique roles:
 - Store of Value – Bitcoin, Litecoin.
 - Utility Tokens – Chainlink, Ethereum.
 - Stablecoins – USDT, USDC.
 - Governance Tokens – UNI, AAVE.
3. Volatility – Prices are highly sensitive to market sentiment, technological developments, and regulatory updates.
4. Data Analysis Importance – Tracking historical trends, market capitalization, and trading volume is essential for understanding asset performance.
5. Advancements – Growing use of DeFi platforms, NFT ecosystems, layer-2 scaling, and cross-chain protocols.

Data Collection Process

The program used the CoinGecko API to automate historical data fetching for Chainlink. The retrieved data was processed with Pandas to compute:

- Daily Returns – Percentage change between consecutive days to measure volatility.
- Moving Averages (MA7, MA30) – Short-term and long-term trend indicators.
- Cumulative Returns – Growth factor over time.
- Rolling Volatility – Short-term risk measurement based on daily return fluctuations.

Visualization & Dashboard

Using Matplotlib, a six-panel dashboard was created to display:

1. Price Trend – Daily price movement over the selected period.
2. Price vs Volume – Correlation between trading activity and price changes.
3. Daily Returns Distribution – Statistical spread of daily gains/losses.
4. Moving Averages – Technical analysis trend lines for price movement.
5. Cumulative Returns – Overall growth performance.
6. Rolling Volatility – Market risk trends over time.

Technological Analysis

Beyond market data, the study covered advancements in blockchain technology, including:

- Consensus Mechanisms – PoW (Proof of Work), PoS (Proof of Stake), and hybrid systems.
- Scalability Solutions – Layer-2 protocols, sharding, and sidechains to increase throughput.
- Interoperability Protocols – Chainlink's CCIP, Polkadot, and Cosmos enabling cross-chain data exchange.
- Privacy Features – Zero-knowledge proofs (zk-SNARKs) and ring signatures to ensure transaction confidentiality.

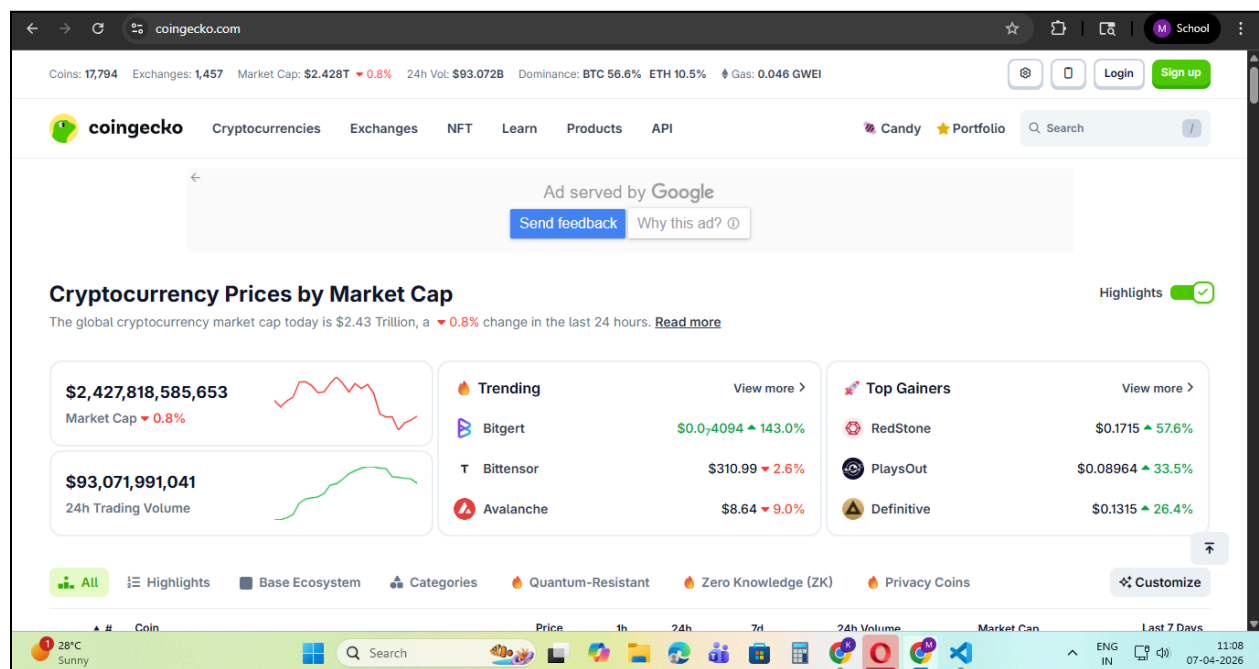
Chainlink's roadmap highlights staking implementation, advanced oracle features, and enhanced cross-chain capabilities, aligning with industry trends toward decentralization, interoperability, and security.

Market Trends & Adoption

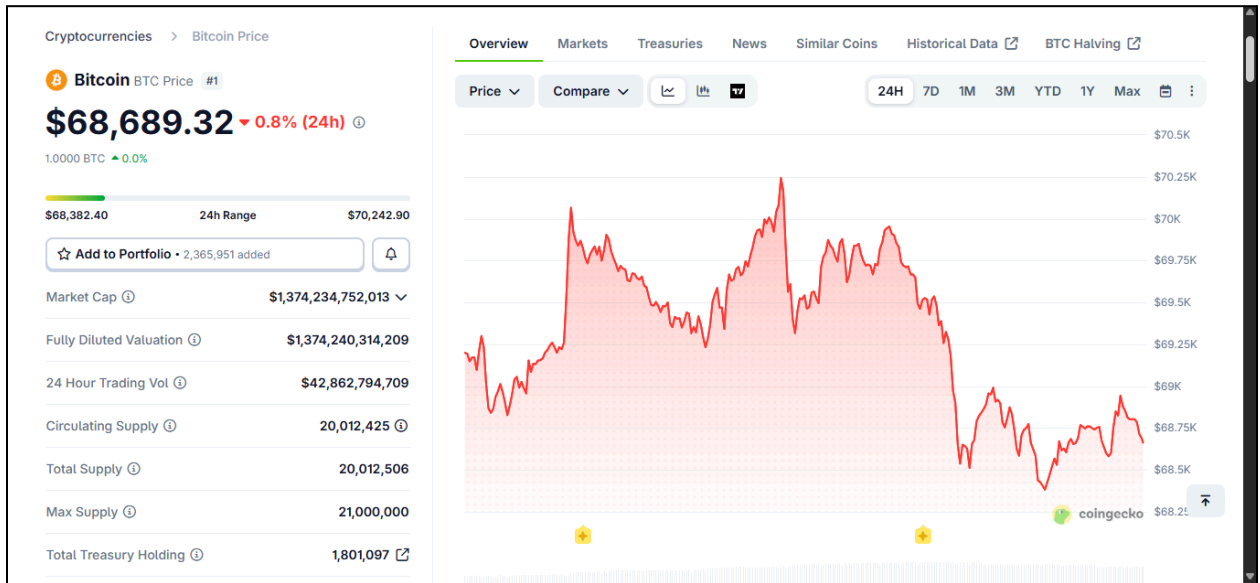
- Growing Merchant Acceptance – More businesses accepting cryptocurrency payments.
- Wallet Growth – Increasing downloads and installations of crypto wallets.
- Network Activity – Higher transaction volumes and active addresses, signaling strong adoption.
- Integration with Traditional Finance – Partnerships between crypto platforms and fintech/payment providers.
- Regulatory Developments – Countries exploring frameworks for safe cryptocurrency adoption.

Implementation:

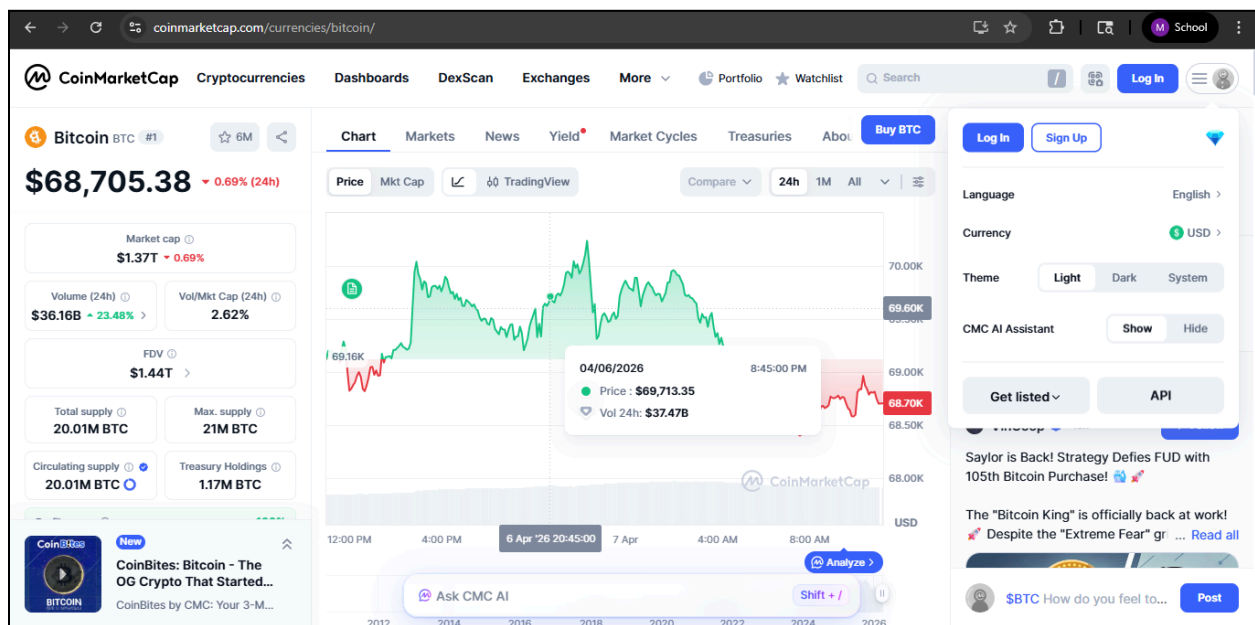
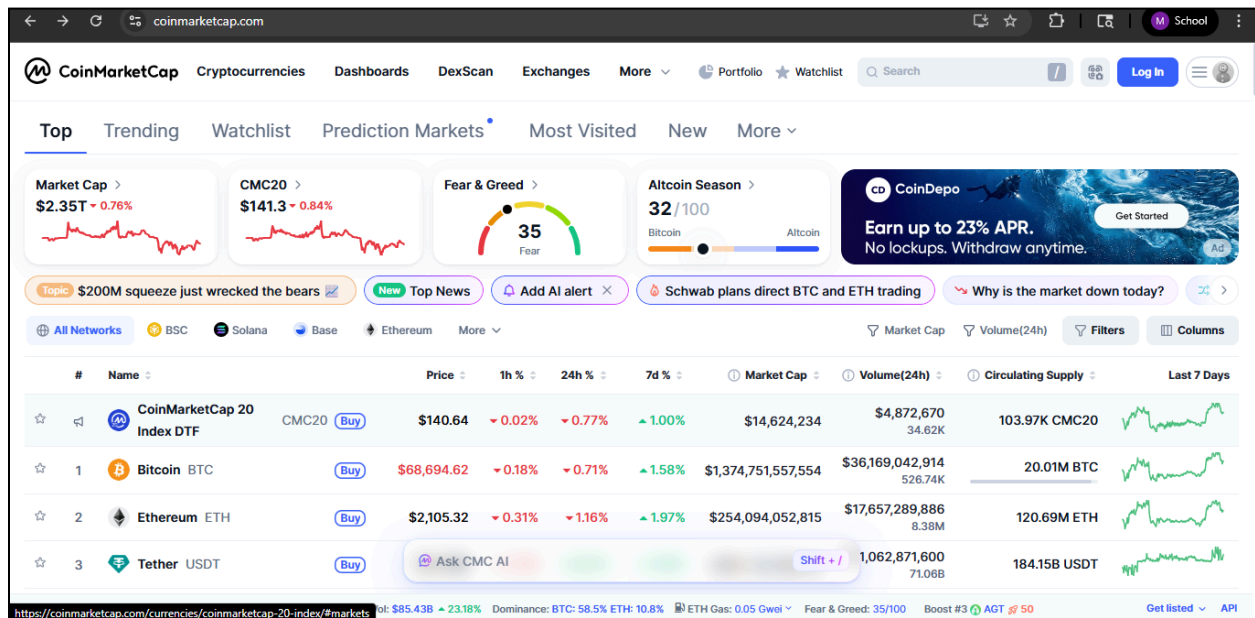
Exploring Cointagecko page-



#	Coin	Price	1h	24h	7d	24h Volume	Market Cap	Last 7 Days
1	Bitcoin BTC	Buy \$68,672.74 ▼ 0.2% ▼ 0.8% ▲ 1.8%				\$42,523,575,764	\$1,374,225,877,398	
2	Ethereum ETH	Buy \$2,105.25 ▼ 0.3% ▼ 1.2% ▲ 1.9%				\$17,156,878,255	\$254,158,182,413	
3	Tether USD	\$0.9999 ▲ 0.0% ▲ 0.0% ▲ 0.1%				\$65,290,470,882	\$184,135,060,815	
4	BNB BNB	Buy \$597.79 ▼ 0.3% ▼ 0.5% ▼ 2.3%				\$742,509,816	\$81,549,954,625	
5	XRP XRP	Buy \$1.31 ▼ 0.4% ▼ 1.9% ▼ 0.9%				\$1,734,842,005	\$80,673,979,841	
6	USDC USDC	\$0.9999 ▼ 0.0% ▲ 0.0% ▲ 0.0%				\$11,682,951,056	\$77,838,047,171	
7	Solana SOL	Buy \$79.71 ▼ 0.2% ▼ 2.7% ▼ 4.5%				\$2,716,924,752	\$45,739,900,622	
8	TRON TRX	Buy \$0.3163 ▲ 0.0% ▼ 0.4% ▼ 0.8%				\$410,740,406	\$29,972,805,075	



Exploring CoinMarketCap-



Code:

(Imported APIs to extract data of market)

```
import requests
import matplotlib.pyplot as plt
import pandas as pd
```

```
# ----- Fetch Data -----
```

```

def get_historical_data(coin_id, days='180', currency='usd'):
    url = f'https://api.coingecko.com/api/v3/coins/{coin_id}/market_chart'
    params = {'vs_currency': currency, 'days': days}
    response = requests.get(url, params=params)
    return response.json()

# ----- Dashboard Function -----
def chainlink_dashboard(days='180'):
    coin_id = 'bitcoin'
    data = get_historical_data(coin_id, days)

    # Create DataFrame
    prices_df = pd.DataFrame(data['prices'], columns=['timestamp', 'price'])
    volumes_df = pd.DataFrame(data['total_volumes'], columns=['timestamp', 'volume'])

    prices_df['timestamp'] = pd.to_datetime(prices_df['timestamp'], unit='ms')
    volumes_df['timestamp'] = pd.to_datetime(volumes_df['timestamp'], unit='ms')

    df = pd.merge(prices_df, volumes_df, on='timestamp')
    df.set_index('timestamp', inplace=True)

    # Calculate extra metrics
    df['returns'] = df['price'].pct_change()
    df['MA7'] = df['price'].rolling(window=7).mean()
    df['MA30'] = df['price'].rolling(window=30).mean()
    df['cumulative'] = (1 + df['returns']).cumprod()
    df['volatility'] = df['returns'].rolling(window=7).std() * 100

    # ----- Plot Dashboard -----
    fig, axes = plt.subplots(3, 2, figsize=(16, 12))
    fig.suptitle(f'{coin_id.capitalize()} Dashboard (Last {days} Days)', fontsize=16, fontweight='bold')

    # 1. Price Trend
    axes[0, 0].plot(df.index, df['price'], color='blue')
    axes[0, 0].set_title('Price Trend')
    axes[0, 0].set_ylabel('Price (USD)')
    axes[0, 0].set_xlabel('Year-Month') # Add xlabel
    axes[0, 0].grid(True)

    # 2. Price vs Volume
    axes[0, 1].plot(df.index, df['price'], color='blue', label='Price')
    axes[0, 1].bar(df.index, df['volume'], color='orange', alpha=0.3, label='Volume')
    axes[0, 1].set_title('Price vs Volume')
    axes[0, 1].set_xlabel('Year-Month') # Add xlabel

```

```

axes[0, 1].set_ylabel('Price (USD) / Volume') # Add ylabel
axes[0, 1].legend()
axes[0, 1].grid(True)

# 3. Daily Returns Distribution
axes[1, 0].hist(df['returns'].dropna(), bins=50, alpha=0.7, color='purple')
axes[1, 0].set_title('Daily Returns Distribution')
axes[1, 0].set_xlabel('Daily Return')
axes[1, 0].set_ylabel('Frequency')
axes[1, 0].grid(True)

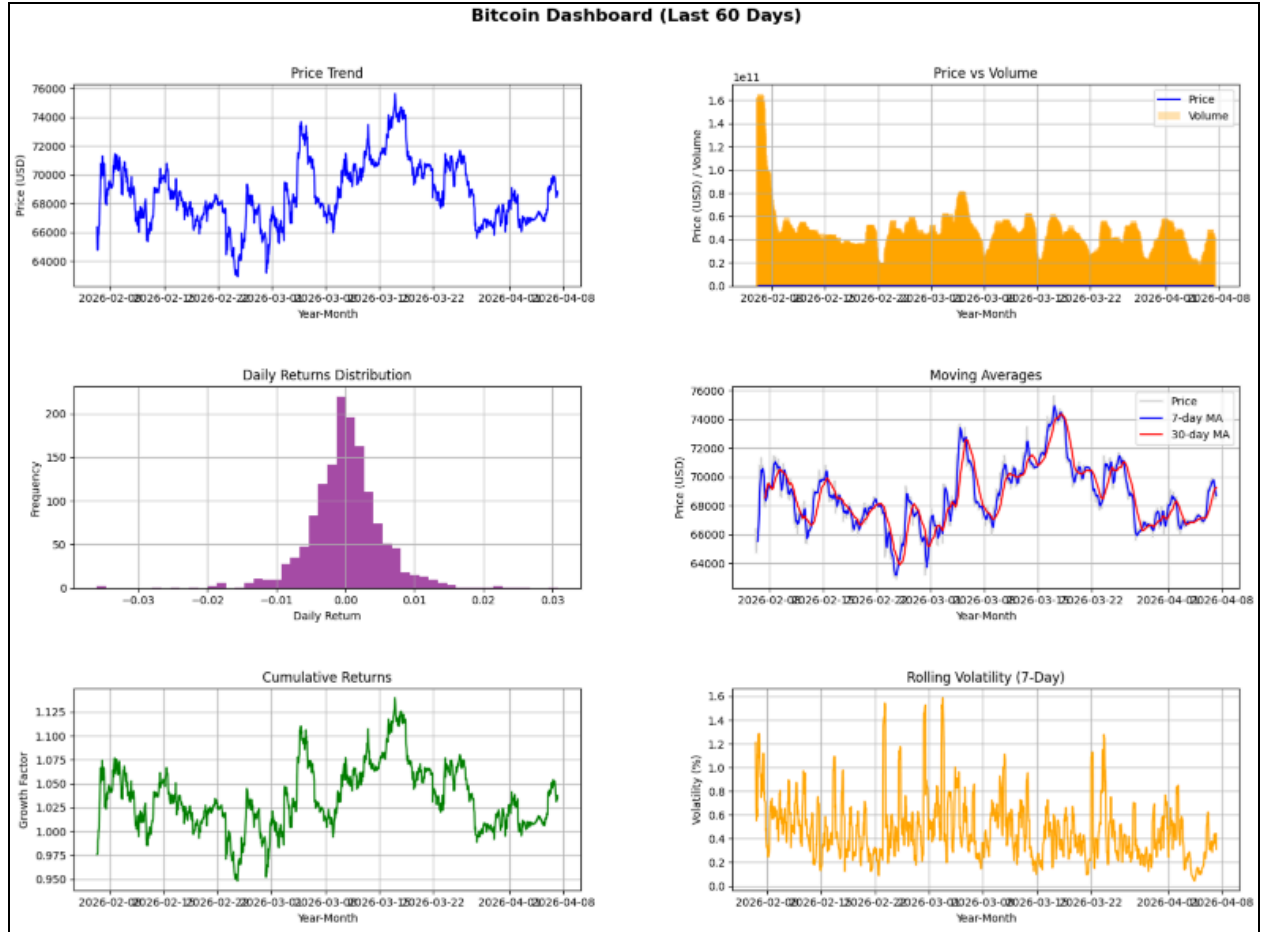
# 4. Moving Averages
axes[1, 1].plot(df.index, df['price'], color='lightgray', label='Price')
axes[1, 1].plot(df.index, df['MA7'], color='blue', label='7-day MA')
axes[1, 1].plot(df.index, df['MA30'], color='red', label='30-day MA')
axes[1, 1].set_title('Moving Averages')
axes[1, 1].set_xlabel('Year-Month')
axes[1, 1].set_ylabel('Price (USD)')
axes[1, 1].legend()
axes[1, 1].grid(True)

# 5. Cumulative Returns
axes[2, 0].plot(df.index, df['cumulative'], color='green')
axes[2, 0].set_title('Cumulative Returns')
axes[2, 0].set_ylabel('Growth Factor')
axes[2, 0].set_xlabel('Year-Month')
axes[2, 0].grid(True)

# 6. Volatility
axes[2, 1].plot(df.index, df['volatility'], color='orange')
axes[2, 1].set_title('Rolling Volatility (7-Day)')
axes[2, 1].set_ylabel('Volatility (%)')
axes[2, 1].set_xlabel('Year-Month')
axes[2, 1].grid(True)

plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.subplots_adjust(hspace=0.5, wspace=0.3) # Adjust vertical and horizontal spacing
plt.show()
# ----- Run Dashboard -----
chainlink_dashboard('60')

```



Conclusion:

By combining automated market data processing, visual trend analysis, blockchain technology evaluation, and adoption metrics, this experiment presents a holistic view of the cryptocurrency ecosystem. It demonstrates how blockchain's technical foundations connect with real-world market behavior, providing insights into both current trends and future advancements in the digital asset industry.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 11

Aim: Implementation of Hyperledger Fabric Network and Asset Management using Chaincode

Roll No.	33
Name	Rujuta Vinit Medhi
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO4:Develop applications in permissioned Hyperledger Fabric network.

Experiment :11

Title:Implementation of Hyperledger Fabric Network and Asset Management using Chaincode

Aim:To implement a Hyperledger Fabric network, deploy chaincode, and perform asset creation, querying, and transfer operations.

Theory:

Hyperledger Fabric is an open-source enterprise-grade blockchain platform designed for building distributed applications. Unlike public blockchains, it is a **permissioned network**, meaning only authorized participants can access and perform transactions.

The architecture of Hyperledger Fabric consists of key components:

- **Peers** – Maintain ledger and run chaincode
- **Orderers** – Ensure transaction ordering
- **Certificate Authority (CA)** – Provides identities for participants
- **Channels** – Private communication layers for transactions

Chaincode (smart contracts) defines the business logic that governs how assets are created, updated, and transferred on the blockchain.

To run Fabric, tools like:

- Docker (for containerization)
- Git (for repository management)
- Node.js (for chaincode execution)
- Windows Subsystem for Linux (for Linux environment on Windows) are required.

This experiment demonstrates how to:

1. Set up a Fabric test network
2. Deploy chaincode
3. Perform asset lifecycle operations

Prerequisites

- Windows Subsystem for Linux (WSL)
- Ubuntu
- Docker

- Git
- Node.js & npm

Procedure:

Step 1: Update the System

Command:

```
sudo apt update  
sudo apt upgrade -y
```

```
root@INFT506-34:~# sudo apt update  
Get:1 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]  
Get:2 https://deb.nodesource.com/node_18.x nodistro InRelease [12.1 kB]  
Hit:3 http://archive.ubuntu.com/ubuntu jammy InRelease  
Get:4 http://archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]  
Get:5 http://archive.ubuntu.com/ubuntu jammy-backports InRelease [127 kB]  
Get:6 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages [3364 kB]  
Get:7 https://deb.nodesource.com/node_18.x nodistro/main amd64 Packages [12.0 kB]  
Ign:1 http://security.ubuntu.com/ubuntu jammy-security InRelease  
Get:1 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]  
Get:8 http://security.ubuntu.com/ubuntu jammy-security/main amd64 Packages [3068 kB]  
Get:9 http://archive.ubuntu.com/ubuntu jammy-updates/main Translation-en [512 kB]  
Get:10 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 c-n-f Metadata [19.6 kB]  
Get:11 http://archive.ubuntu.com/ubuntu jammy-updates/restricted amd64 Packages [5608 kB]  
Get:12 http://security.ubuntu.com/ubuntu jammy-security/main Translation-en [438 kB]  
Get:13 http://security.ubuntu.com/ubuntu jammy-security/main amd64 c-n-f Metadata [14.1 kB]  
Get:14 http://security.ubuntu.com/ubuntu jammy-security/restricted amd64 Packages [5356 kB]  
Get:15 http://archive.ubuntu.com/ubuntu jammy-updates/restricted Translation-en [1075 kB]  
Get:16 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 Packages [1262 kB]  
Get:17 http://security.ubuntu.com/ubuntu jammy-security/restricted Translation-en [1025 kB]  
Get:18 http://archive.ubuntu.com/ubuntu jammy-updates/universe Translation-en [316 kB]  
Get:19 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 c-n-f Metadata [30.5 kB]  
Get:20 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 Packages [1025 kB]  
Get:21 http://security.ubuntu.com/ubuntu jammy-security/universe Translation-en [226 kB]  
Get:22 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 c-n-f Metadata [22.8 kB]  
Fetched 23.7 MB in 1min 57s (203 kB/s)  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
47 packages can be upgraded. Run 'apt list --upgradable' to see them.  
root@INFT506-34:~#
```

```
root@INFT506-34:~# sudo apt upgrade -y  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
Calculating upgrade... Done  
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.  
root@INFT506-34:~#
```

Explanation:

This step updates the system package index and upgrades all installed packages to their latest versions. It ensures that the system has the most recent security patches and compatible libraries.

required for Hyperledger Fabric. Running this step avoids dependency conflicts and installation failures later.

Step 2: Install Required Dependencies

Command:

```
sudo apt install jq curl git nodejs npm -y
node -v
npm -v
git --version
docker --version
docker ps
```

```
root@INFT506-34:~# node -v
v18.20.8
root@INFT506-34:~# npm -v
10.8.2
root@INFT506-34:~# git --version
git version 2.34.1
root@INFT506-34:~# docker --version
Docker version 29.1.5, build 0e6fee6
root@INFT506-34:~# docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
5d0000f6b4e3	mariadb:10.6	"docker-entrypoint.s..."	6 weeks ago	Up 3 minutes	3306/tcp
3d67c106ff3d	ghcr.io/zammad/zammad:6.5.2-67	"/docker-entrypoint..."	2 months ago	Up 3 minutes	0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
c2acd1eae6e0	ghcr.io/zammad/zammad:6.5.2-67	"/docker-entrypoint..."	2 months ago	Up 3 minutes	
fd57f58245b0	ghcr.io/zammad/zammad:6.5.2-67	"/docker-entrypoint..."	2 months ago	Up 3 minutes	
64d9da24d3e4	ghcr.io/zammad/zammad:6.5.2-67	"/docker-entrypoint..."	2 months ago	Up 3 minutes	
5f2a8d964ea5	ghcr.io/zammad/zammad:6.5.2-67	"/docker-entrypoint..."	2 months ago	Up 3 minutes	
09407446a971	postgres:17.7-alpine	"docker-entrypoint.s..."	2 months ago	Up 3 minutes	5432/tcp
e91a72c5e3ee	redis:7.4.7-alpine	"docker-entrypoint.s..."	2 months ago	Up 3 minutes	6379/tcp
77c366a563e3	memcached:1.6.40-alpine	"docker-entrypoint.s..."	2 months ago	Up 3 minutes	11211/tcp

Explanation:

This step installs all necessary tools required to run Hyperledger Fabric. Docker is used to run blockchain components as containers, Git is used to clone repositories, and Node.js with npm is required for chaincode execution. Version checks confirm successful installation, while **docker ps** ensures Docker is running properly.

Step 3: Clone Fabric Samples

Command:

```
git clone https://github.com/hyperledger/fabric-samples
cd fabric-samples
```



```

root@INFT506-34:~# git clone https://github.com/hyperledger/fabric-samples
Cloning into 'fabric-samples'...
remote: Enumerating objects: 15618, done.
remote: Counting objects: 100% (221/221), done.
remote: Compressing objects: 100% (126/126), done.
remote: Total 15618 (delta 149), reused 95 (delta 95), pack-reused 15397 (from 3)
Receiving objects: 100% (15618/15618), 24.09 MiB | 369.00 KiB/s, done.
Resolving deltas: 100% (8801/8801), done.
root@INFT506-34:~# |

```

```

root@INFT506-34:~# cd fabric-samples
root@INFT506-34:~/fabric-samples# curl -sSLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fa^C
root@INFT506-34:~/fabric-samples# curl -sSLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary
./install-fabric.sh docker

Clone hyperledger/fabric-samples repo

==> Already in fabric-samples repo
fabric-samples v2.5.15 does not exist, defaulting to main. fabric-samples main branch is intended to work with recent versions of fabric.

Pull Hyperledger Fabric binaries

====> Downloading version 2.5.15 platform specific fabric binaries
====> Downloading: https://github.com/hyperledger/fabric/releases/download/v2.5.15/hyperledger-fabric-linux-amd64-2.5.15.tar.gz
====> Will unpack to: /root/fabric-samples

```

% Total	% Received	% Xferd	Average Speed	Time	Time	Time	Current
			Dload Upload	Total	Spent	Left	Speed
0	0	0	0	0	0	0	0
16	120M	16	20.0M	0	0	946k	0
				0:02:10	0:00:21	0:01:49	1121k

Explanation:

This step downloads the official Hyperledger Fabric sample repository. It contains pre-built network configurations, scripts, and example chaincodes. Moving into this directory allows us to access all required files for setting up the blockchain network.

Step 4: Install Fabric Binaries and Images

Command:

```

curl -sSLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary
./install-fabric.sh docker

```

```

root@INFT506-34:~# git clone https://github.com/hyperledger/fabric-samples
fatal: destination path 'fabric-samples' already exists and is not an empty directory.
root@INFT506-34:~# cd fabric-samples
root@INFT506-34:~/fabric-samples# curl -sSLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary
./install-fabric.sh docker

Clone hyperledger/fabric-samples repo

==> Already in fabric-samples repo
fabric-samples v2.5.15 does not exist, defaulting to main. fabric-samples main branch is intended to work with recent versions of fabric.

Pull Hyperledger Fabric binaries

====> Downloading version 2.5.15 platform specific fabric binaries
====> Downloading: https://github.com/hyperledger/fabric/releases/download/v2.5.15/hyperledger-fabric-linux-amd64-2.5.15.tar.gz
====> Will unpack to: /root/fabric-samples

```

% Total	% Received	% Xferd	Average Speed	Time	Time	Time	Current
0	0	0	0	0	0	0	0
16	120M	16	20.0M	0	0	946k	0
				0:02:10	0:00:21	0:01:49	1121k

Show desktop

Pull Hyperledger Fabric docker images

```
FABRIC_IMAGES: peer orderer ccenv baseos
====> Pulling fabric Images
====> ghcr.io/hyperledger/fabric-peer:2.5.15
2.5.15: Pulling from hyperledger/fabric-peer
Digest: sha256:3e25adc31a757ee19dd8a24afdc9ac5aa46a86b3cb56fc93068b0e594706d6aa
Status: Image is up to date for ghcr.io/hyperledger/fabric-peer:2.5.15
ghcr.io/hyperledger/fabric-peer:2.5.15
====> ghcr.io/hyperledger/fabric-orderer:2.5.15
2.5.15: Pulling from hyperledger/fabric-orderer
Digest: sha256:9972c209be47f45e377a24c88f2e3d21fae4dc224751c68bdd33f2e7a603ffa8
Status: Image is up to date for ghcr.io/hyperledger/fabric-orderer:2.5.15
ghcr.io/hyperledger/fabric-orderer:2.5.15
====> ghcr.io/hyperledger/fabric-ccenv:2.5.15
2.5.15: Pulling from hyperledger/fabric-ccenv
Digest: sha256:86dded7eda9268e2cbf3691cb952edf545dc5faea1a79bfc9b47ec47d3ecfa9e
Status: Image is up to date for ghcr.io/hyperledger/fabric-ccenv:2.5.15
ghcr.io/hyperledger/fabric-ccenv:2.5.15
====> ghcr.io/hyperledger/fabric-baseos:2.5.15
2.5.15: Pulling from hyperledger/fabric-baseos
Digest: sha256:654bd4554bf97c70902e56d530294795372518da7ae2a7cdfccbe397a878c513
Status: Image is up to date for ghcr.io/hyperledger/fabric-baseos:2.5.15
ghcr.io/hyperledger/fabric-baseos:2.5.15
====> Pulling fabric ca Image
====> ghcr.io/hyperledger/fabric-ca:1.5.17
1.5.17: Pulling from hyperledger/fabric-ca
Digest: sha256:453a0a727e82c4960b797e379adc231e853ee23ae7526db53afef77b5074117e
Status: Image is up to date for ghcr.io/hyperledger/fabric-ca:1.5.17
```

Explanation:

This step downloads and executes a script that installs Hyperledger Fabric binaries and required Docker images. It pulls essential components like peer, orderer, and certificate authority images. These components are necessary to create and manage the blockchain network.

Step 5: Verify Docker Images

Command:

docker images | grep fabric

```
root@INFT506-34:~/fabric-samples# docker images | grep fabric
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
ghcr.io/hyperledger/fabric-baseos:2.5.15      654bd4554bf9      250MB      80.4MB
ghcr.io/hyperledger/fabric-ca:1.5.17          453a0a727e82      381MB      123MB
ghcr.io/hyperledger/fabric-ccenv:2.5.15       86dded7eda92      1GB        255MB
ghcr.io/hyperledger/fabric-orderer:2.5.15     9972c209be47      178MB      49.5MB
ghcr.io/hyperledger/fabric-peer:2.5.15        3e25adc31a75      230MB      66.2MB
hyperledger/fabric-baseos:2.5                 654bd4554bf9      250MB      80.4MB
hyperledger/fabric-baseos:2.5.15              654bd4554bf9      250MB      80.4MB
hyperledger/fabric-baseos:latest              654bd4554bf9      250MB      80.4MB
hyperledger/fabric-ca:1.5                     453a0a727e82      381MB      123MB
hyperledger/fabric-ca:1.5.17                  453a0a727e82      381MB      123MB
hyperledger/fabric-ca:latest                  453a0a727e82      381MB      123MB
hyperledger/fabric-ccenv:2.5                  86dded7eda92      1GB        255MB
hyperledger/fabric-ccenv:2.5.15               86dded7eda92      1GB        255MB
hyperledger/fabric-ccenv:latest               86dded7eda92      1GB        255MB
hyperledger/fabric-orderer:2.5                9972c209be47      178MB      49.5MB
hyperledger/fabric-orderer:2.5.15             9972c209be47      178MB      49.5MB
hyperledger/fabric-orderer:latest             9972c209be47      178MB      49.5MB
hyperledger/fabric-peer:2.5                   3e25adc31a75      230MB      66.2MB
hyperledger/fabric-peer:2.5.15                3e25adc31a75      230MB      66.2MB
hyperledger/fabric-peer:latest                3e25adc31a75      230MB      66.2MB
root@INFT506-34:~/fabric-samples# |
```

Explanation:

This command checks whether all required Hyperledger Fabric Docker images are successfully downloaded. It ensures that components such as peer and order are available before starting the network. Missing images may cause deployment errors.

Step 6: Set Environment Variables

Command:

```
export PATH=$PATH:$HOME/fabric-samples/bin
export FABRIC_CFG_PATH=$HOME/fabric-samples/config
peer version
```

```
root@INFT506-34:~/fabric-samples# export PATH=$PATH:$HOME/fabric-samples/bin
root@INFT506-34:~/fabric-samples# export FABRIC_CFG_PATH=$HOME/fabric-samples/config
root@INFT506-34:~/fabric-samples# peer version
peer:
  Version: v2.5.15
  Commit SHA: 83c7930
  Go version: go1.26.0
  OS/Arch: linux/amd64
  Chaincode:
    Base Docker Label: org.hyperledger.fabric
    Docker Namespace: hyperledger

root@INFT506-34:~/fabric-samples# |
```

Explanation:

This step sets environment variables required to run Fabric CLI commands. The PATH variable allows access to Fabric binaries, while FABRIC_CFG_PATH points to configuration files. Running the peer **version** verifies that the setup is successful.

Step 7: Start Network and Create Channel

Command:

```
cd ~/fabric-samples/test-network
```

```
./network.sh up createChannel
```

```
root@INF1506-34:~/fabric-samples# cd ~/fabric-samples/test-network
./network.sh up createChannel
Using docker and docker-compose
Creating channel 'mychannel'.
If network is not up, starting nodes with CLI timeout of '5' tries and CLI delay of '3' seconds and using database 'leveldb with crypto from 'crypto
gen'
Bringing up network
LOCAL_VERSION=v2.5.15
DOCKER_IMAGE_VERSION=v2.5.15
/root/fabric-samples/test-network/./bin/cryptogen
Generating certificates using cryptogen tool
Creating Org1 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org1.yaml --output=organizations
org1.example.com
+ res=0
Creating Org2 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org2.yaml --output=organizations
org2.example.com
+ res=0
Creating Orderer Org Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-orderer.yaml --output=organizations
+ res=0
Generating CCP files for Org1 and Org2
WARN[0000] No services to build
[*] up 7/7
✓Network fabric_test Created 0.0s
✓Volume compose_orderer.example.com Created 0.0s
✓Volume compose_peer0.org1.example.com Created 0.0s
✓Volume compose_peer0.org2.example.com Created 0.0s
```

```
ed with status (VALID) at localhost:7051
2026-04-07 11:43:58.528 +0530 0002 INFO [chaincodeCmd] ClientWait -> txid [08e465dad2f26121c0b15c68e167f2ebda1bf2c32295ae2423e9bb7c58931ce2] committ
ed with status (VALID) at localhost:7051
Chaincode definition committed on channel 'mychannel'
Using organization 1
Querying chaincode definition on peer0.org1 on channel 'mychannel'...
Attempting to Query committed status on peer0.org1. Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vsc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org1 on channel 'mychannel'
Using organization 2
Querying chaincode definition on peer0.org2 on channel 'mychannel'...
Attempting to Query committed status on peer0.org2. Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vsc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org2 on channel 'mychannel'
Chaincode initialization is not required
```

Explanation:

This step initializes the blockchain network by starting Docker containers for peers and orderers. It also creates a communication channel where transactions will occur. Certificates and configurations are automatically generated during this process.

Step 8: Deploy Chaincode

Command:

```
./network.sh deployCC \
  -ccn basic \
  -ccp ../asset-transfer-basic/chaincode-javascript \
  -ccl javascript
```

```

root@19f1506-34:~/fabric-samples/test-network# ./network.sh deployCC -ccn basic -ccp ../asset-transfer-basic/chaincode-javascript -ccl javascript
ipt
Using docker and docker-compose
deploying chaincode on channel 'mychannel'
executing with the following
- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
- CC_END_POLICY: NA
- CC_COLL_CONFIG: NA
- CC_INIT_FCN: NA
- DELAY: 3
- MAX_RETRY: 5
- VERBOSE: false
executing with the following
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
+ '[' false = true ']'
+ peer lifecycle chaincode package basic.tar.gz --path ../asset-transfer-basic/chaincode-javascript --lang node --label basic_1.0
+ res=0
Chaincode is packaged
Installing chaincode on peer0.org1...
Using organization 1
+ peer lifecycle chaincode queryinstalled --output json
+ jq -r 'try (.installed_chaincodes[].package_id)'
+ grep '^basic_1.0:1d4d445573112813889f87c307bc2b5f5be664c87b24c035bee15cbcc7f59b629$'
+ test 1 -ne 0
+ peer lifecycle chaincode install basic.tar.gz

```

Explanation:

This step deploys the smart contract (chaincode) onto the blockchain network. The chaincode defines the business logic for asset operations such as creation, transfer, and querying. Once deployed, it is ready to interact with the ledger.

Step 9: Set Peer Environment

Command:

```

export FABRIC_CFG_PATH=$HOME/fabric-samples/config
export PATH=$PATH:$HOME/fabric-samples/bin
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export
CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export
CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051

```

```
root@INFT506-34:~/fabric-samples/test-network# export FABRIC_CFG_PATH=$HOME/fabric-samples/config
export PATH=$PATH:$HOME/fabric-samples/bin
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051
root@INFT506-34:~/fabric-samples/test-network# |
```

Explanation:

This step configures the identity and communication settings of the peer node. It specifies organization details, TLS certificates, and network addresses. These settings are required for secure interaction with the blockchain network.

Step 10: Initialize Ledger

Command:

```
chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile
$HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/
orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \
-C mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/p
eer0.org1.example.com/tls/ca.crt \
--peerAddresses localhost:9051 \
--tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/p
eer0.org2.example.com/tls/ca.crt \
--waitForEvent \
-c '{"function": "InitLedger", "Args": []}'
```

```

root@INFT506-34:~/fabric-samples/test-network# peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \
-C mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt \
--peerAddresses localhost:9051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt \
--waitForEvent \
-c '{"Function":"InitLedger","Args":[]}'
2026-04-07 11:52:33.509 +0530 0001 INFO [chaincodeCmd] ClientWait -> txid [cbe2e1a8048ef95418c228b192dee1d7ff4a14947549ab0a8166865435ecc369] committed with status (VALID) at localhost:7051
2026-04-07 11:52:33.550 +0530 0002 INFO [chaincodeCmd] ClientWait -> txid [cbe2e1a8048ef95418c228b192dee1d7ff4a14947549ab0a8166865435ecc369] committed with status (VALID) at localhost:9051
2026-04-07 11:52:33.550 +0530 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200
root@INFT506-34:~/fabric-samples/test-network#

```

Explanation:

This step initializes the blockchain ledger with default sample assets. It writes initial data to the ledger, which can later be queried or modified. This operation is a transaction that gets recorded on the blockchain.

Step 11: Query All Assets

Command:

```
peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
```

```

root@INFT506-34:~/fabric-samples/test-network# peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"docType":"asset"}, {"AppraisedValue":400,"Color":"red","ID":"asset2","Owner":"Brad","Size":5,"docType":"asset"}, {"AppraisedValue":500,"Color":"green","ID":"asset3","Owner":"Jin Soo","Size":10,"docType":"asset"}, {"AppraisedValue":600,"Color":"yellow","ID":"asset4","Owner":"Max","Size":10,"docType":"asset"}, {"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"}, {"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Michel","Size":15,"docType":"asset"}]
root@INFT506-34:~/fabric-samples/test-network#

```

Explanation:

This step retrieves all assets stored in the ledger. It performs a read-only operation and displays current data without modifying the blockchain. It helps verify that the ledger has been initialized correctly.

Step 12: Read Specific Asset

Command:

```
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset5"]}'
```

```

root@INFT506-34:~/fabric-samples/test-network# peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset5"]}'
{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"}
root@INFT506-34:~/fabric-samples/test-network#

```

Explanation:

This step retrieves detailed information of a specific asset using its ID. It demonstrates how individual records can be accessed from the ledger. This is also a read-only operation.

Step 13: Create New Asset

Command:

peer chaincode invoke \

-o localhost:7050 \

--ordererTLSHostnameOverride orderer.example.com \

--tls \

--cafile

\$HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/
orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \

-C mychannel \

-n basic \

--peerAddresses localhost:7051 \

--tlsRootCertFiles

\$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/p
eer0.org1.example.com/tls/ca.crt \

--waitForEvent \

-c '{"function": "CreateAsset", "Args": ["asset10", "yellow", "10", "Alice", "500"]}'

```
root@INF1506-34:~/fabric-samples/test-network# peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.examp
e.com-cert.pem \
-C mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt \
--waitForEvent \
-c '{"function": "CreateAsset", "Args": ["asset10", "yellow", "10", "Alice", "500"]}'
2026-04-07 12:00:21.104 +0530 0001 INFO [chaincodeCmd] ClientWait -> txid [9b065c629e96af164059bcb494038aa21e9ee4b40281d06d913b1c4bdb414d22] committe
d with status (VALID) at localhost:7051
2026-04-07 12:00:21.105 +0530 0002 INFO [chaincodeCmd] ClientWait -> txid [9b065c629e96af164059bcb494038aa21e9ee4b40281d06d913b1c4bdb414d22] committe
d with status (VALID) at localhost:7051
2026-04-07 12:00:21.105 +0530 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200 payload:{"ID":"\
asset10", "Color":"yellow", "Size":10, "Owner":"Alice", "AppraisedValue":500}
root@INF1506-34:~/fabric-samples/test-network#
```

```
root@INF1506-34:~/fabric-samples/test-network# peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.examp
e.com-cert.pem \
-C mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt \
--waitForEvent \
-c '{"function": "InitLedger", "Args": []}'
2026-04-07 11:59:20.742 +0530 0001 INFO [chaincodeCmd] ClientWait -> txid [e3d456a0f6ee4c6132a8e71ea451491f49cb1f2926215aefc7691e0cfa0c3401] committe
d with status (VALID) at localhost:7051
2026-04-07 11:59:20.743 +0530 0002 INFO [chaincodeCmd] ClientWait -> txid [e3d456a0f6ee4c6132a8e71ea451491f49cb1f2926215aefc7691e0cfa0c3401] committe
d with status (VALID) at localhost:7051
2026-04-07 11:59:20.743 +0530 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200
root@INF1506-34:~/fabric-samples/test-network# peer chaincode query \
-C mychannel \
-n basic \
-c '{"Args": ["GetAllAssets"]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"docType":"asset"}, {"AppraisedValue":400,"Color":"red","ID":"asset2","
Owner":"Brad","Size":5,"docType":"asset"}, {"AppraisedValue":500,"Color":"green","ID":"asset3","Owner":"Jin Soo","Size":10,"docType":"asset"}, {"Appra
isedValue":600,"Color":"yellow","ID":"asset4","Owner":"Max","Size":10,"docType":"asset"}, {"AppraisedValue":700,"Color":"black","ID":"asset5","Owner"
:"Adriana","Size":15,"docType":"asset"}, {"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Michel","Size":15,"docType":"asset"}]
root@INF1506-34:~/fabric-samples/test-network# peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
```


Explanation:

This step creates a new asset on the blockchain with specified attributes such as ID, color, size, owner, and value. The owner name is set to Manav, which confirms that the transaction was performed by the user. This operation updates the ledger.

Step 14: Transfer Asset

Command:

```
peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile
$HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/
orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \
  -C mychannel \
  -n basic \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/p
eer0.org1.example.com/tls/ca.crt \
  --peerAddresses localhost:9051 \
  --tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/p
eer0.org2.example.com/tls/ca.crt \
  --waitForEvent \
  -c '{"function": "TransferAsset", "Args": ["asset10", "Bob"]}'
```

```
root@INF1596-34:~/fabric-samples/test-network# peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.examl
e.com-cert.pem \
-C mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt \
--peerAddresses localhost:9051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt \
--waitForEvent \
-c '{"function": "TransferAsset", "Args": ["asset10", "Bob"]}'
2026-04-07 12:01:59.356 +0530 0001 INFO [chaincodeCmd] ClientWait -> txid [eee32197d9cd297abdd9c5800d4a0a48934210e17f8ea648fc3298aa45db7ab4] committ
ed with status (VALID) at localhost:9051
2026-04-07 12:01:59.387 +0530 0002 INFO [chaincodeCmd] ClientWait -> txid [eee32197d9cd297abdd9c5800d4a0a48934210e17f8ea648fc3298aa45db7ab4] committ
ed with status (VALID) at localhost:7051
2026-04-07 12:01:59.387 +0530 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200 payload:"Alice"
root@INF1596-34:~/fabric-samples/test-network#
```

Explanation:

This step transfers ownership of an asset from one user to another. It updates the owner field in the ledger. This demonstrates how blockchain ensures secure and traceable asset transfers.

Step 15: Stop Network

Command:

./network.sh down

```
root@INFT506-34:~/fabric-samples/test-network# ./network.sh down
Using docker and docker-compose
Stopping network
[+] down 7/7
✓Container orderer.example.com      Removed      0.8s
✓Container peer0.org2.example.com    Removed      1.4s
✓Container peer0.org1.example.com    Removed      1.5s
✓Network fabric_test                Removed      0.4s
✓Volume compose_peer0.org1.example.com Removed      0.0s
✓Volume compose_orderer.example.com  Removed      0.0s
✓Volume compose_peer0.org2.example.com Removed      0.0s
Error response from daemon: get docker_orderer.example.com: no such volume
Error response from daemon: get docker_peer0.org1.example.com: no such volume
Error response from daemon: get docker_peer0.org2.example.com: no such volume
Removing remaining containers
Removing generated chaincode docker images
Untagged: dev-peer0.org2.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5f5be664c87b24c035bee15cbcc7f59b629-c68f8ecc2a0ff0cc1497615f5e33a8bfa908
e0fd3555f864cf1398b52d341cb:latest
Deleted: sha256:e396fa43457a17b394bb3b26314fb3cb58ba10f8bc73879e81f3cde20298e7e5
Untagged: dev-peer0.org1.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5f5be664c87b24c035bee15cbcc7f59b629-d3cf9b1f4b9747cc8433ac0e74bfaade1cf
0b97f5c1163835f604aa885d60dc:latest
Deleted: sha256:429fe488758bb9495f7cd4969cc5336c04932468f2eaaa3bd0bf2b98cde109ff
Unable to find image 'busybox:latest' locally
latest: Pulling from library/busybox
481282afbc43: Pull complete
aecba0016ef2: Download complete
Digest: sha256:1487d0af5f52b4ba31c7e465126ee2123fe3f2305d638e7827681e7cf6c83d5e
Status: Downloaded newer image for busybox:latest
root@INFT506-34:~/fabric-samples/test-network# |
```

Conclusion :

Successfully implemented a Hyperledger Fabric network, deployed chaincode, and performed asset creation, querying, and transfer operations.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801: Blockchain and DLT Lab
Experiment 12

Aim: Mini Project

Roll No.	33
Name	Rujuta Vinit Medhi
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO4:Develop applications in permissioned Hyperledger Fabric network. CO5:Interpret different Crypto assets and Crypto currencies CO6:Analyze the use of Blockchain with AI, IoT and Cyber Security using case studies.

Sathi

Trust-Based Collateral-Free Micro-Lending Platform

Shravani Patil · Madhura Jangale · Rujuta Medhi · Akruti Dabas

Background & Introduction

What is Sathi?

Sathi is a blockchain-enabled micro-lending platform designed to make loans accessible to people who lack traditional collateral.

By leveraging decentralized trust mechanisms, community lending pools, and automated smart contracts, Sathi bridges the gap between underserved borrowers and cautious lenders.

Domain: Finance · FinTech · Micro-Lending · DeFi

Why This Problem Matters



Unbanked Millions

Millions worldwide lack access to formal banking systems



Collateral Barriers

Asset requirements exclude low-income borrowers entirely



Informal Exploitation

Informal lending leads to predatory interest rates



Financial Inclusion

Inclusion is key to sustainable economic growth

Problem Statement

Core Problem: Traditional micro-lending depends on collateral, preventing asset-poor individuals from accessing credit.

Challenges

- Collateral mandatory for loans
- High loan rejection rates
- High interest for risky borrowers
- Lenders face high default risk
- Lack of trust between parties

Stakeholders

- Borrowers (no assets)
- Lenders (fear defaults)
- Community members
- Platform operators
- Financial regulators

Root Causes

- Trust deficit between parties
- No alternative credit scoring
- Centralized financial control
- Risk on individual lenders
- Exclusion of poor populations

Sathi

Existing System – Traditional Approach

HOW IT WORKS

- 1 Borrower applies for loan
- 2 Collateral is evaluated
- 3 Bank / lender approves or rejects
- 4 Loan issued with fixed interest
- 5 Recovery through legal / asset seizure

LIMITATIONS & ROOT CAUSES

-  Asset-based eligibility only
-  Exclusion of poor populations
-  Lengthy verification process
-  Centralized decision-making
-  Expensive recovery processes
-  No trust-based alternative scoring

Sathi

Why Blockchain?

Type: Ethereum (Public Blockchain) — Solidity Smart Contracts · ERC-20 Stablecoin · ethers.js · MetaMask



Decentralized Trust

No single authority controls lending. Consensus replaces centralized gatekeepers.



Transparent Records

All loan transactions are publicly verifiable on the blockchain ledger.



Tamper-Proof History

Immutable loan repayment history builds a portable financial reputation.



Smart Contracts

Automated loan agreements, interest calculation, and default handling — no manual intervention.



Reduced Fraud

Decentralized verification eliminates manipulation and fraudulent records.



Lender Confidence

Safety Pool + trust scores give lenders quantifiable risk protection.

Sathi

Proposed Solution

Trust Score + Community Lending Pools + Safety Pool Protection



Trust Score System

- Higher score → lower interest rates
- Higher score → higher loan limits
- Built from on-chain behavior
- Repayment history creates reputation
- Replaces centralized credit bureaus



Community Lending Pools

- Friend Lending Pools reduce risk
- Collective lending model
- Risk distributed across community
- More lenders encouraged to join
- Reduces over-collateralization

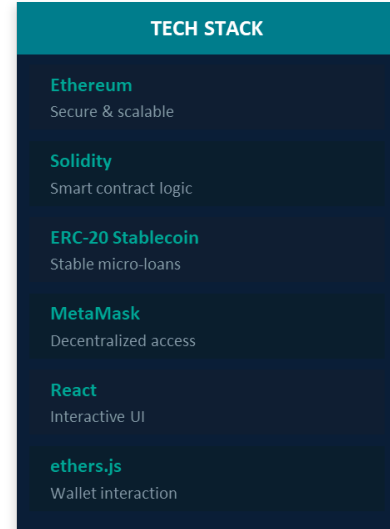
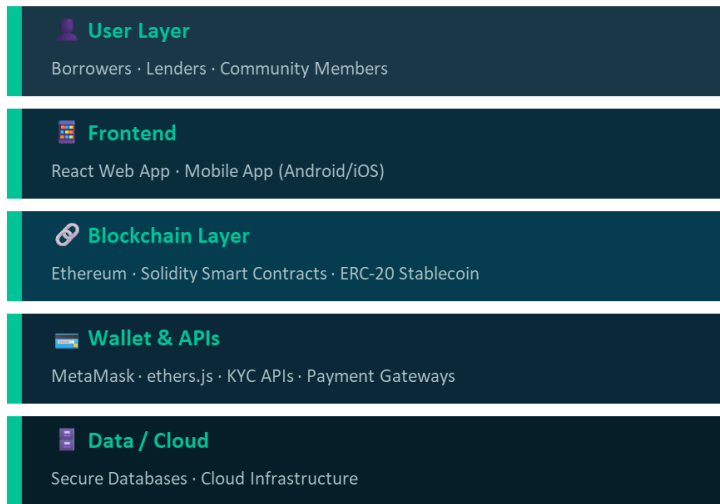


Safety Pool

- 10% of lender profit is saved
- Covers 50% losses on defaults
- Community-backed risk buffer
- Partial compensation on default
- Smart contract enforced

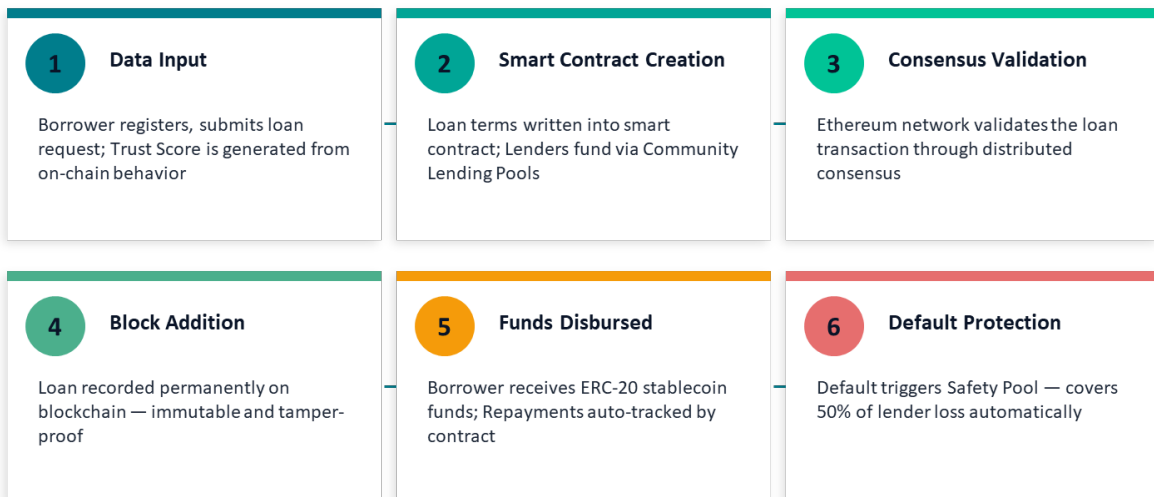
Sathi

Technical Approach



Sathi

Working Mechanism



Sathi

In fact, 6 benefits

Meet Aakash — The Real-World Problem

A freelancer with stable income but no CIBIL score. Banks reject him. DeFi demands 150%+ collateral. Sathi's decentralized trust score and Safety Pool changes that — making lending safer, fairer, and truly inclusive.



Fair Capital Access

First-time borrowers get capital without assets



Portable Reputation

Tamper-proof financial reputation across platforms



Lender Confidence

Safety Pool partially compensates losses on defaults



More Participation

Encourages more lenders to join the ecosystem



Distributed Risk

Risk shared across community, not one lender



Smart Contract Law

Trust score & Safety Pool enforced on-chain

SWOT Analysis

Strengths

- Collateral-free lending model
- Transparent blockchain system
- Reduced default risk via Safety Pool
- Community participation model
- Automated smart contract execution

Weaknesses

- Dependence on digital literacy
- Initial trust score accuracy
- Technology adoption barriers
- Internet dependency in rural areas
- High initial setup cost

Opportunities

- Financial inclusion expansion
- Rural credit market penetration
- Integration with government schemes
- Global microfinance market
- DeFi & digital identity integration

Threats

- Regulatory uncertainty
- Cybersecurity risks
- User resistance to blockchain
- Market competition
- Blockchain scalability challenges

Feasibility & Implementation Plan

Technical Feasibility

- Ethereum + Solidity stack ready
- Smart contract frameworks mature
- Mobile penetration supports adoption

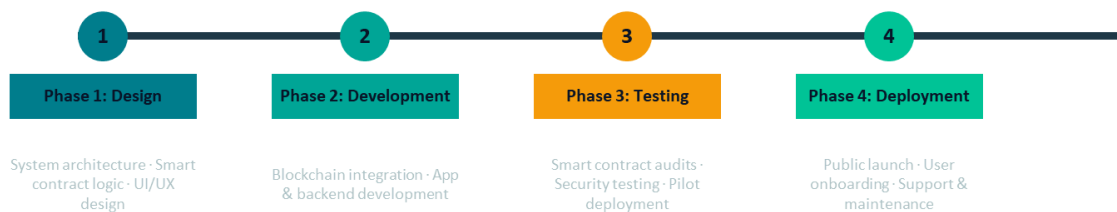
Economic Feasibility

- Lower operational costs
- Reduced recovery expenses
- Shared risk financial model

Scalability Feasibility

- Modular architecture design
- Expandable user base
- Cross-region deployment possible

Implementation Phases



Challenges & Future Scope

⚠️ Challenges & Limitations

-  **High Initial Setup Cost**
Blockchain infrastructure requires significant upfront investment
-  **Technical Complexity**
Smart contract development needs specialized expertise
-  **Regulatory Compliance**
Varying financial regulations across regions
-  **Internet Dependency**
Rural areas with poor connectivity face barriers
-  **Adoption Resistance**
Trust in new technology is slow to build

🚀 Future Enhancements

-  **AI-Based Trust Scoring**
Dynamic scoring using machine learning & behavioral analytics
-  **Mobile Application**
Dedicated Android & iOS app for easy lending, borrowing, and tracking
-  **Insurance-Backed Safety Pool**
Expanded protection with insurance integrations
-  **Cross-Border Lending**
Extend micro-lending across geographies seamlessly
-  **DeFi & Digital Identity**
Integration with decentralized identity and broader DeFi protocols

Sathi

Conclusion

Sathi transforms micro-lending by replacing collateral with trust and community-backed financial security using blockchain technology. It builds a transparent, scalable Web3 credit system designed for real-world financial inclusion.

 Trust replaces collateral via blockchain

 Community risk sharing through lending pools

 Automated smart contract protection



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801: Blockchain and DLT Lab
Experiment 13

Aim: Assignment 1

Roll No.	33
Name	Rujuta Vinit Medhi
Class	D20A
Subject	ITC801: Blockchain and DLT Lab Question : Explore following 4 websites and answer the questions bitcoin.org Blockchain.com https://coinmarketcap.com/ https://www.investopedia.com/
CO Mapped	CO1,CO2

Hard Copy Submitted



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801: Blockchain and DLT Lab
Experiment 14

Aim: Assignment 2

Roll No.	33
Name	Rujuta Vinit Medhi
Class	D20A
Subject	ITC801: Blockchain and DLT Lab Questions ● Draw a neat diagram and explain components of Hyperledger Fabric? ● Draw a neat diagram and explain the transaction flow in Hyperledger Fabric? ● State the requirements of a consensus algorithm. Explain PAXOS and kafka consensus ● Explain ERC 20 token standard and its functions and Compare how ERC 20 token is different than ERC 721 token ● What is Tokenization? Explain its significance and challenges/limitation with example ● Compare Fungible and non Fungible tokens (Give examples of each) ● How can blockchain-based IoT networks improve device management, data monetization, and user privacy in the rapidly growing IoT ecosystem? ● In what ways can blockchain technology enhance transparency, security, and efficiency in government services and record-keeping? Discuss the challenges and benefits of implementing blockchain-based voting systems for elections.
CO Mapped	CO4,CO5,CO6

Hard Copy Submitted
ITC801: Blockchain and DLT Lab



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26

Experiment 15

Aim: Assignment 3

Roll No.	33
Name	Rujuta Vinit Medhi
Class	D20A
Subject	ITC801: Blockchain and DLT Lab Q1 Analyze & compare different consensus mechanisms such as PoW, PoS, Delegated Proof of Stake & PBFT in terms of security, scalability, energy efficiency & fault tolerance. Illustrate your answer with examples like Bitcoin & Ethereum. Q2 Analyze different types of nodes in a blockchain network & explain their role in maintaining the system. Compare how each node contributes to validation, storage & network efficiency. (Compare UTXO model) Q3 Analyze the UTXO model & explain how it ensures transaction transparency & security in blockchain systems. Illustrate with an example such as Bitcoin. Q4 Explain crypto asset & cryptocurrency. Examine & analyze the role of different types of crypto assets (utility tokens, security tokens, crypto coins) in the blockchain ecosystem. Evaluate their impact on digital finance & decentralized applications. Q5 Analyze the role of payment scripts in blockchain transactions. Examine & analyze different types of payment scripts. Explain how they ensure secure & verifiable transfer of coins. Illustrate with an example. Q6 What is a smart contract? Explain lifecycle of smart contracts. Explain advantages, limitations, risks & opportunities of smart contracts. Q7 Explain the structure of BTC transaction & ETH transaction. Q8 Explain the difference between mining & minting and explain the process
CO Mapped	CO1- CO6

Hard Copy Submitted